

Deep Learning for Computer Vision

114-1/Fall 2025

<https://cool.ntu.edu.tw/courses/53063> (NTU COOL)

<http://vllab.ee.ntu.edu.tw/dlcv.html> (Public website)

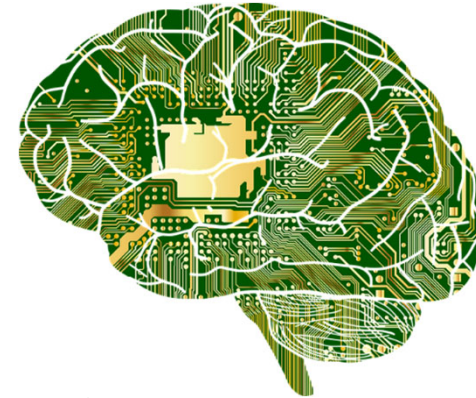
Yu-Chiang Frank Wang 王鈺強, Professor

Dept. Electrical Engineering, National Taiwan University

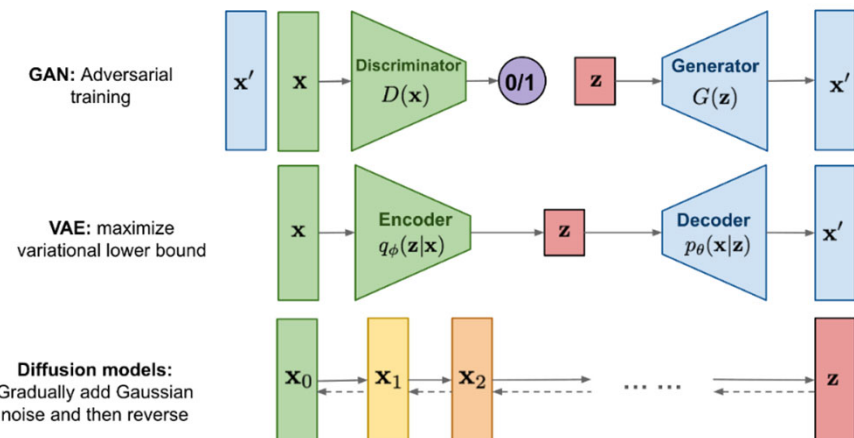
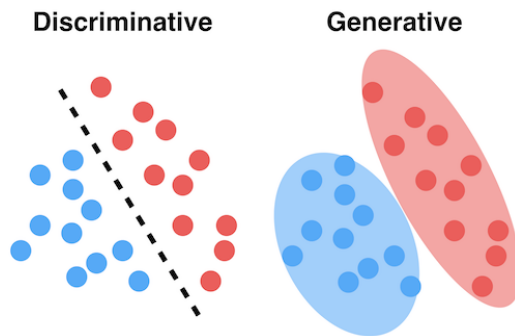
2025/10/01

What's to Be Covered Today...

- Generative Models
 - Diffusion Model
 - Generative Adversarial Network
- Further Updates on Final Projects (by **PicCollage**)
 - Possible reschedule to 12/19 Fri (instead of 12/23 Tue)
 - Company visits (with capacity limitation)
 - Top teams w/ prizes & company sharing



PIC•COLLAGE



Discriminative vs. Generative Models (cont'd)

Discriminative Model:

Learn a probability distribution $p(y|x)$

Generative Model:

Learn a probability distribution $p(x)$

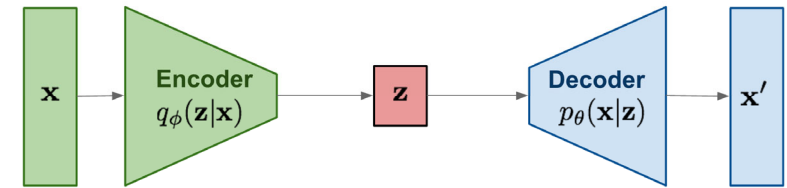
Conditional Generative Model: Learn $p(x|y)$

Recall **Bayes' Rule**:

$$\underbrace{P(x | y)}_{\text{Conditional Generative Model}} = \frac{\underbrace{P(y | x)}_{\text{Discriminative Model}} \underbrace{P(x)}_{\text{(Unconditional) Generative Model}}}{\underbrace{P(y)}_{\text{Prior over labels}}}$$

We can build a conditional generative model from other components!

Recap: VAE



- Recall that, we aim to maximize

$$\Rightarrow p_{\theta}(x) = \int p_{\theta}(x, z) dz = \int p_{\theta}(x|z)p_{\theta}(z) dz$$

we have...

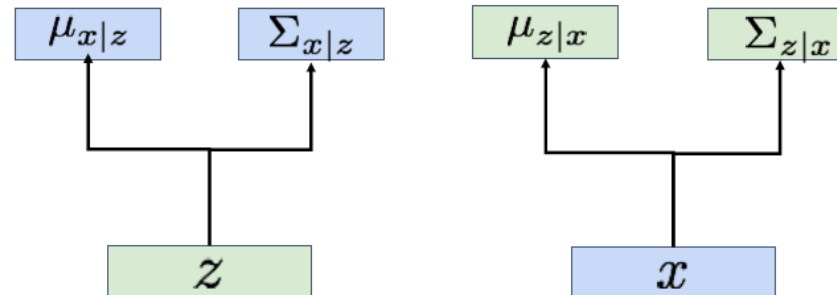
Decoder network inputs

latent code z , gives
distribution over data x

Encoder network inputs

data x , gives distribution
over latent codes z

$$p_{\theta}(x | z) = N(\mu_{x|z}, \Sigma_{x|z}) \quad q_{\phi}(z | x) = N(\mu_{z|x}, \Sigma_{z|x})$$

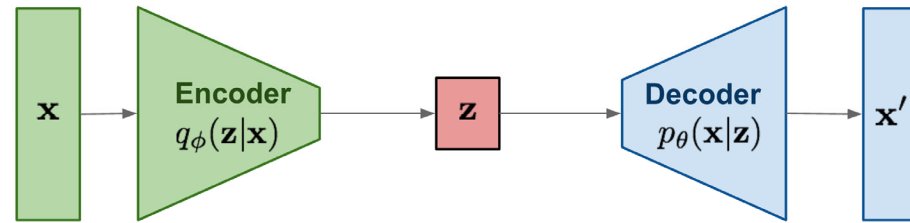


- Via training, if we ensure $q_{\phi}(z | x) \approx p_{\theta}(z | x)$

then we have

$$p_{\theta}(x) = \frac{p_{\theta}(x | z)p_{\theta}(z)}{p_{\theta}(z | x)} \approx \frac{p_{\theta}(x | z)p(z)}{q_{\phi}(z | x)}$$

Training VAE



$$\log p_{\theta}(x) = \log \frac{p_{\theta}(x | z)p(z)}{p_{\theta}(z | x)} = \log \frac{p_{\theta}(x|z)p(z)q_{\phi}(z|x)}{p_{\theta}(z|x)q_{\phi}(z|x)}$$

$$= E_z [\log p_{\theta}(x|z)] - E_z \left[\log \frac{q_{\phi}(z|x)}{p(z)} \right] + E_z \left[\log \frac{q_{\phi}(z|x)}{p_{\theta}(z|x)} \right]$$

$$= E_{z \sim q_{\phi}(z|x)} [\log p_{\theta}(x|z)] - D_{KL} \left(q_{\phi}(z|x), p(z) \right) + D_{KL} (q_{\phi}(z|x), p_{\theta}(z|x))$$

Data reconstruction

KL divergence

between sample
distribution from the
encoder and the prior

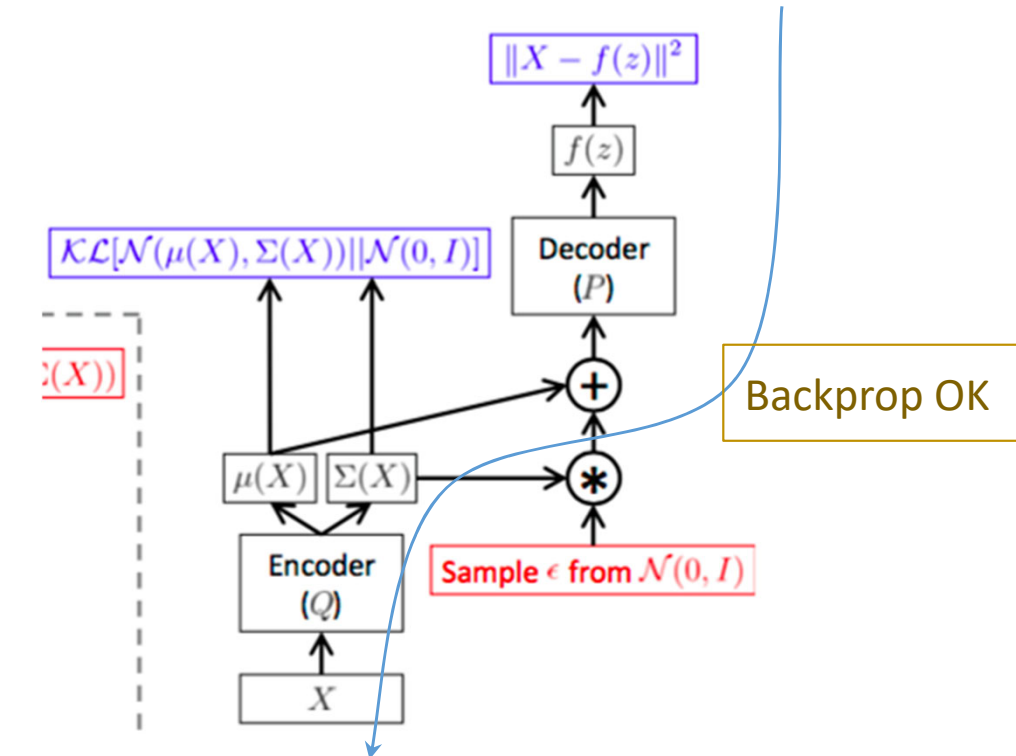
KL divergence between

sample distribution
from the encoder and
the posterior of data

$$\Rightarrow \log p_{\theta}(x) \geq \underbrace{E_{z \sim q_{\phi}(z|x)} [\log p_{\theta}(x|z)]}_{(1)} - \underbrace{D_{KL} \left(q_{\phi}(z|x), p(z) \right)}_{(2)}$$

which is the **variational lower bound**, aka **evidence lower bound (ELBO)**,
on the data likelihood $p_{\theta}(x)$

Implementation of VAE Training



Initialize parameters of encoder and decoder
Repeat:

Get mini-batch of X
 $\mathbf{mu_X}, \mathbf{var_X} = \text{encoder}(X)$

$\epsilon = \text{sampling from Normal}(0, I)$

$z = \mathbf{mu_X} + \epsilon * \mathbf{var_X}$

$X' = \text{decoder}(z)$

$\text{recon_loss} = \text{MSE}(X, X')$

$\text{latent_loss} =$

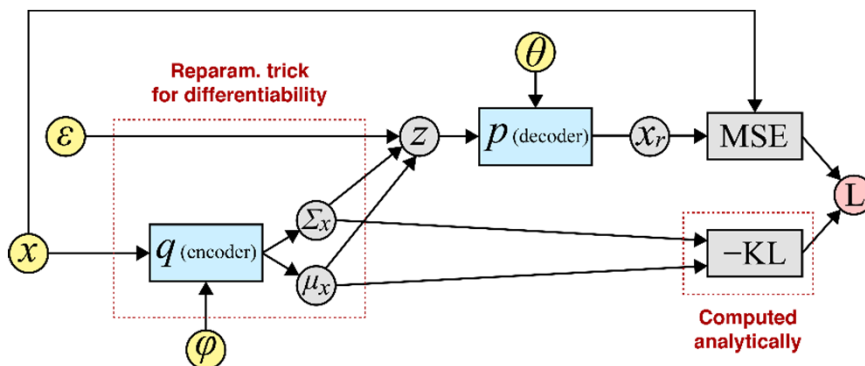
$\text{KLD}(\text{Normal}(\mathbf{mu_X}, \mathbf{var_X}) || \text{Normal}(0, I))$

$\text{all_loss} = \text{recon_loss} + \text{latent_loss}$

$\text{all_loss.backward}()$

Until: parameters of **encoder** & **decoder** converge

Return parameters of **encoder** and **decoder**



1. Reparameterization trick:

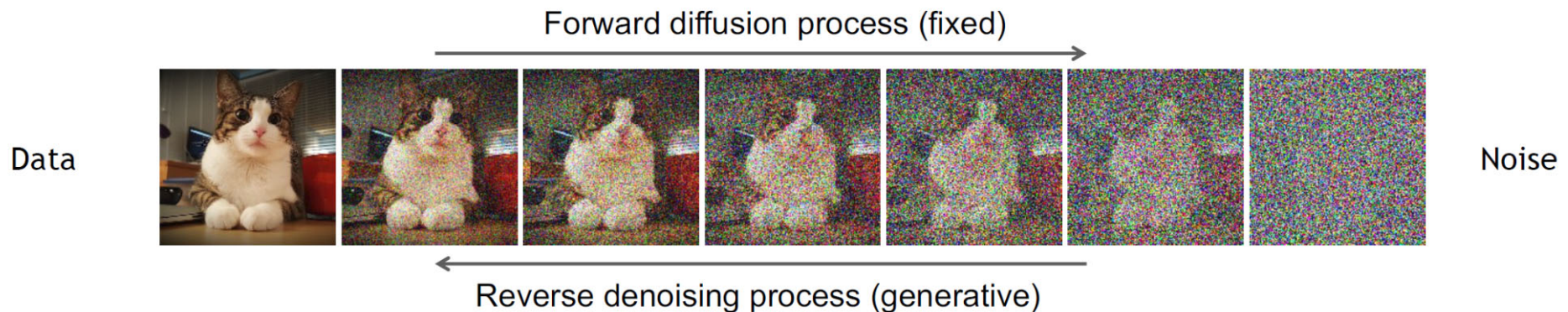
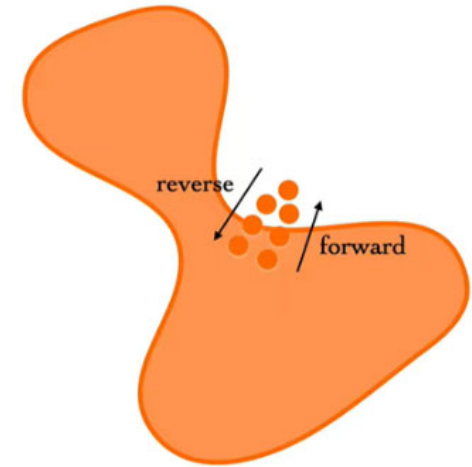
First sample noise ϵ from $\text{Normal}(0, I)$, then reparameterize z by $\mu_x + \epsilon * \text{var}_x$. The model is now differentiable!

2. Training techniques...

Recap: Denoising Diffusion Probabilistic Models (DDPM)

Learning to generate by denoising

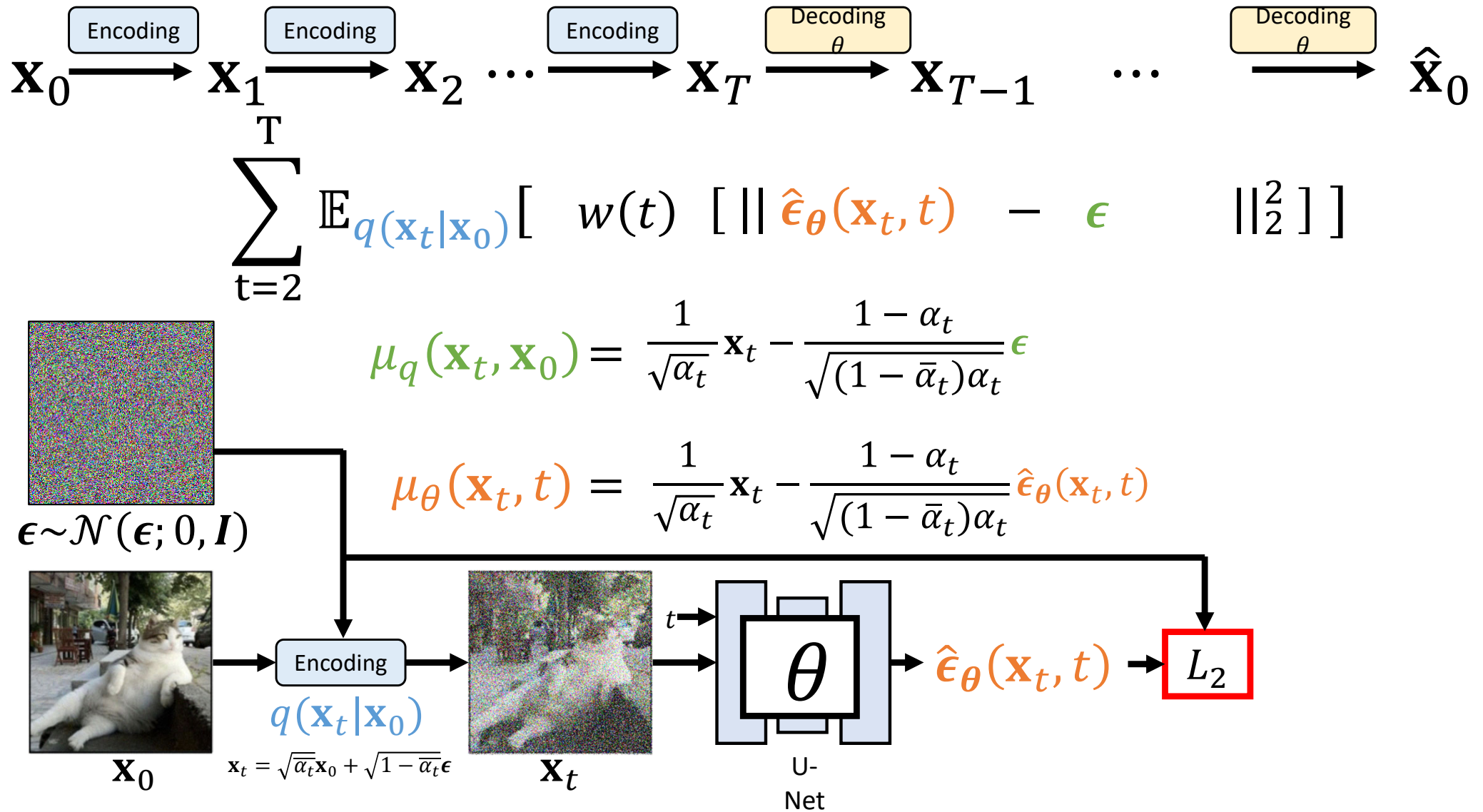
- 2 processes required for training:
 - **Forward diffusion process**
 - gradually add noise to input
 - **Reverse diffusion process**
 - learns to generate/restore data by denoising
 - typically implemented via a **conditional U-net**



[Ho et al., Denoising Diffusion Probabilistic Models, NeurIPS 2020](#)

[Song et al., Score-Based Generative Modeling through Stochastic Differential Equations, ICLR 2021](#)

Observation #2



Training vs. Inference

- Summary

Algorithm 1 Training

```

1: repeat
2:    $\mathbf{x}_0 \sim q(\mathbf{x}_0)$ 
3:    $t \sim \text{Uniform}(\{1, \dots, T\})$ 
4:    $\epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ 
5:   Take gradient descent step on
      $\nabla_{\theta} \|\epsilon - \epsilon_{\theta}(\sqrt{\alpha_t} \mathbf{x}_0 + \sqrt{1 - \alpha_t} \epsilon, t)\|^2$ 
6: until converged
    
```

$\mathbf{x}_t$

Algorithm 2 Sampling

```

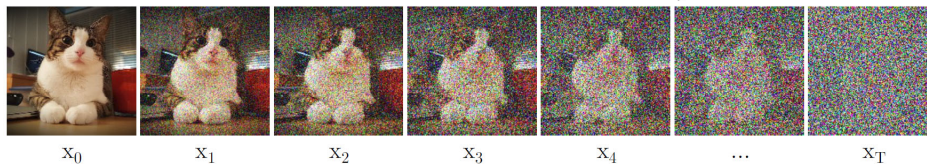
1:  $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ 
2: for  $t = T, \dots, 1$  do
3:    $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ 
4:    $\mathbf{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left( \mathbf{x}_t - \frac{1 - \alpha_t}{\sqrt{1 - \alpha_t}} \epsilon_{\theta}(\mathbf{x}_t, t) \right) + \sigma_t \mathbf{z}$ 
5: end for
6: return  $\mathbf{x}_0$ 
    
```

allowing
generation
diversity

$$\mathbf{x}_t \xrightarrow{\text{model}} \epsilon_{\theta}(\mathbf{x}_t, t) \xrightarrow{P(\mathbf{x}_t|\mathbf{x}_0) \rightarrow P(\mathbf{x}_0|\mathbf{x}_t, \epsilon_{\theta})} \hat{\mathbf{x}}_0(\mathbf{x}_t, \epsilon_{\theta}) \longrightarrow \mu(\mathbf{x}_t, \hat{\mathbf{x}}_0), \beta_t \xrightarrow{P(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0)} \hat{\mathbf{x}}_{t-1}$$

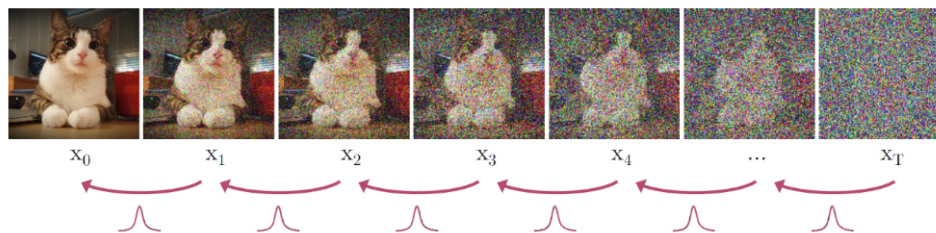
Forward diffusion process (fixed)

Data

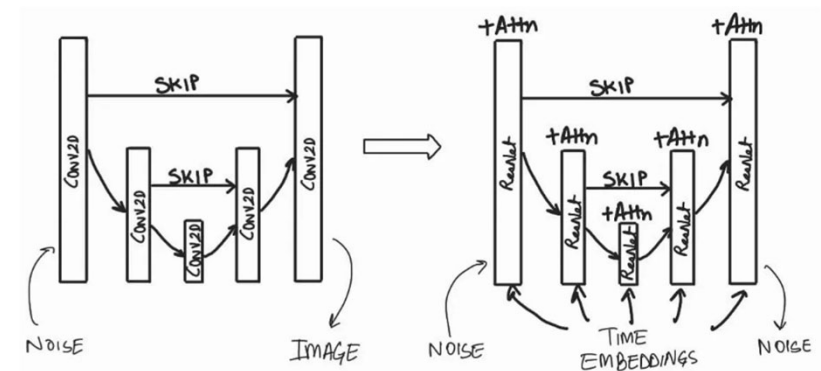


Reverse denoising process (generative)

Data



Noise



Noise

<https://medium.com/@vedantjumble/class-conditioned-diffusion-models-using-keras-and-tensorflow-9997fa6d958c>

Training vs. Inference

- Summary

Algorithm 1 Training

```

1: repeat
2:    $\mathbf{x}_0 \sim q(\mathbf{x}_0)$ 
3:    $t \sim \text{Uniform}(\{1, \dots, T\})$ 
4:    $\epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ 
5:   Take gradient descent step on
        $\nabla_{\theta} \|\epsilon - \epsilon_{\theta}(\sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon, t)\|^2$ 
6: until converged
    
```

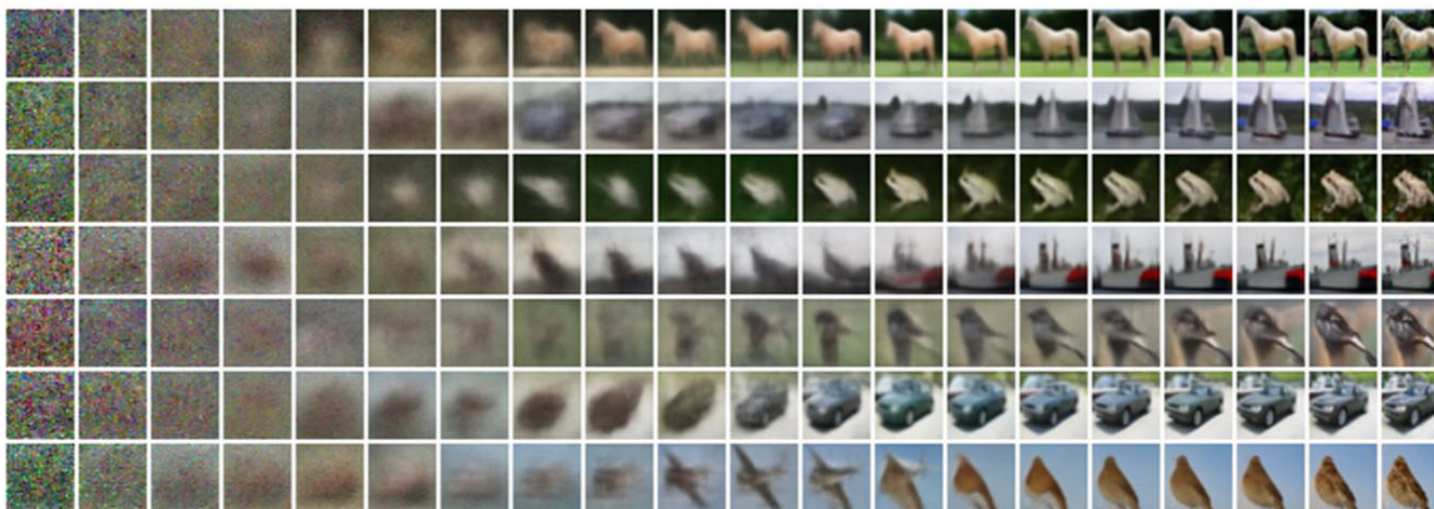
$\mathbf{x}_t$

Algorithm 2 Sampling

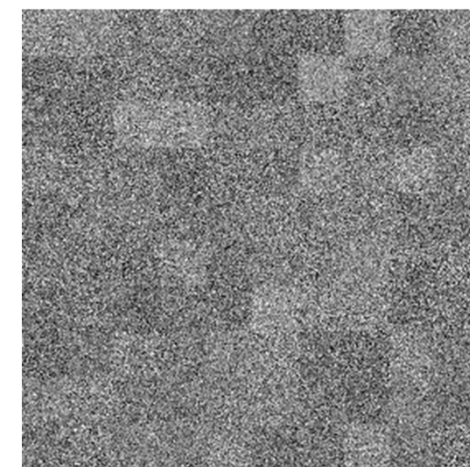
```

1:  $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ 
2: for  $t = T, \dots, 1$  do
3:    $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ 
4:    $\mathbf{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left( \mathbf{x}_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_{\theta}(\mathbf{x}_t, t) \right) + \sigma_t \mathbf{z}$ 
5: end for
6: return  $\mathbf{x}_0$ 
    
```

allowing
generation
diversity



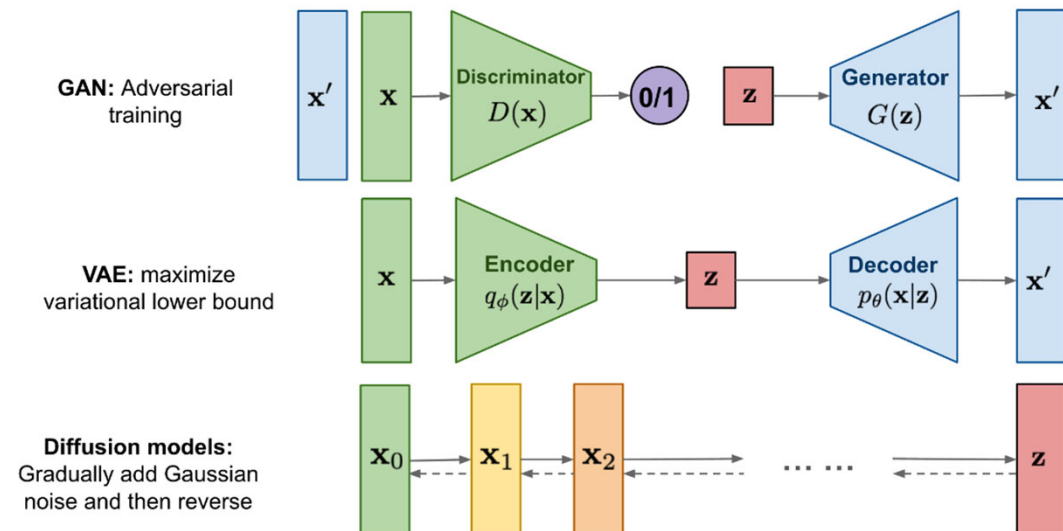
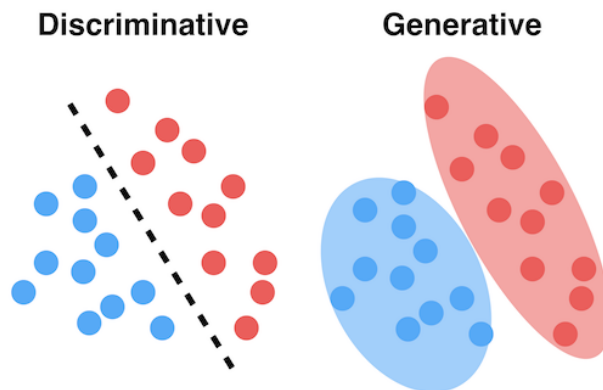
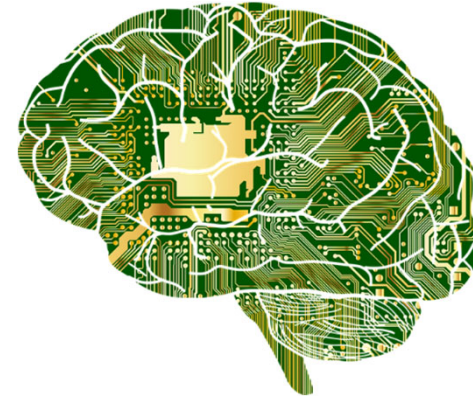
More steps

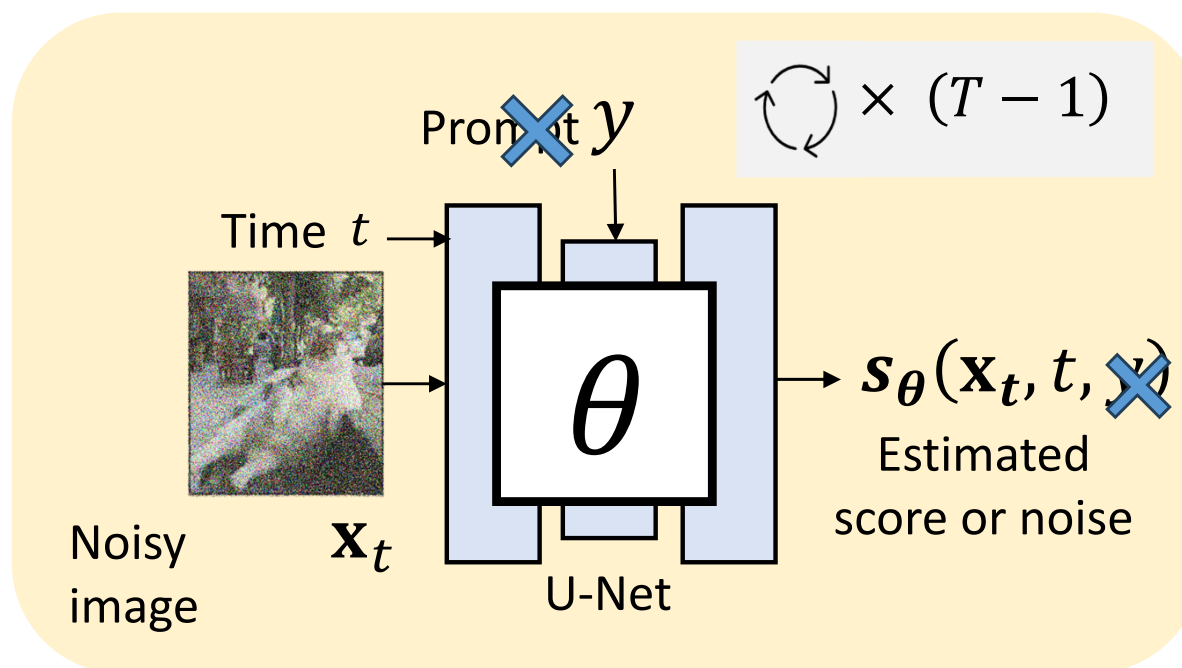


MNIST handwritten
image data

What's to Be Covered Today...

- Generative Models
 - Diffusion Model
 - Denoising Diffusion Probabilistic Model (DDPM)
 - Latent Diffusion Model
 - Denoising Diffusion Implicit Model (DDIM)

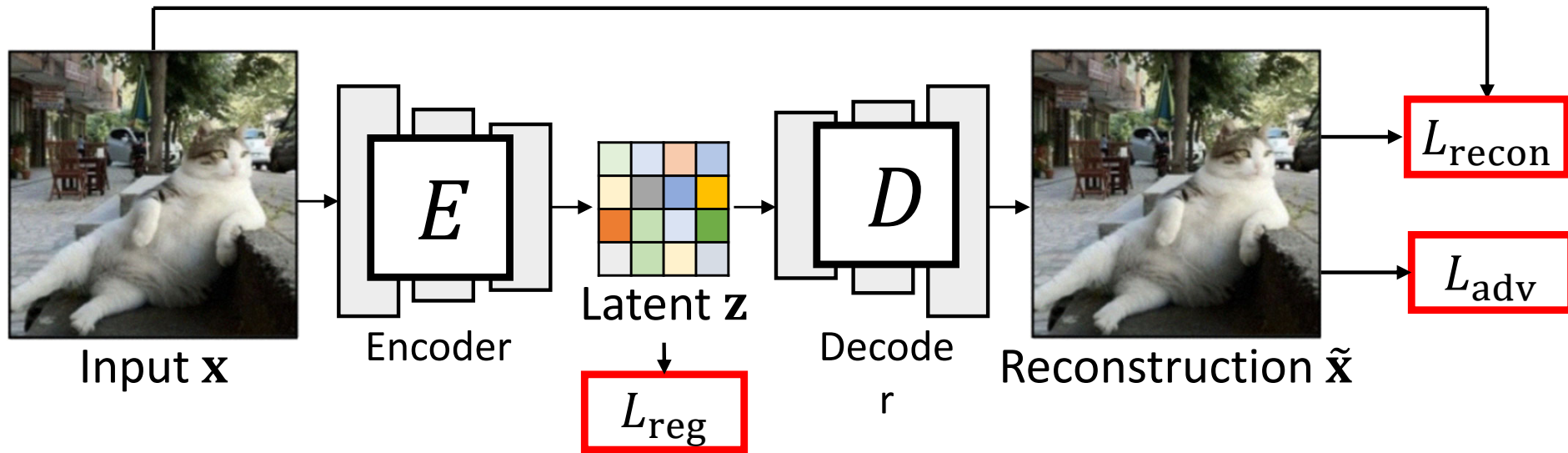




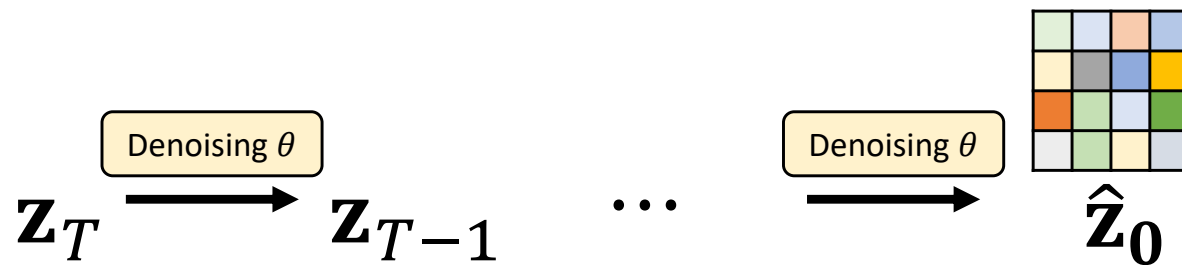
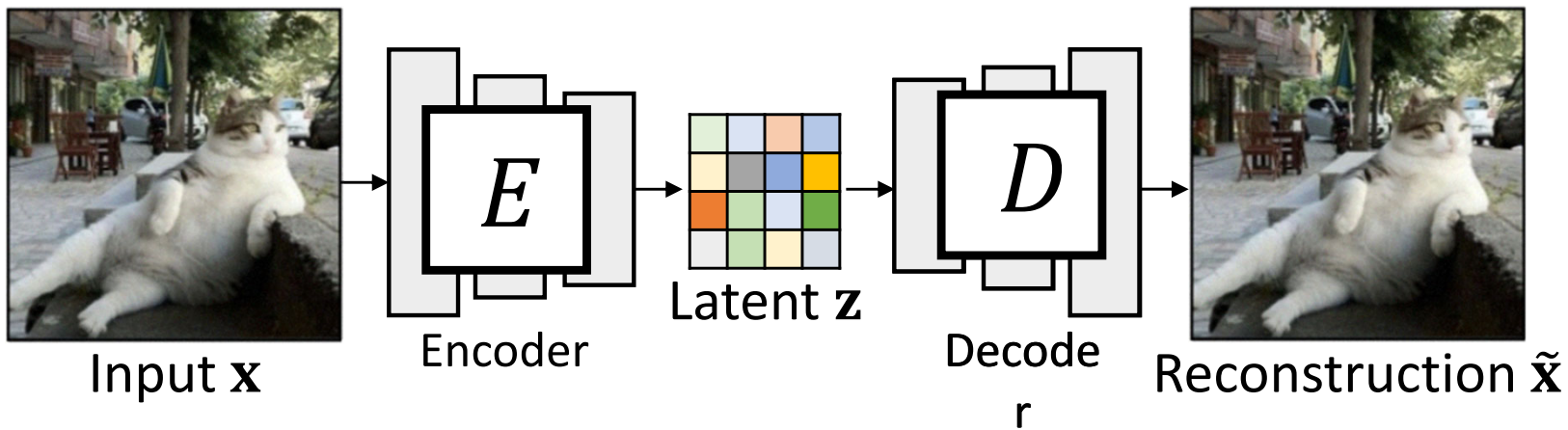
Any concern?



Latent Diffusion Models

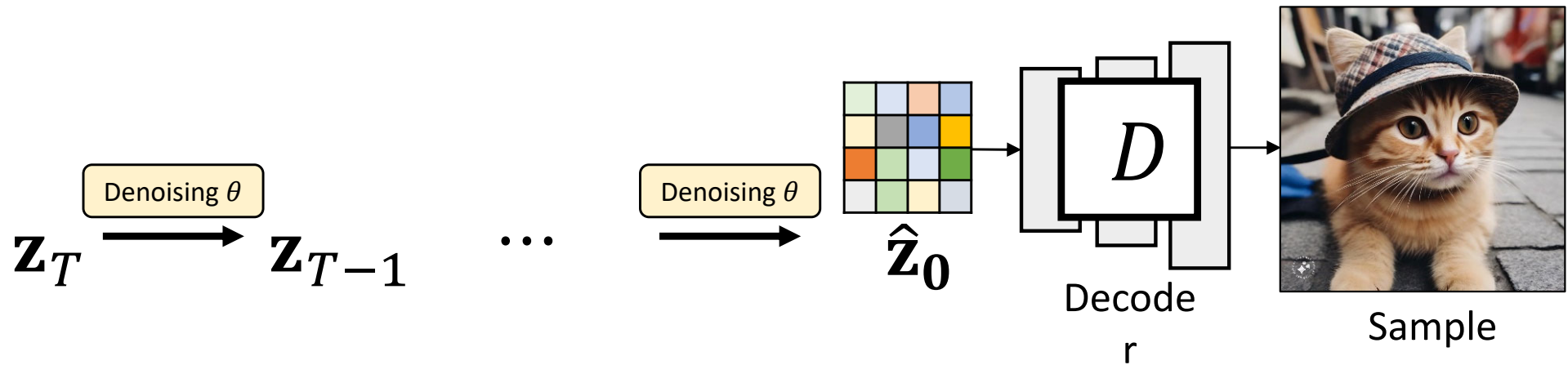
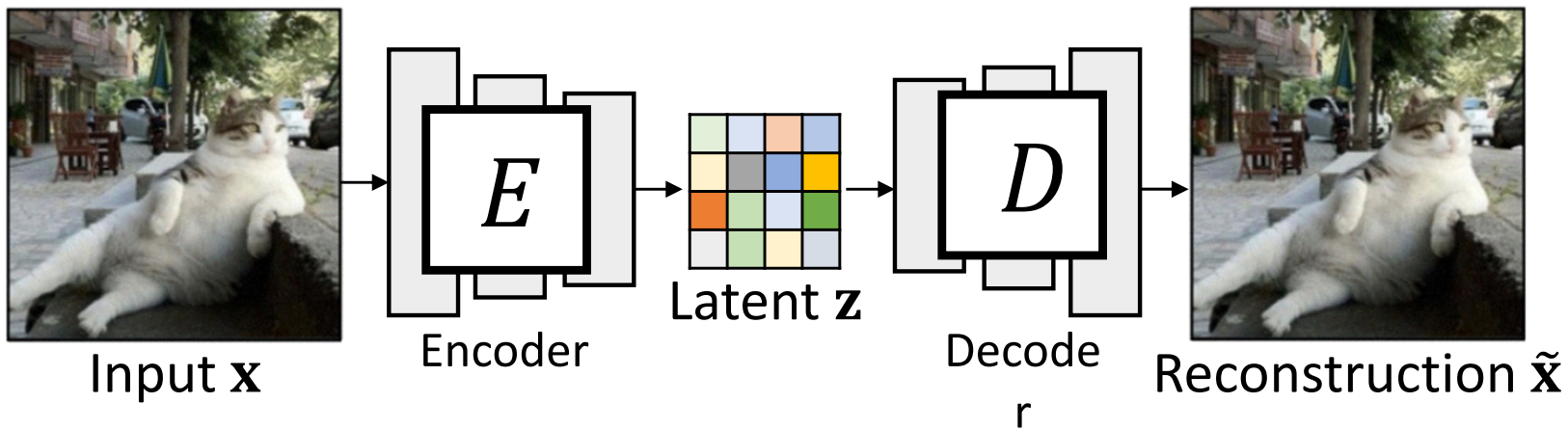


Latent Diffusion Models



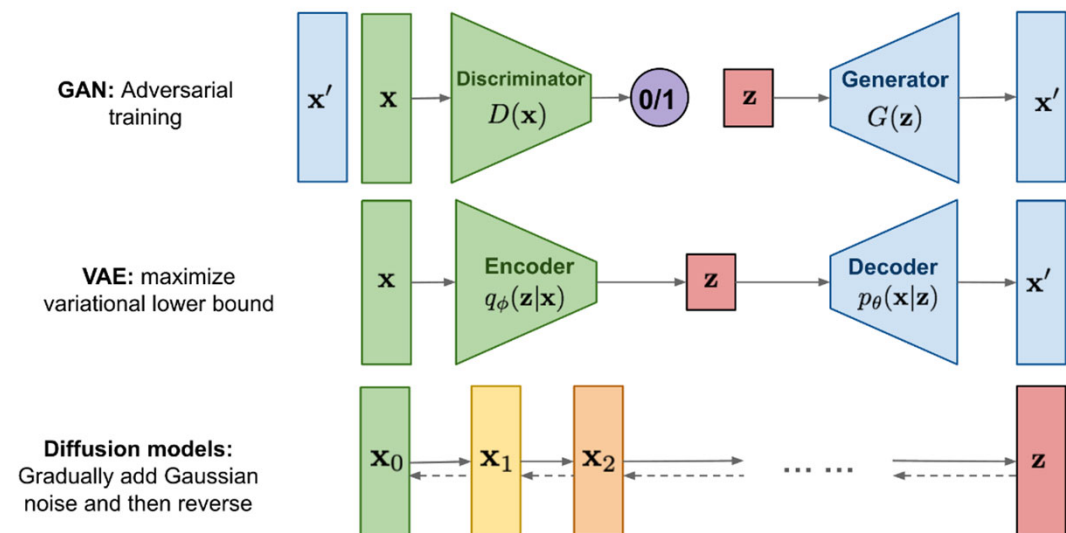
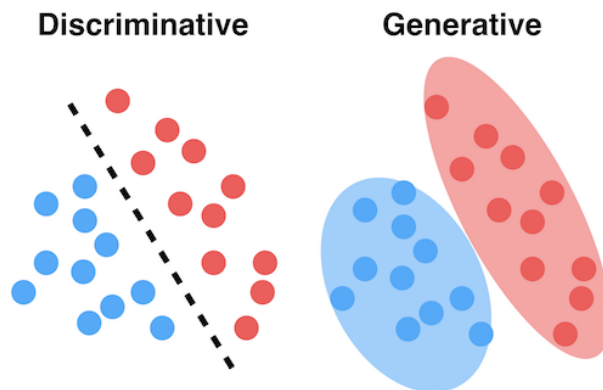
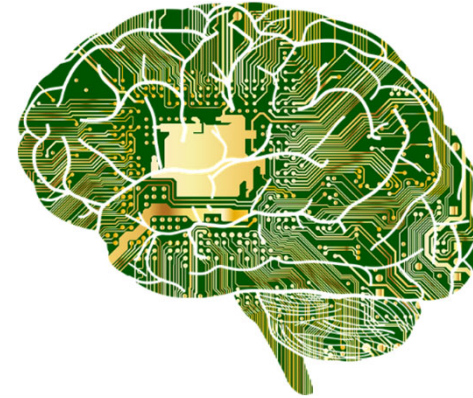


Latent Diffusion Models



What's to Be Covered Today...

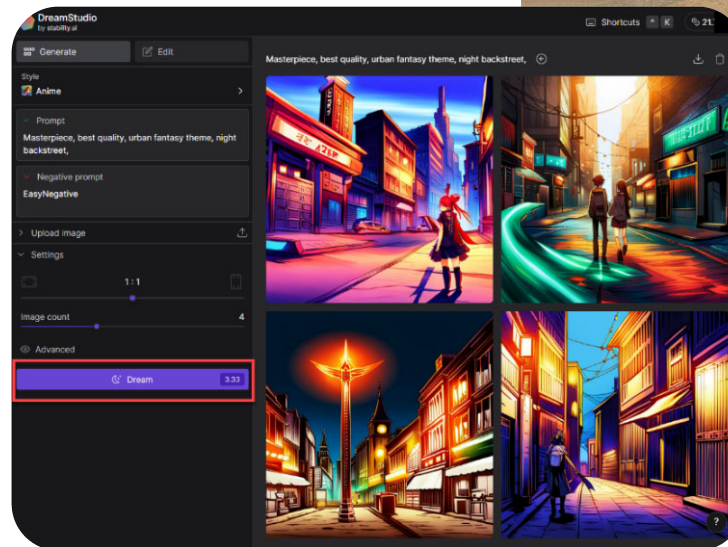
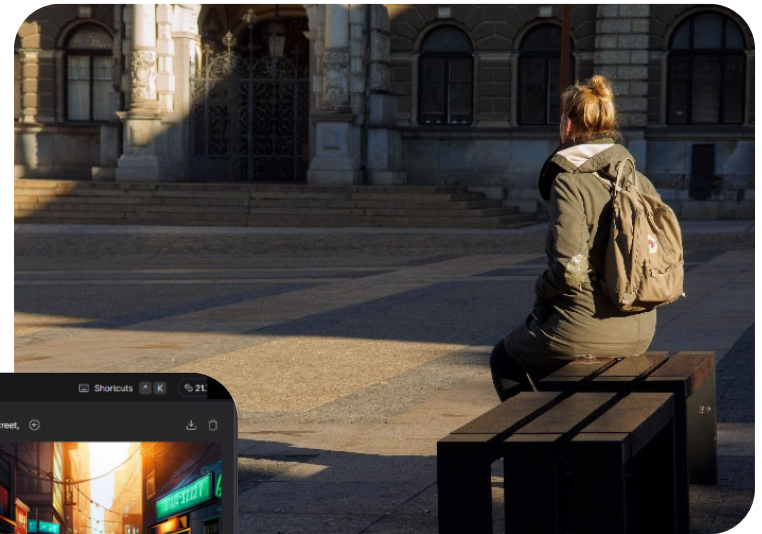
- Generative Models
 - Diffusion Model
 - Denoising Diffusion Probabilistic Model (DDPM)
 - Latent Diffusion Model
 - Denoising Diffusion Implicit Model (DDIM)



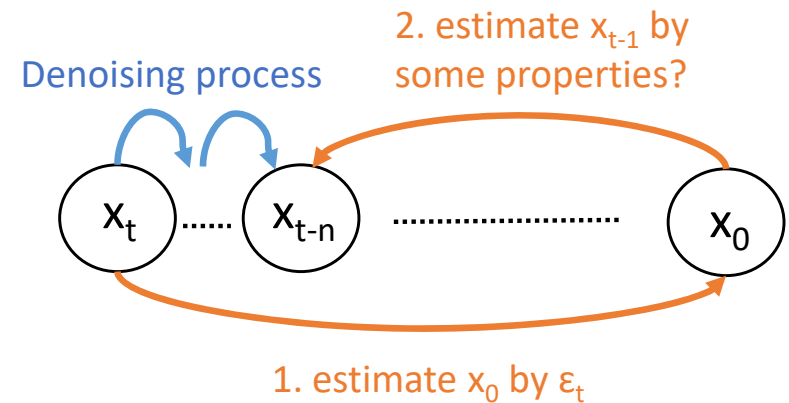
From DDPM to DDIM:

Denoising Diffusion Implicit Models (Song *et al.*, ICLR'21)

- Song et al. 2020:
"It takes around 20 hours to sample 50k images of size 32×32 pixels from a DDPM, but less than a minute to do so from a GAN on an Nvidia 2080 Ti GPU."
- Motivation/Goals
 - Accelerating sampling process
 - A deterministic process
 - Inversion
 - Manipulation
 - Interpolation
 - ...
- Applications
 - Stable Diffusion



From DDPM to DDIM: Denoising Diffusion Implicit Models



- DDPM
 - Pros
 - High quality image generation without adversarial training
 - Cons
 - Require simulating a **Markov chain** for **many steps** in order to produce a sample. (i.e., x_t relies on x_{t-1})
- DDIM

3.1 NON-MARKOVIAN FORWARD PROCESSES

Let us consider a family $\mathcal{Q}$ of inference distributions, indexed by a real vector $\sigma \in \mathbb{R}_{\geq 0}^T$:

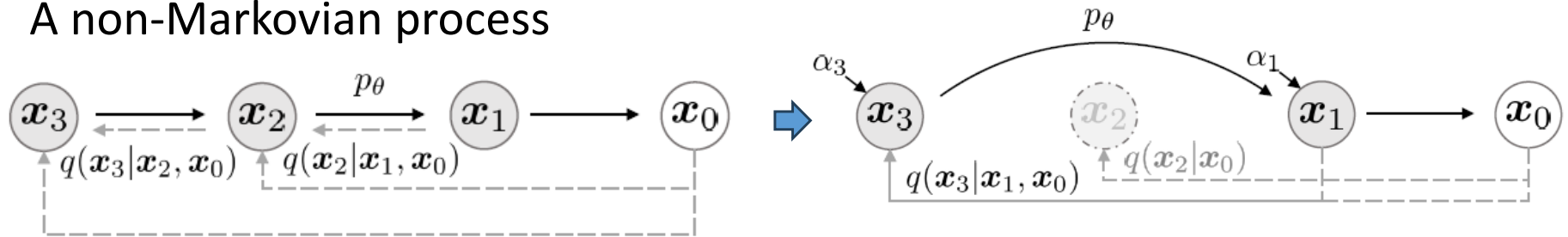
$$q_{\sigma}(\mathbf{x}_{1:T}|\mathbf{x}_0) := q_{\sigma}(\mathbf{x}_T|\mathbf{x}_0) \prod_{t=2}^T q_{\sigma}(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0) \quad (6)$$

where $q_{\sigma}(\mathbf{x}_T|\mathbf{x}_0) = \mathcal{N}(\sqrt{\alpha_T}\mathbf{x}_0, (1 - \alpha_T)\mathbf{I})$ and for all $t > 1$,

$$q_{\sigma}(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0) = \mathcal{N}\left(\sqrt{\alpha_{t-1}}\mathbf{x}_0 + \sqrt{1 - \alpha_{t-1} - \sigma_t^2} \cdot \frac{\mathbf{x}_t - \sqrt{\alpha_t}\mathbf{x}_0}{\sqrt{1 - \alpha_t}}, \sigma_t^2 \mathbf{I}\right). \quad (7)$$

From DDPM to DDIM: Denoising Diffusion Implicit Models

- DDIM
 - A non-Markovian process



- Non-Markovian forward process

$$q_{\sigma}(\mathbf{x}_{1:T}|\mathbf{x}_0) := q_{\sigma}(\mathbf{x}_T|\mathbf{x}_0) \prod_{t=2}^T q_{\sigma}(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0)$$

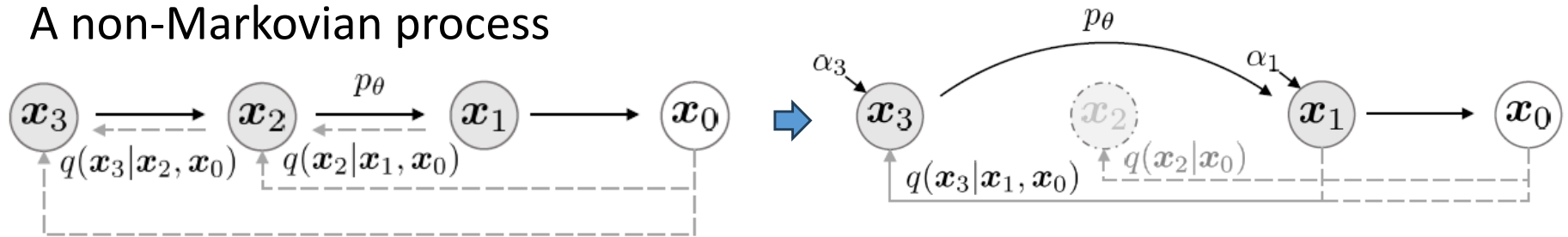
$$\mathbf{x}_t = \sqrt{\alpha_t} \mathbf{x}_0 + \sqrt{1 - \alpha_t} \epsilon, \quad \text{where } \epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$$

$$q_{\sigma}(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0) = \mathcal{N} \left(\sqrt{\alpha_{t-1}} \mathbf{x}_0 + \sqrt{1 - \alpha_{t-1} - \sigma_t^2} \cdot \frac{\mathbf{x}_t - \sqrt{\alpha_t} \mathbf{x}_0}{\sqrt{1 - \alpha_t}}, \sigma_t^2 \mathbf{I} \right)$$

From DDPM to DDIM: Denoising Diffusion Implicit Models

- DDIM

- A non-Markovian process



- Generative (denoising) process

From the
forward process

$$\mathbf{x}_t = \sqrt{\alpha_t} \mathbf{x}_0 + \sqrt{1 - \alpha_t} \epsilon, \quad \text{where } \epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$$

Denoised
observation

$$f_{\theta}^{(t)}(\mathbf{x}_t) := (\mathbf{x}_t - \sqrt{1 - \alpha_t} \cdot \epsilon_{\theta}^{(t)}(\mathbf{x}_t)) / \sqrt{\alpha_t}$$

$$\Rightarrow p_{\theta}^{(t)}(\mathbf{x}_{t-1} | \mathbf{x}_t) = \begin{cases} \mathcal{N}(f_{\theta}^{(1)}(\mathbf{x}_1), \sigma_1^2 \mathbf{I}) & \text{if } t = 1 \\ q_{\sigma}(\mathbf{x}_{t-1} | \mathbf{x}_t, f_{\theta}^{(t)}(\mathbf{x}_t)) & \text{otherwise} \end{cases}$$

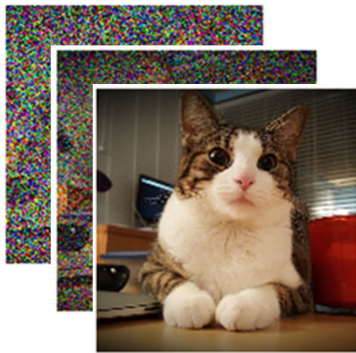
- The above forward/denoising processes results in **the same training objectives** as those of DDPM (see DDIM paper)

From DDPM to DDIM:

Denoising Diffusion Implicit Models

- DDIM
 - Sampling process for generation

$$\mathbf{x}_{t-1} = \underbrace{\sqrt{\alpha_{t-1}} \left(\frac{\mathbf{x}_t - \sqrt{1 - \alpha_t} \epsilon_{\theta}^{(t)}(\mathbf{x}_t)}{\sqrt{\alpha_t}} \right)}_{\text{"predicted } \mathbf{x}_0"} + \underbrace{\sqrt{1 - \alpha_{t-1} - \sigma_t^2} \cdot \epsilon_{\theta}^{(t)}(\mathbf{x}_t)}_{\text{"direction pointing to } \mathbf{x}_t"} + \underbrace{\sigma_t \epsilon_t}_{\text{random noise}}$$



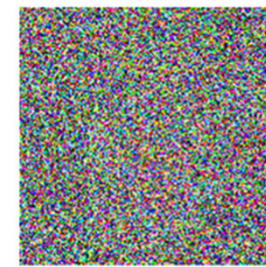
Step 0



Step 123



Step 456



Step 999

- Additional comment on σ_t^2 : *stochastic* vs. *deterministic* generation process
- Since DDIM and DDPM share the same objective function, so one can use a **pretrained** DDPM for DDIM generation.

Image quality in terms of
the **FID (Fréchet inception distance)** score:

	CIFAR10 (32×32)					CelebA (64×64)				
	10	20	50	100	1000	10	20	50	100	1000
DDIM	13.36	6.84	4.67	4.16	4.04	17.33	13.73	9.17	6.53	3.51
DDPM	367.43	133.37	32.72	9.99	3.17	299.71	183.83	71.71	45.20	3.26

DDIM [Song et al. 2021]

From DDPM to DDIM

Example results



We haven't talked about some practical issues...
e.g., resolution, guidance, etc.

Figure 5: Samples from DDIM with the same random x_T and different number of steps.

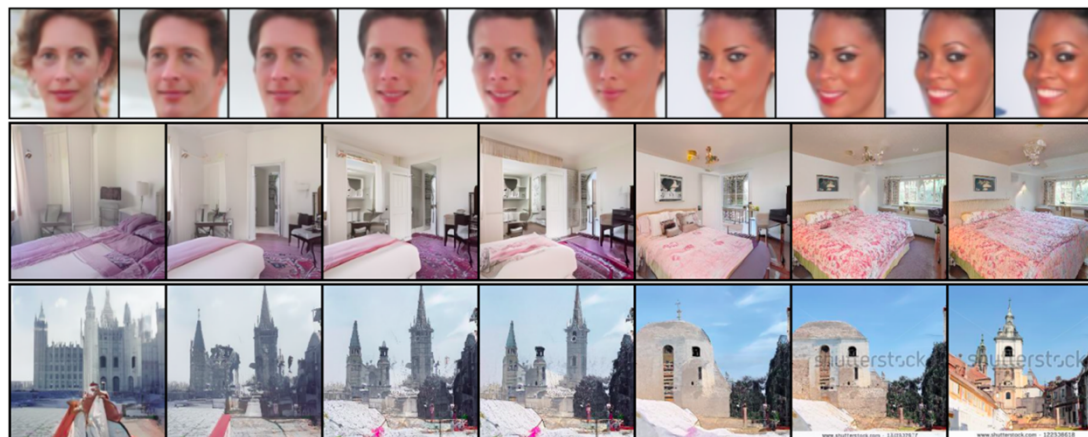
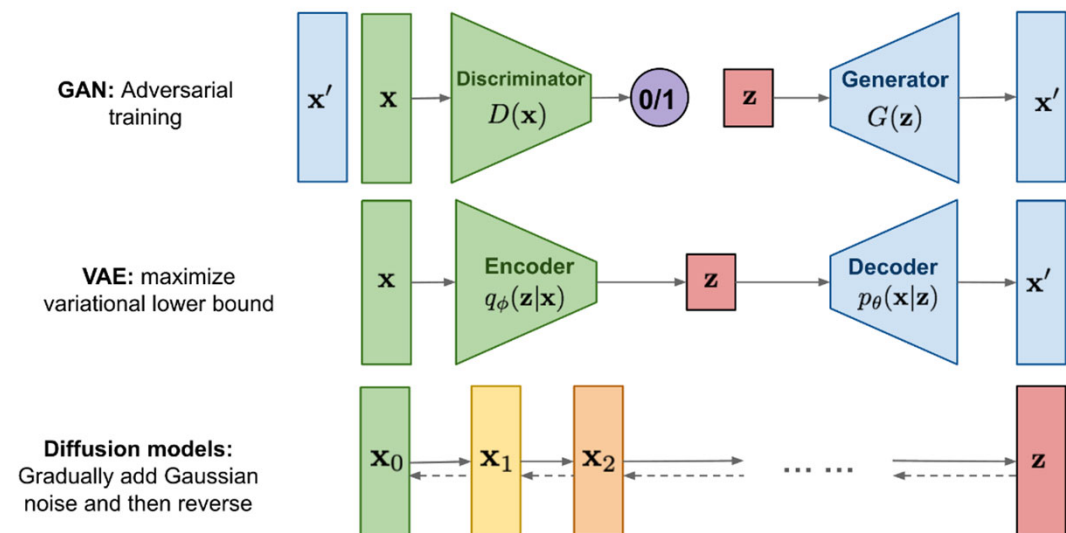
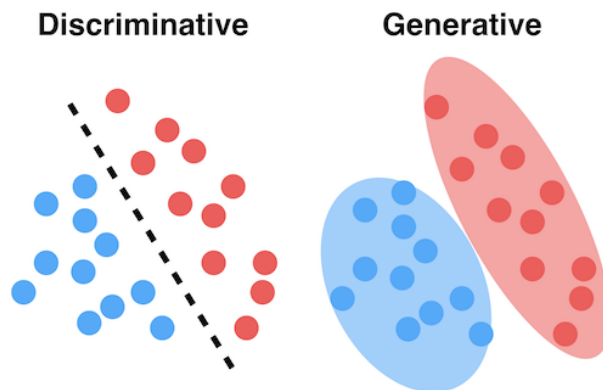
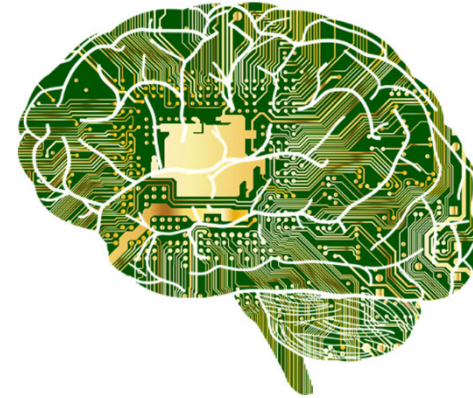


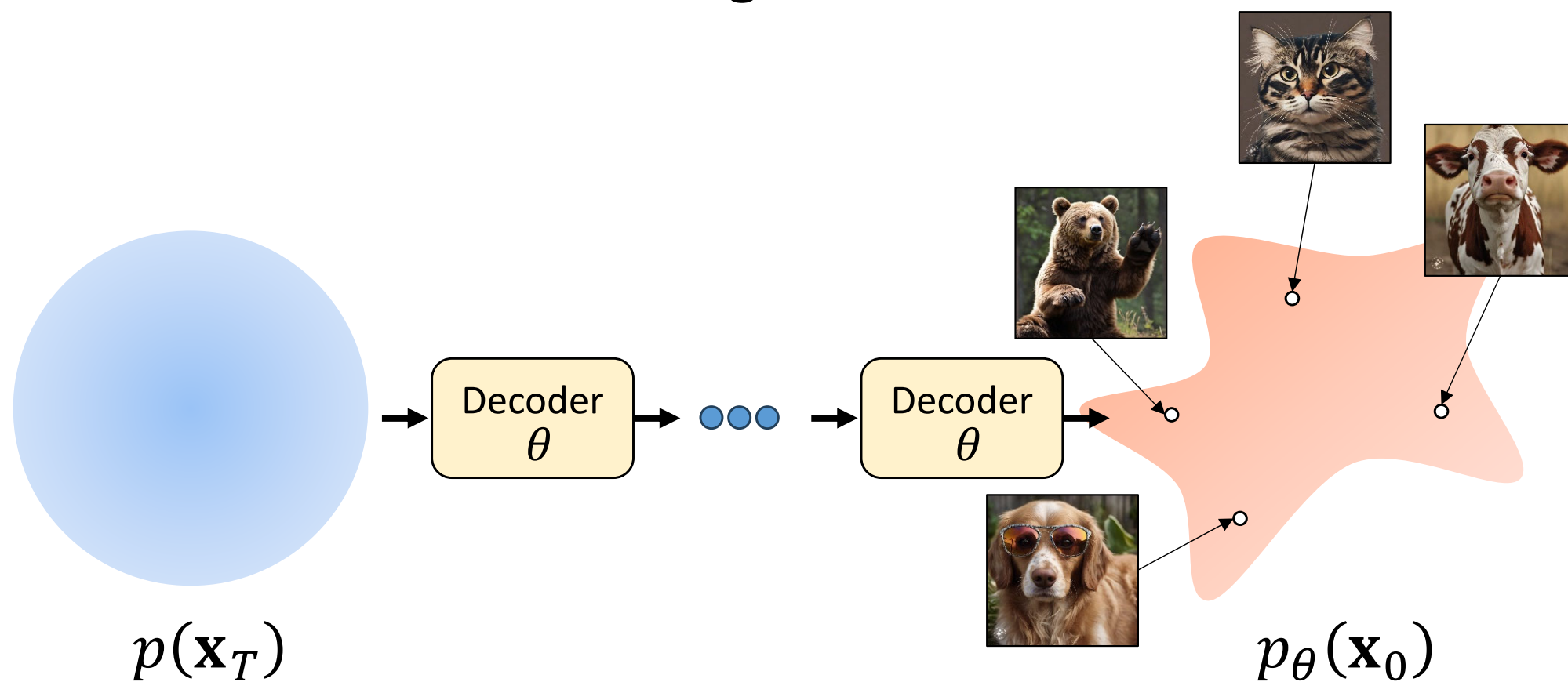
Figure 6: Interpolation of samples from DDIM with $\dim(\tau) = 50$.

What's to Be Covered Today...

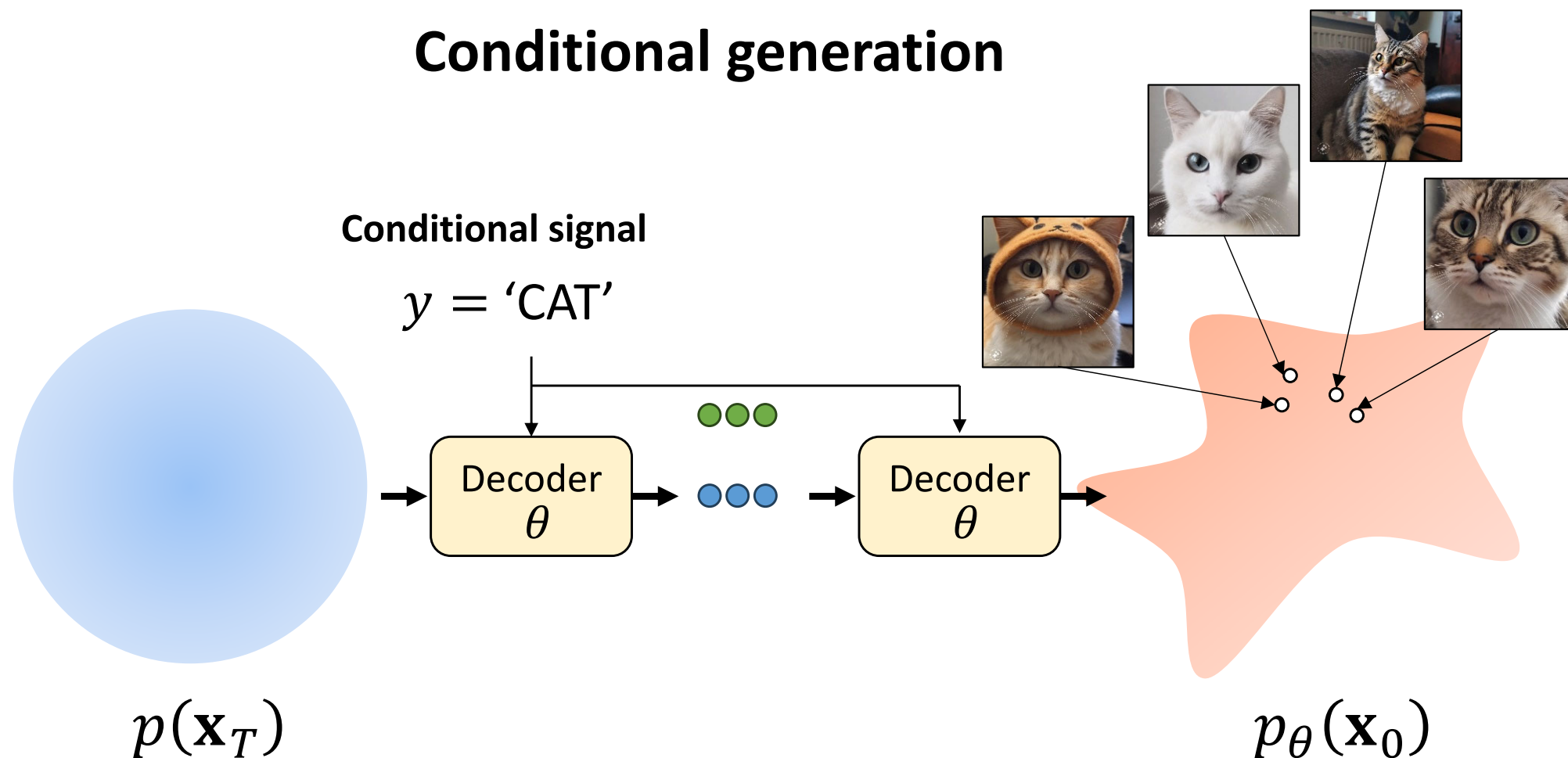
- Generative Models
 - Diffusion Model
 - Conditional Diffusion Model
 - Classifier Guidance
 - Personalization via Diffusion Model
 - Generative Adversarial Network



Unconditional generation



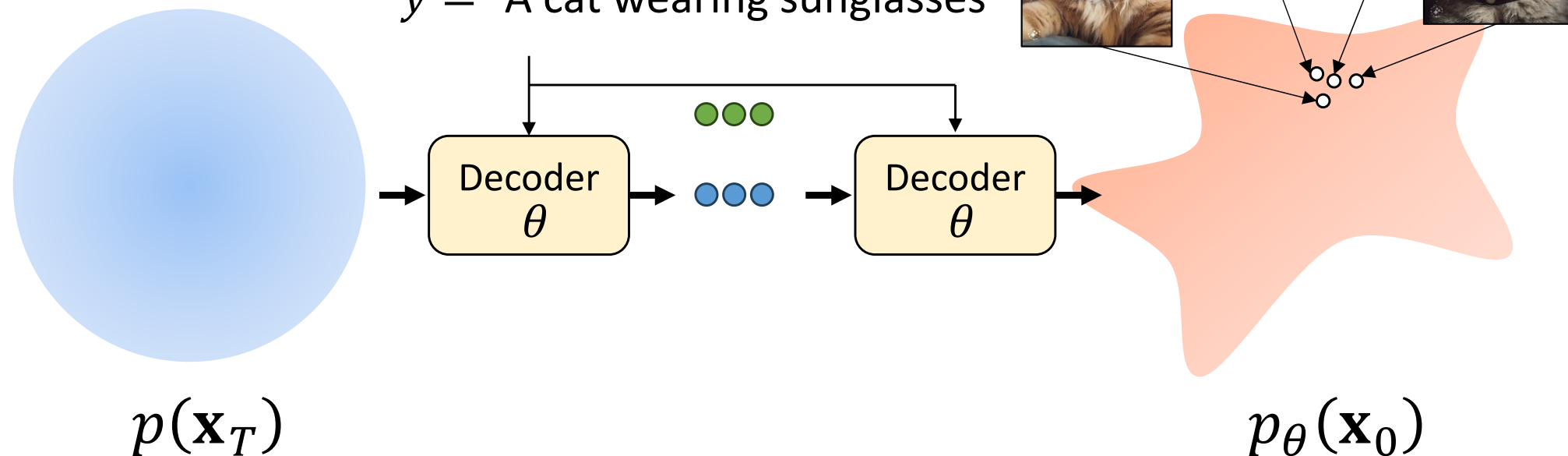
Conditional generation



Conditional generation

Conditional signal

$y = \text{"A cat wearing sunglasses"}$



Unconditional generation

$\mathbf{x}_t$

$p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t)$

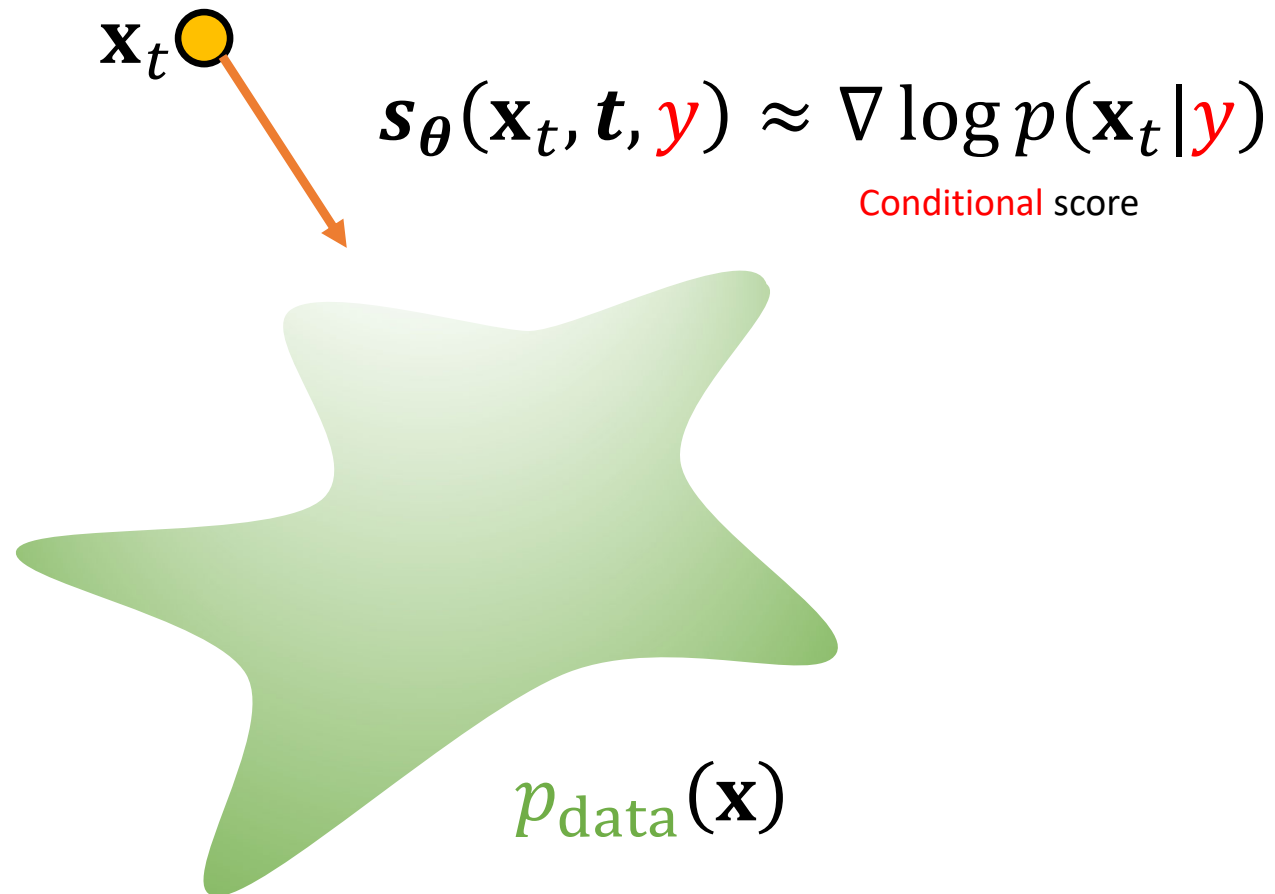
$$\mathbf{s}_\theta(\mathbf{x}_t, t) \approx \nabla \log p(\mathbf{x}_t)$$

Unconditional score

$$\mu_\theta(\mathbf{x}_t, t) = \frac{1}{\sqrt{\alpha_t}} \mathbf{x}_t + \frac{1 - \alpha_t}{\sqrt{\alpha_t}} \mathbf{s}_\theta(\mathbf{x}_t, t)$$

$p_{\text{data}}(\mathbf{x})$

Conditional generation



Conditional generation

$\mathbf{x}_t$

$$s_{\theta}(\mathbf{x}_t, t, \mathbf{y}) \approx \nabla \log p(\mathbf{x}_t | \mathbf{y})$$

Conditional score

$p_{\text{data}}(\mathbf{x})$

Conditional generation

$\mathbf{x}_t$

$$s_{\theta}(\mathbf{x}_t, t, \mathbf{y}) \approx \nabla \log p(\mathbf{x}_t | \mathbf{y})$$

Conditional score

$$\nabla \log p(\mathbf{x}_t | \mathbf{y}) = \nabla \log \left(\frac{p(\mathbf{x}_t) p(\mathbf{y} | \mathbf{x}_t)}{p(\mathbf{y})} \right)$$

Conditional score

$p_{\text{data}}(\mathbf{x})$

Conditional generation

$\mathbf{x}_t$

$$s_{\theta}(\mathbf{x}_t, t, \mathbf{y}) \approx \nabla \log p(\mathbf{x}_t | \mathbf{y})$$

Conditional score

$$\begin{aligned} \nabla \log p(\mathbf{x}_t | \mathbf{y}) &= \nabla \log p(\mathbf{x}_t) + \nabla \log p(\mathbf{y} | \mathbf{x}_t) \\ &\quad - \nabla \log p(\mathbf{y}) \end{aligned}$$

Conditional score

$p_{\text{data}}(\mathbf{x})$

Conditional generation



Classifier
Guidance

$\mathbf{x}_t$

$$s_{\theta}(\mathbf{x}_t, t, \mathbf{y}) \approx \nabla \log p(\mathbf{x}_t | \mathbf{y})$$

Conditional score



$p_{\text{data}}(\mathbf{x})$

$$\nabla \log p(\mathbf{x}_t | \mathbf{y}) = \nabla \log p(\mathbf{x}_t) + \nabla \log p(\mathbf{y} | \mathbf{x}_t)$$

Conditional score

Unconditional score

Adversarial gradient

Conditional generation



Classifier
Guidance

$\mathbf{x}_t$

$$s_{\theta}(\mathbf{x}_t, t, \mathbf{y}) \approx \nabla \log p(\mathbf{x}_t | \mathbf{y})$$

Conditional score



$p_{\text{data}}(\mathbf{x})$

$$\nabla \log p_{\gamma}(\mathbf{x}_t | \mathbf{y}) = \nabla \log p(\mathbf{x}_t) + \gamma \nabla \log p(\mathbf{y} | \mathbf{x}_t)$$

Conditional score

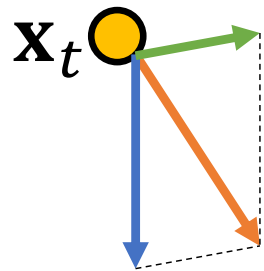
Unconditional score

Adversarial gradient

Conditional generation



Classifier
Guidance



$$s_{\theta}(\mathbf{x}_t, t, \mathbf{y}) \approx \nabla \log p(\mathbf{x}_t | \mathbf{y})$$

Conditional score

$p_{\text{data}}(\mathbf{x})$

$$\nabla \log p_{\gamma}(\mathbf{x}_t | \mathbf{y}) = \nabla \log p(\mathbf{x}_t) + \gamma \nabla \log p(\mathbf{y} | \mathbf{x}_t)$$

Conditional score

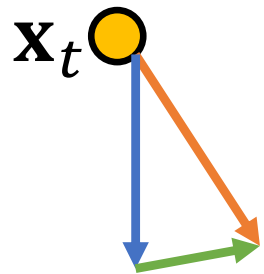
Unconditional score

Adversarial gradient

Conditional generation



Classifier
Guidance



$$s_{\theta}(\mathbf{x}_t, t, \mathbf{y}) \approx \nabla \log p(\mathbf{x}_t | \mathbf{y})$$

Conditional score



$p_{\text{data}}(\mathbf{x})$

$$\nabla \log p_{\gamma}(\mathbf{x}_t | \mathbf{y}) = \nabla \log p(\mathbf{x}_t) + \gamma \nabla \log p(\mathbf{y} | \mathbf{x}_t)$$

Conditional score

Unconditional score

Adversarial gradient

Conditional generation



Classifier
Guidance



Classifier guidance scale = 1



Classifier guidance scale = 10

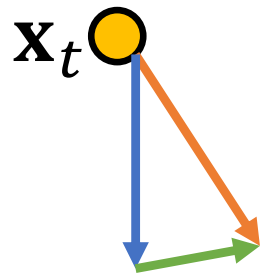
$p_{data}(x)$
 y = "Pembroke Welsh corgi"

[Dhariwal & Nichol 2021]

Conditional generation



Classifier
Guidance



$$s_{\theta}(\mathbf{x}_t, t, \mathbf{y}) \approx \nabla \log p(\mathbf{x}_t | \mathbf{y})$$

Conditional score



$p_{\text{data}}(\mathbf{x})$

$$\nabla \log p_{\gamma}(\mathbf{x}_t | \mathbf{y}) = \nabla \log p(\mathbf{x}_t) + \gamma \nabla \log p(\mathbf{y} | \mathbf{x}_t)$$

Conditional score

Unconditional score

Adversarial gradient

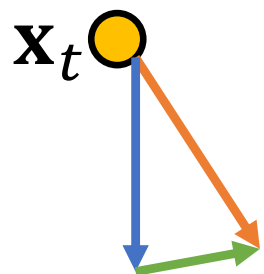
Conditional generation



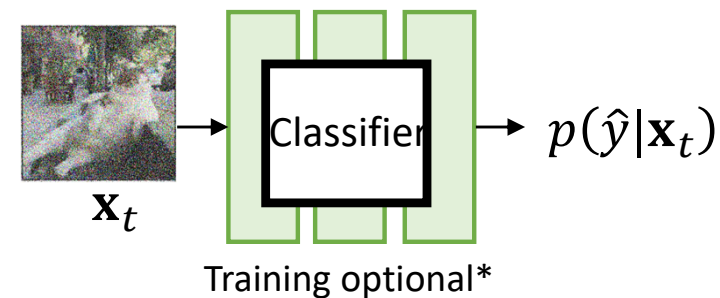
Classifier Guidance

$$\boxed{\nabla \log p_\gamma(\mathbf{x}_t | \mathbf{y})} = \boxed{\nabla \log p(\mathbf{x}_t)} + \boxed{\gamma \nabla \log p(\mathbf{y} | \mathbf{x}_t)}$$

Conditional score Unconditional score Adversarial gradient



$p_{\text{data}}(\mathbf{x})$



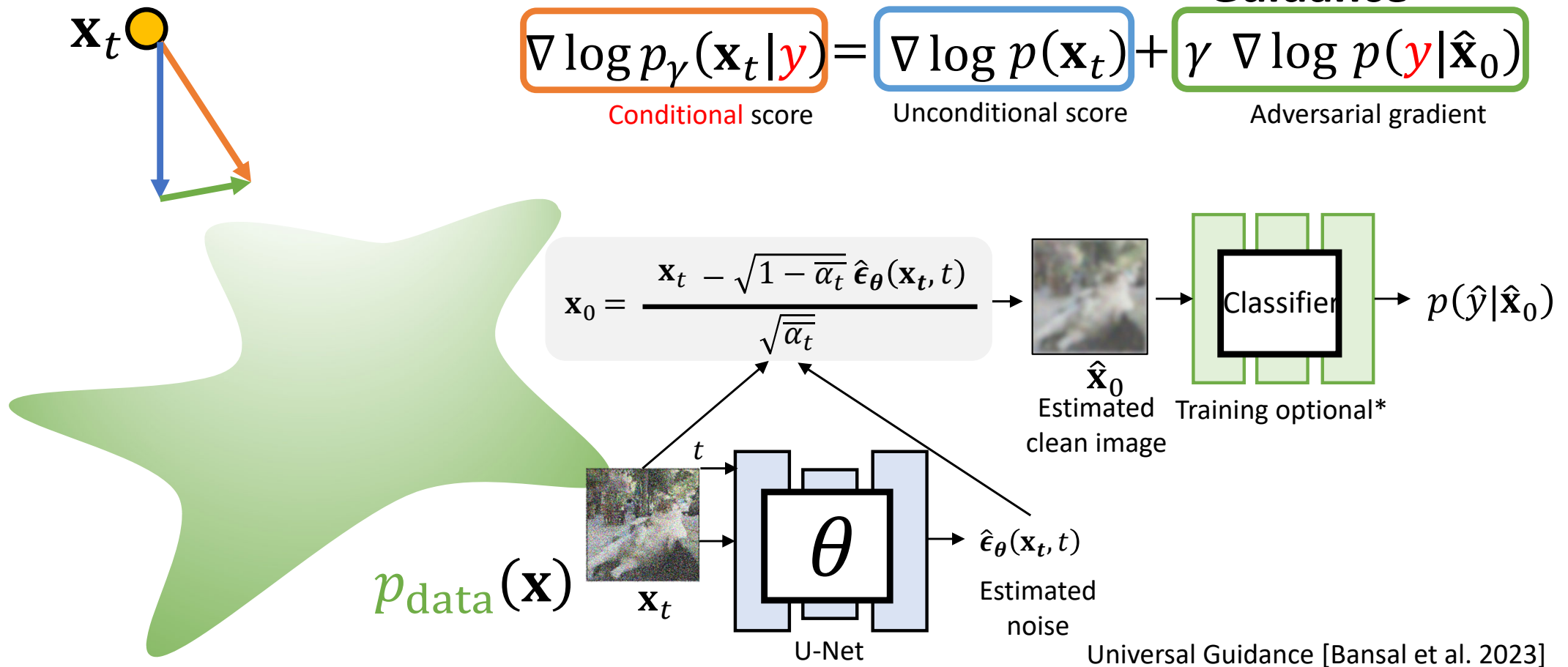
Conditional generation



Classifier Guidance

$$\nabla \log p_{\gamma}(\mathbf{x}_t | \mathbf{y}) = \nabla \log p(\mathbf{x}_t) + \gamma \nabla \log p(\mathbf{y} | \hat{\mathbf{x}}_0)$$

Conditional score
Unconditional score
Adversarial gradient



Conditional Generation with Classifier-Guidance

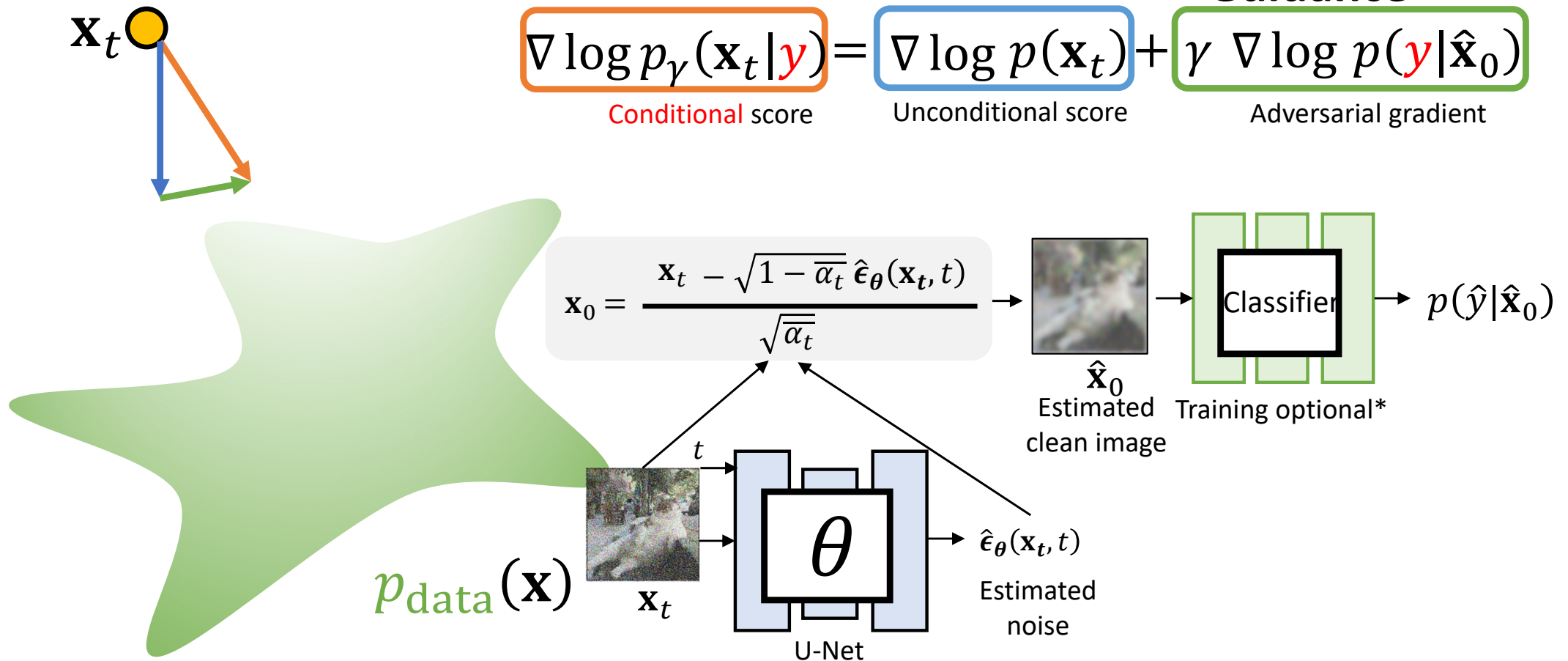
- Any price to pay?



Classifier Guidance

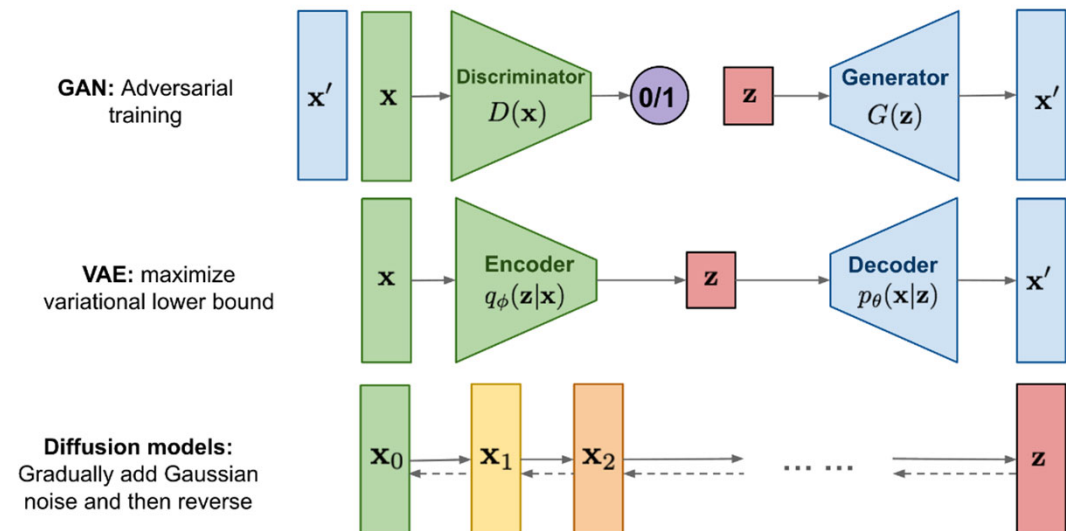
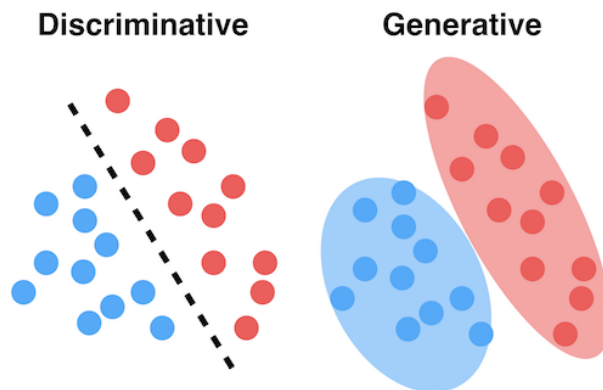
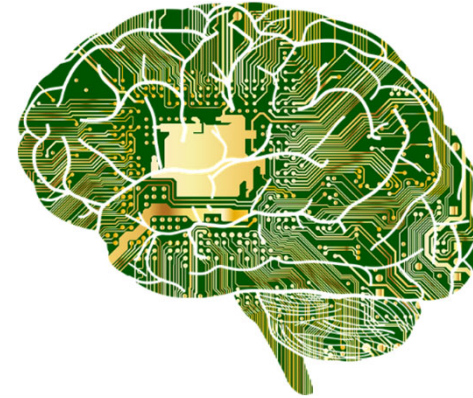
$$\nabla \log p_\gamma(\mathbf{x}_t | \mathbf{y}) = \nabla \log p(\mathbf{x}_t) + \gamma \nabla \log p(\mathbf{y} | \hat{\mathbf{x}}_0)$$

Conditional score
Unconditional score
Adversarial gradient



What's to Be Covered Today...

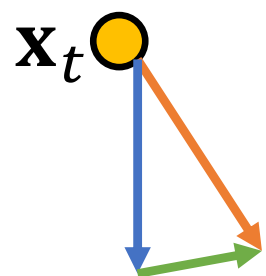
- Generative Models
 - Diffusion Model
 - Conditional Diffusion Model
 - Classifier Guidance
 - Classifier-Free Guidance
 - Personalization via Diffusion Model
 - Generative Adversarial Network



Conditional generation



Classifier Guidance



$$\nabla \log p_\gamma(\mathbf{x}_t | \mathbf{y}) = \nabla \log p(\mathbf{x}_t) + \gamma \nabla \log p(\mathbf{y} | \mathbf{x}_t)$$

Conditional score Unconditional score Adversarial gradient

$p_{\text{data}}(\mathbf{x})$

Conditional generation

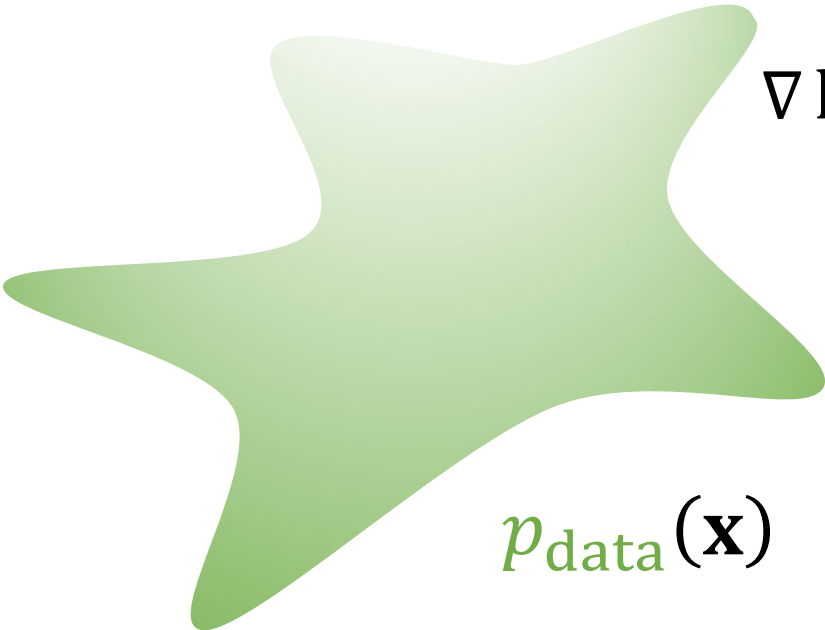
$\mathbf{x}_t$ 

$$\nabla \log p_\gamma(\mathbf{x}_t | \mathbf{y}) = \nabla \log p(\mathbf{x}_t) + \gamma \nabla \log p(\mathbf{y} | \mathbf{x}_t)$$

Conditional score

Unconditional score

Adversarial gradient


$$\nabla \log p(\mathbf{y} | \mathbf{x}_t) = \nabla \log \left(\frac{p(\mathbf{x}_t | \mathbf{y}) p(\mathbf{y})}{p(\mathbf{x}_t)} \right)$$

$p_{\text{data}}(\mathbf{x})$

Conditional generation

$\mathbf{x}_t$ 

$$\nabla \log p_\gamma(\mathbf{x}_t | \mathbf{y}) = \nabla \log p(\mathbf{x}_t) + \gamma \nabla \log p(\mathbf{y} | \mathbf{x}_t)$$

Conditional score

Unconditional score

Adversarial gradient

$$\begin{aligned} \nabla \log p(\mathbf{y} | \mathbf{x}_t) &= \nabla \log p(\mathbf{x}_t | \mathbf{y}) - \nabla \log p(\mathbf{x}_t) \\ &\quad + \nabla \log p(\mathbf{y}) \end{aligned}$$

$p_{\text{data}}(\mathbf{x})$

Conditional generation

$\mathbf{x}_t$ 

$$\nabla \log p_\gamma(\mathbf{x}_t|\mathbf{y}) = \nabla \log p(\mathbf{x}_t) + \gamma \nabla \log p(\mathbf{y}|\mathbf{x}_t)$$

Conditional score

Unconditional score

Adversarial gradient

$$\nabla \log p(\mathbf{y}|\mathbf{x}_t) = \nabla \log p(\mathbf{x}_t|\mathbf{y}) - \nabla \log p(\mathbf{x}_t)$$

$p_{\text{data}}(\mathbf{x})$



Conditional generation

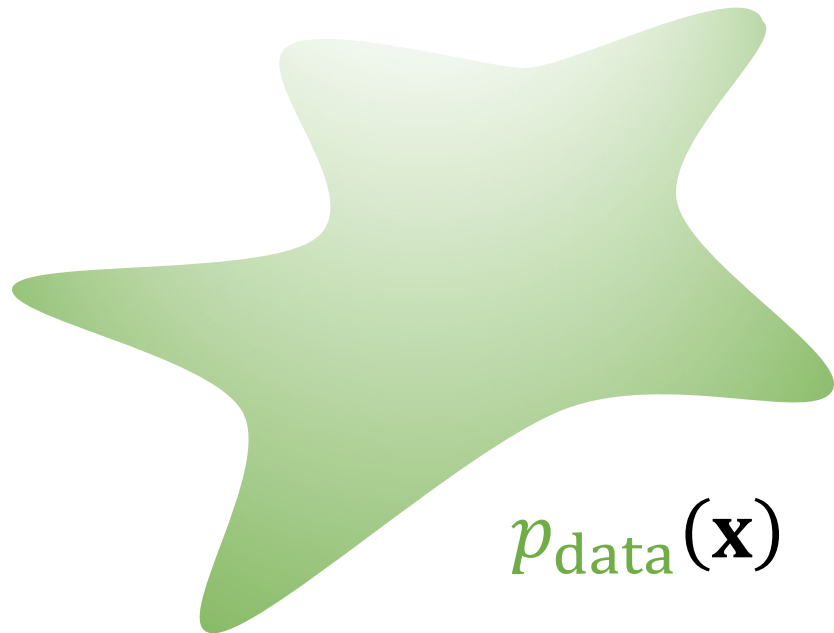
$\mathbf{x}_t$ 

$$\nabla \log p_\gamma(\mathbf{x}_t | \textcolor{red}{y}) = \nabla \log p(\mathbf{x}_t) + \gamma(\nabla \log p(\mathbf{x}_t | \textcolor{red}{y}) - \nabla \log p(\mathbf{x}_t))$$

Conditional score

Unconditional score

Adversarial gradient

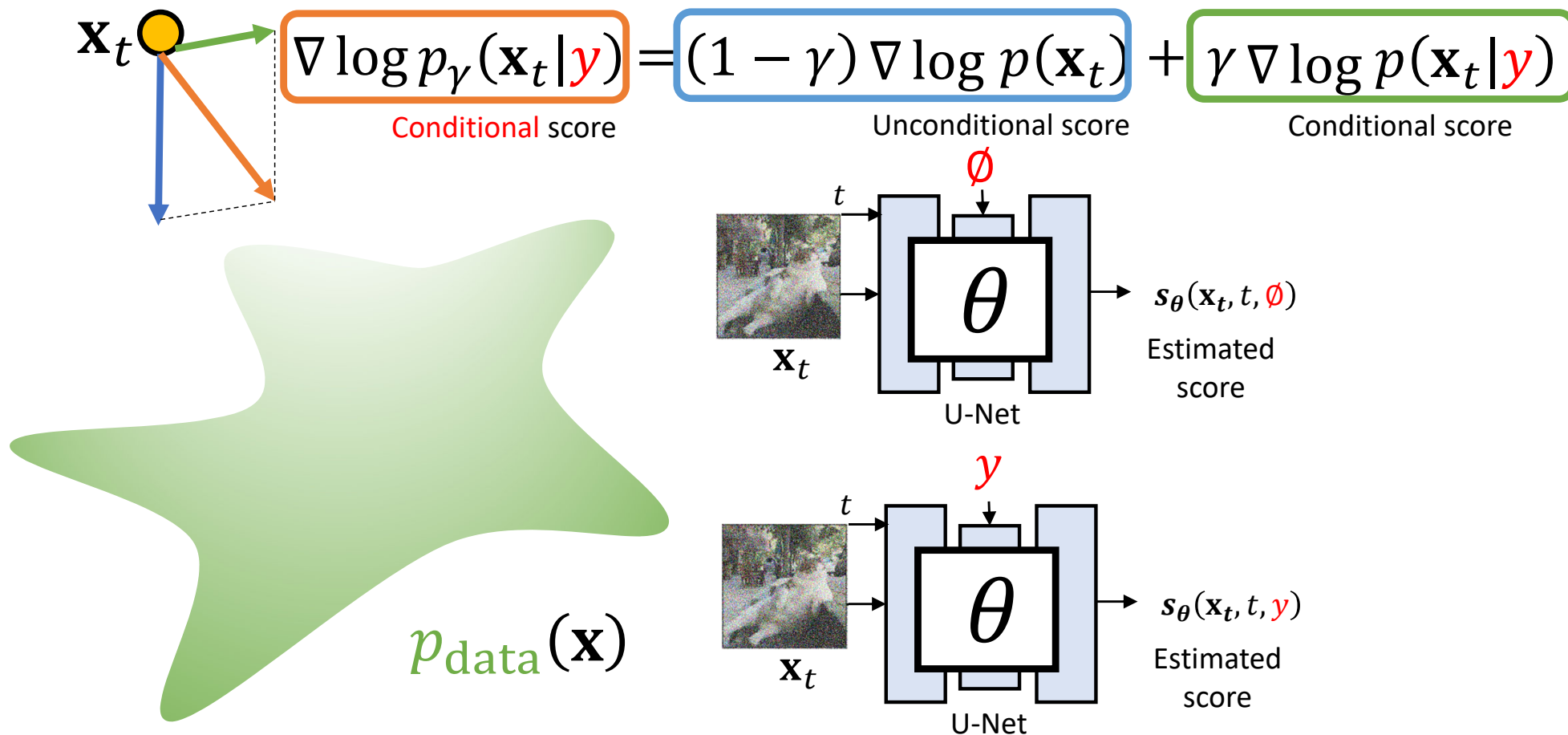


$p_{\text{data}}(\mathbf{x})$

Conditional generation



Classifier-free Guidance



Conditional generation

Classifier-free Guidance



Unguided samples

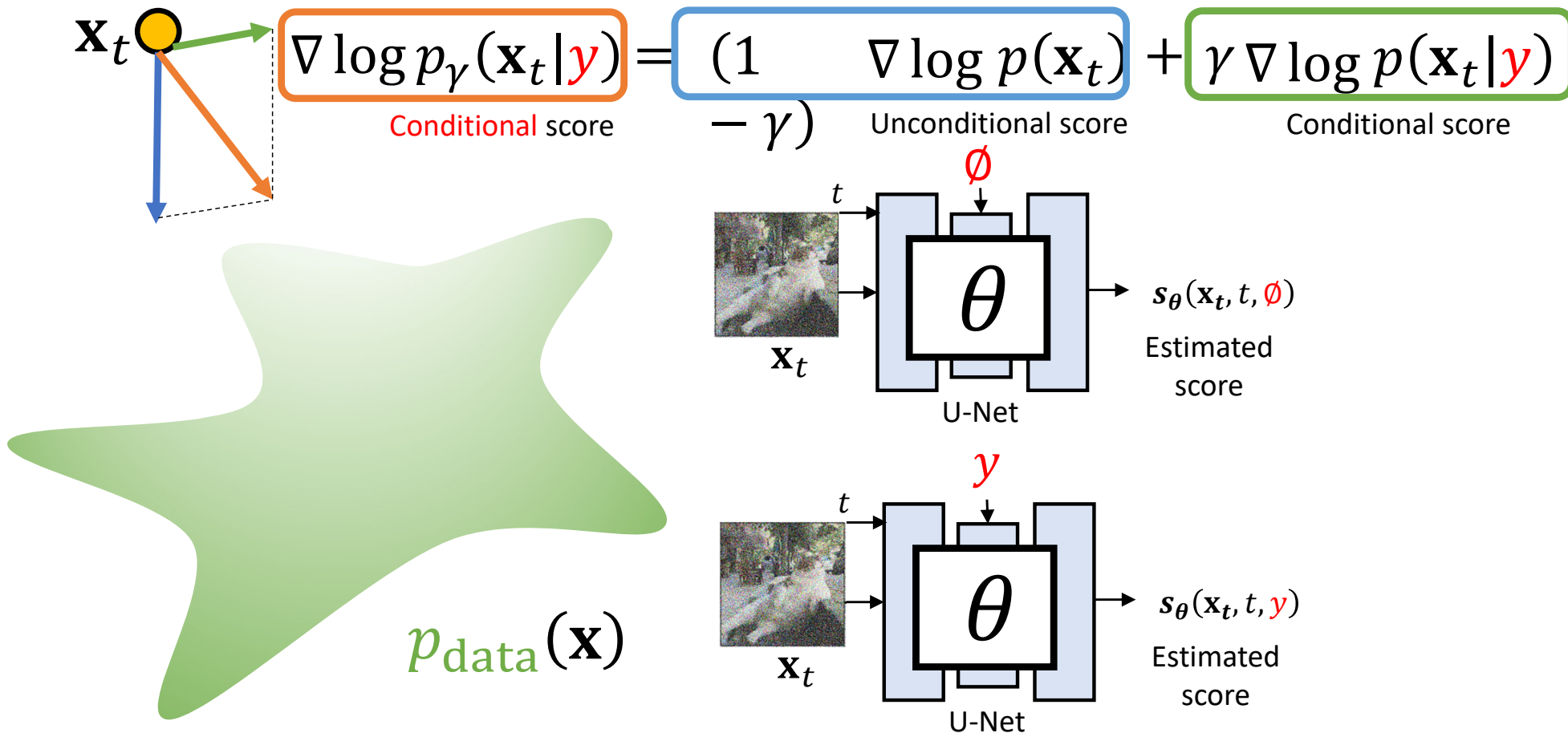
Guided samples

[Ho and Salimans 2021]

Conditional Generation with Classifier-Free Guidance

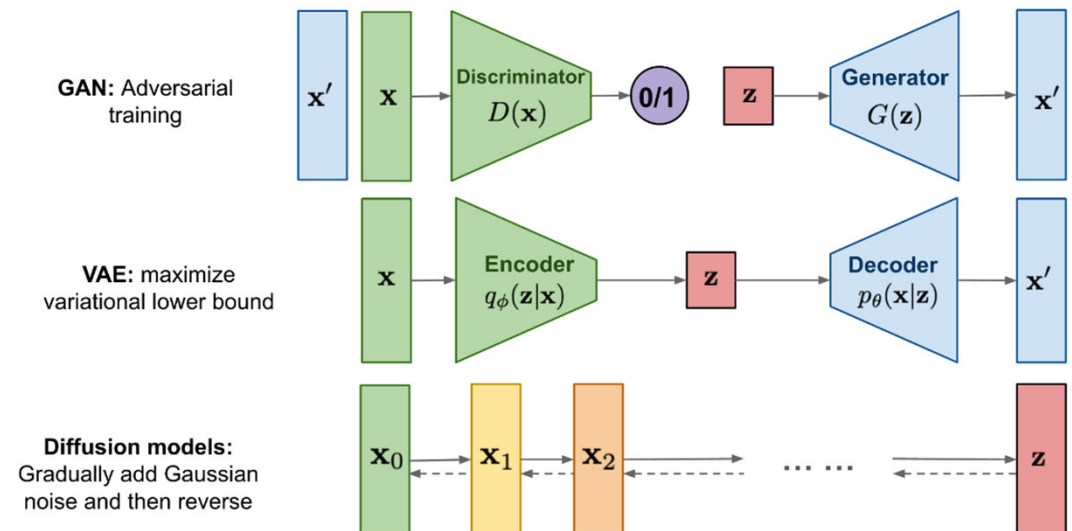
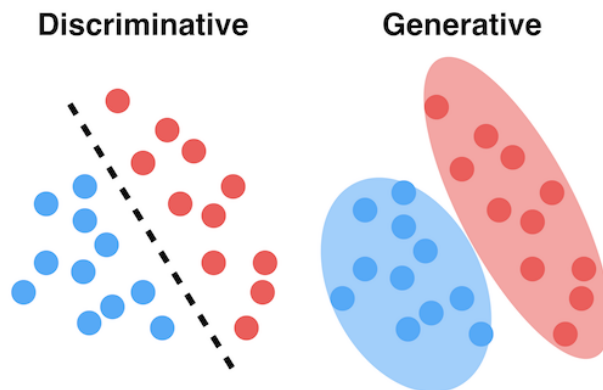
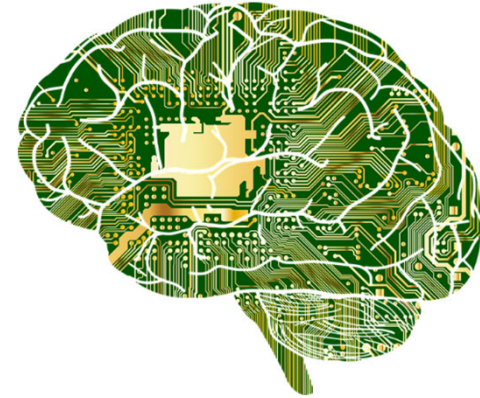
- Any price to pay?

Classifier-free Guidance



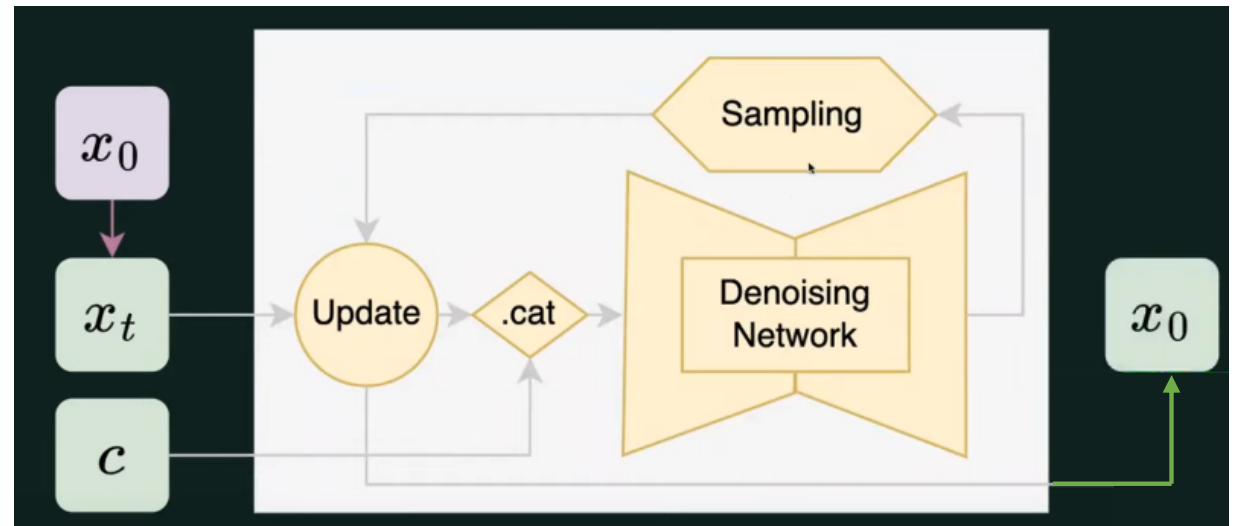
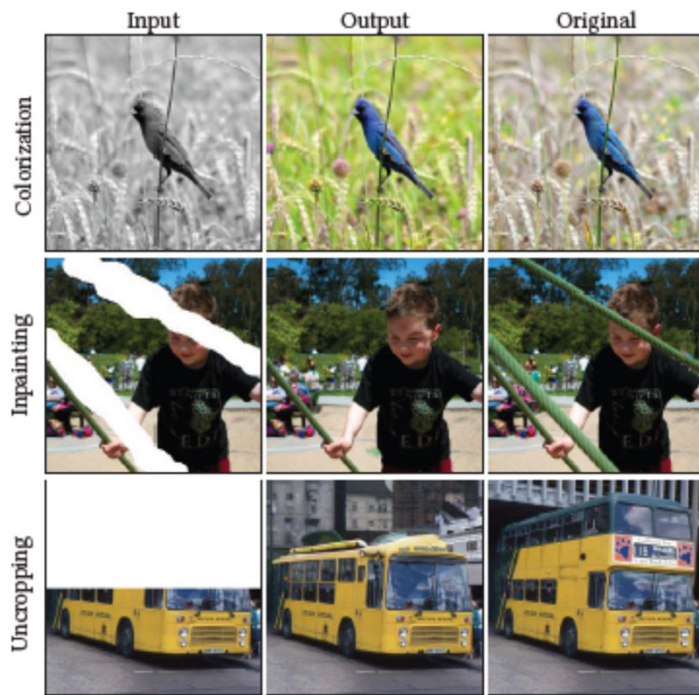
What's to Be Covered Today...

- Generative Models
 - Diffusion Model
 - Conditional Diffusion Model
 - Classifier Guidance
 - Classifier-Free Guidance
 - Text/Image Guidance
 - Personalization via Diffusion Model
 - Generative Adversarial Network



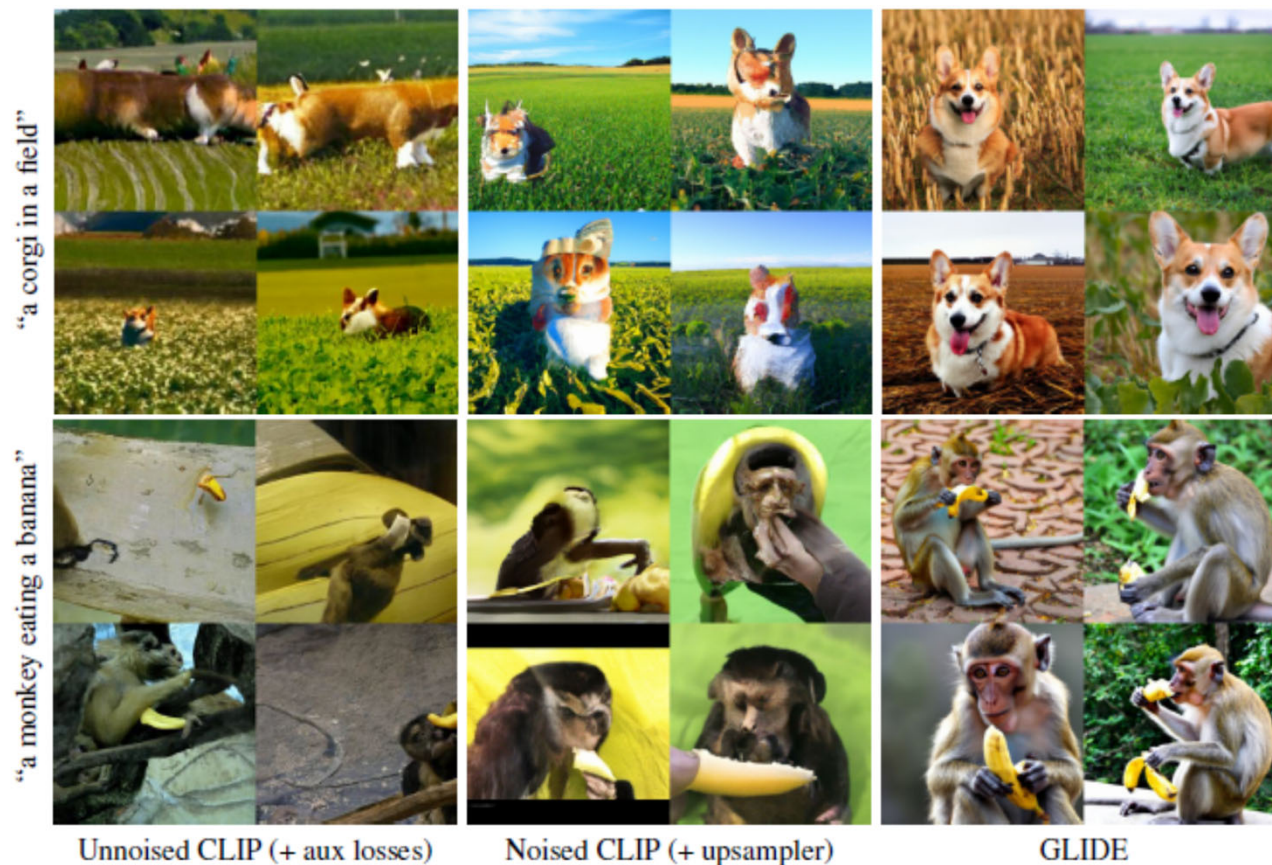
Conditional Generation with Image Guidance

- *Palette: Image-to-Image Diffusion Models*, Google Research, arXiv 2022
 - Applications: colorization, inpainting, outpainting, etc.
 - Input image as condition (via concatenation)



Conditional Generation with Text Guidance

- *GLIDE: Towards Photorealistic Image Generation and Editing with Text-Guided Diffusion Models*, OpenAI, arXiv 2022
 - CLIP (Contrastive Language-Image Pretraining) is previously proposed to measure alignment between **text** and **image** data
 - Guidance: Classifier -> CLIP
 - What is CLIP anyway? (see next slide)



CLIP: Contrastive Language-Image Pretraining

- *Learning Transferable Visual Models From Natural Language Supervision*, OpenAI, NeurIPS WS 2021 (w/ 43000+ citations)
- Why not just CNN?
 - Require **annotated data** for training image classification
 - **Domain gap** between closed and open-world domain data
 - Lack of ability for **zero-shot classification**



let

76.2%



geNet V2

64.3%



ImageNet Rendition

37.7%



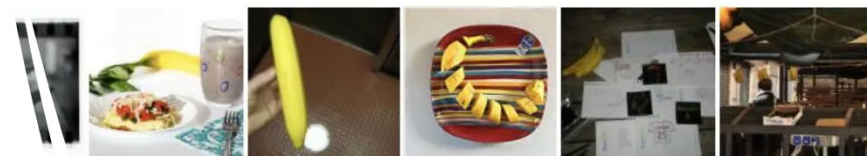
ObjectNet

32.6%



ImageNet Sketch

25.2%



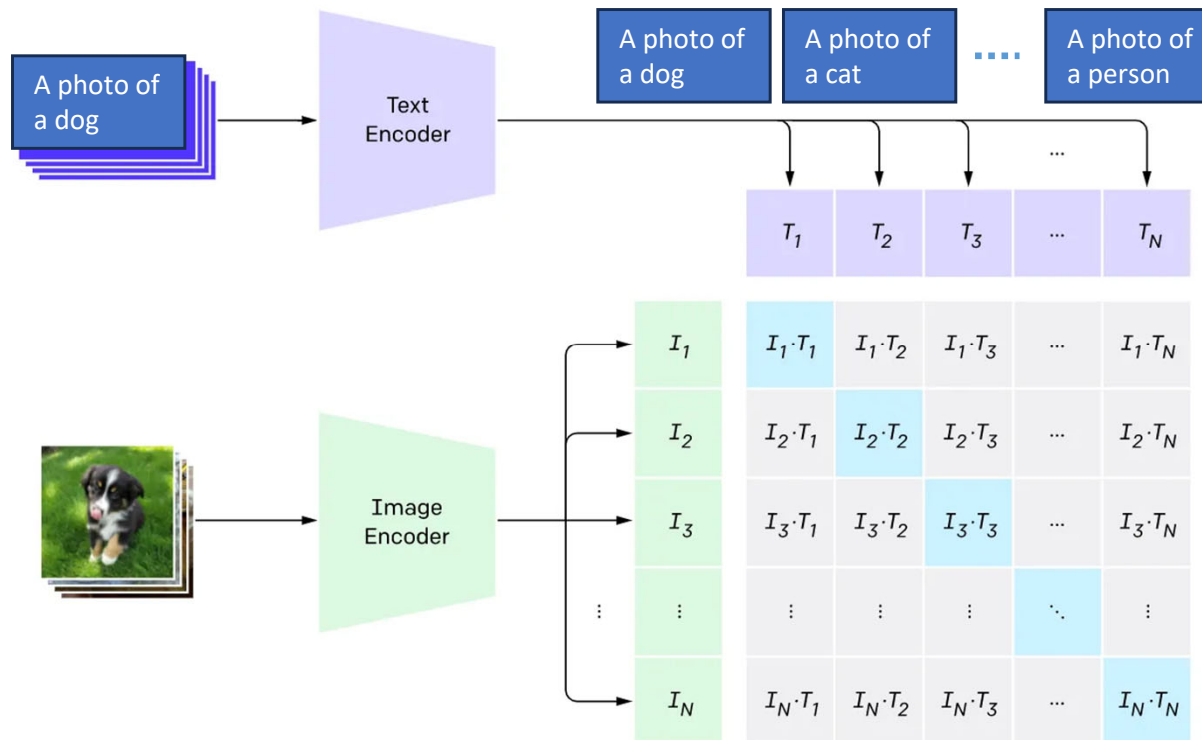
Not Adversarial

2.7%

CLIP (cont'd)

- Objectives
 - Cross-domain contrastive learning from large-scale image-language data
 - Next-token prediction (**what's this & why?**);
will talk more about this for the lecture of Transformer

1. Contrastive pre-training



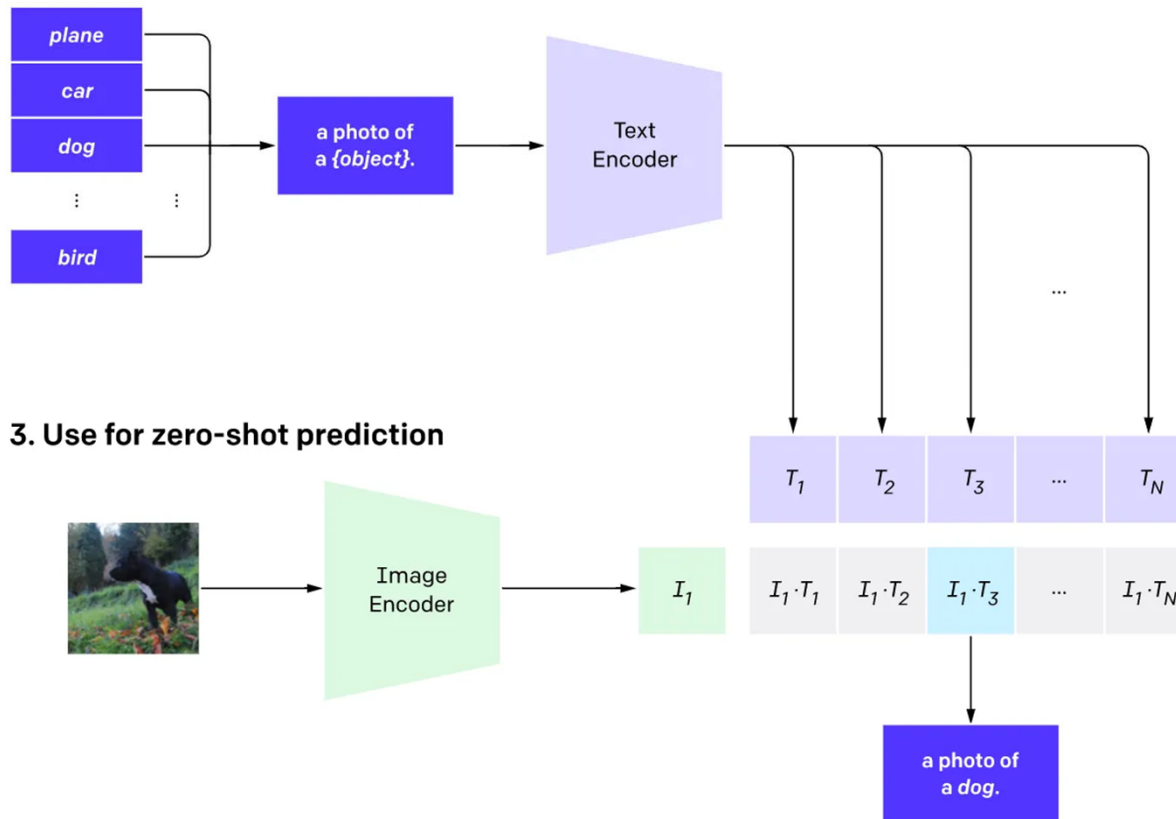
2. Next-token prediction

e.g., a ____;
a photo ____;
a photo of ____, etc.

CLIP (cont'd)

- (Zero-shot) Inference:

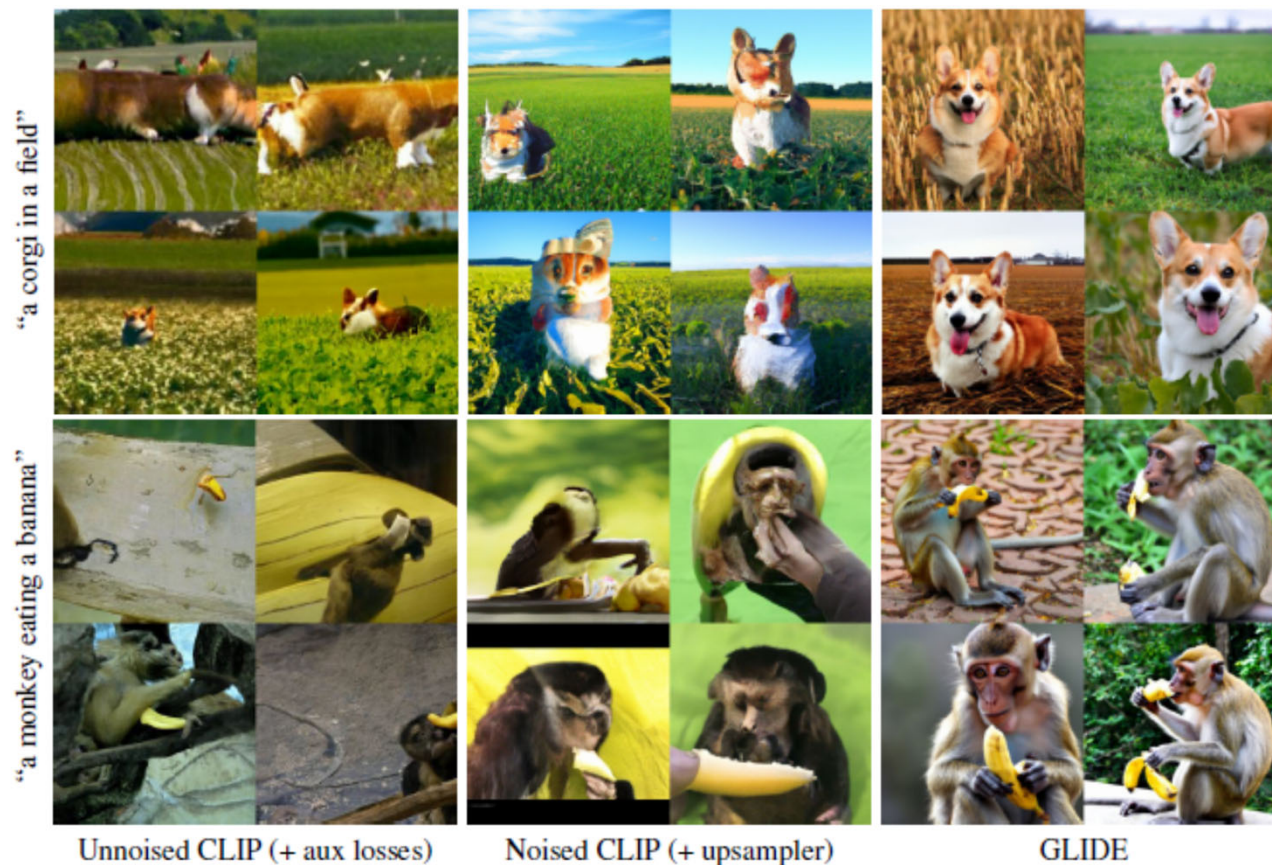
2. Create dataset classifier from label text



- Potential concerns/disadvantages of CLIP?

Conditional Generation with Text Guidance

- *GLIDE: Towards Photorealistic Image Generation and Editing with Text-Guided Diffusion Models*, OpenAI, arXiv 2022
 - CLIP (Contrastive Language-Image Pretraining) is previously proposed to measure alignment between **text** and **image** data
 - Guidance: Classifier -> CLIP
 - What is CLIP anyway? (see next slide)



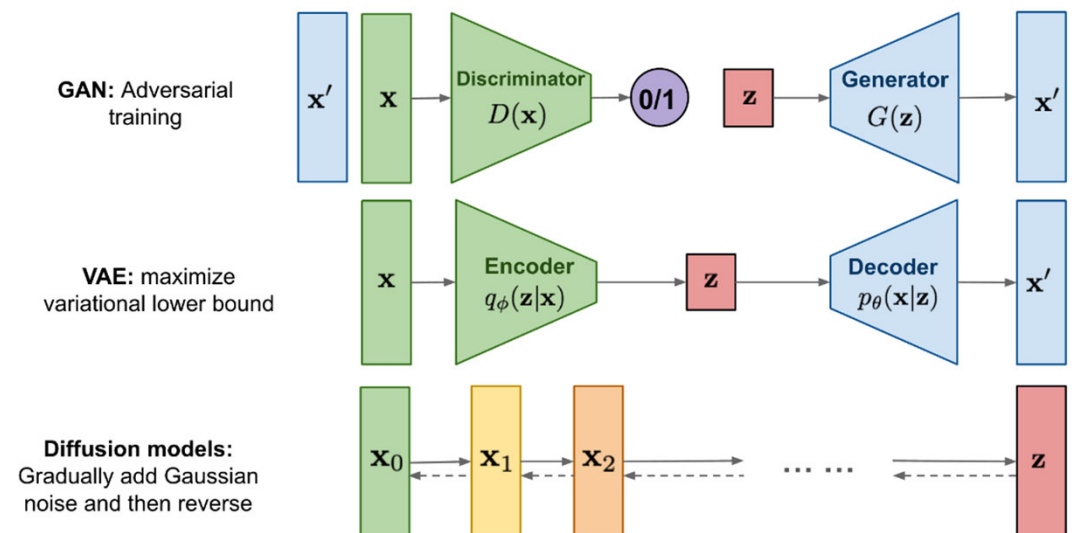
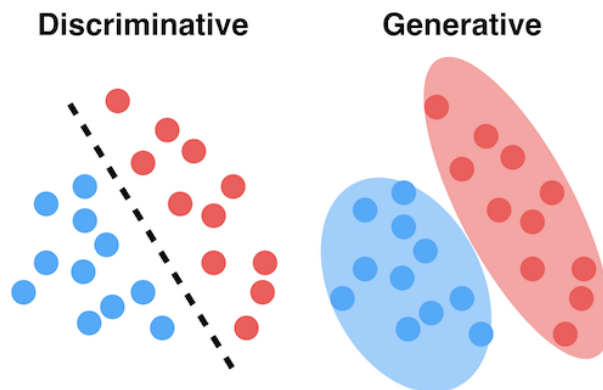
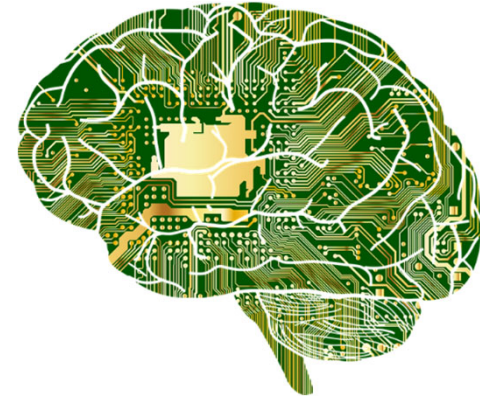
Questions for Image Generation

- How to evaluate your unconditional image generation results?
- How to evaluate your conditional image generation results?
- Any objective/subjective metric for quantitative/qualitative evaluation?



What's to Be Covered Today...

- Generative Models
 - Diffusion Model
 - Conditional Diffusion Model
 - Personalization via Diffusion Model
 - Generative Adversarial Network



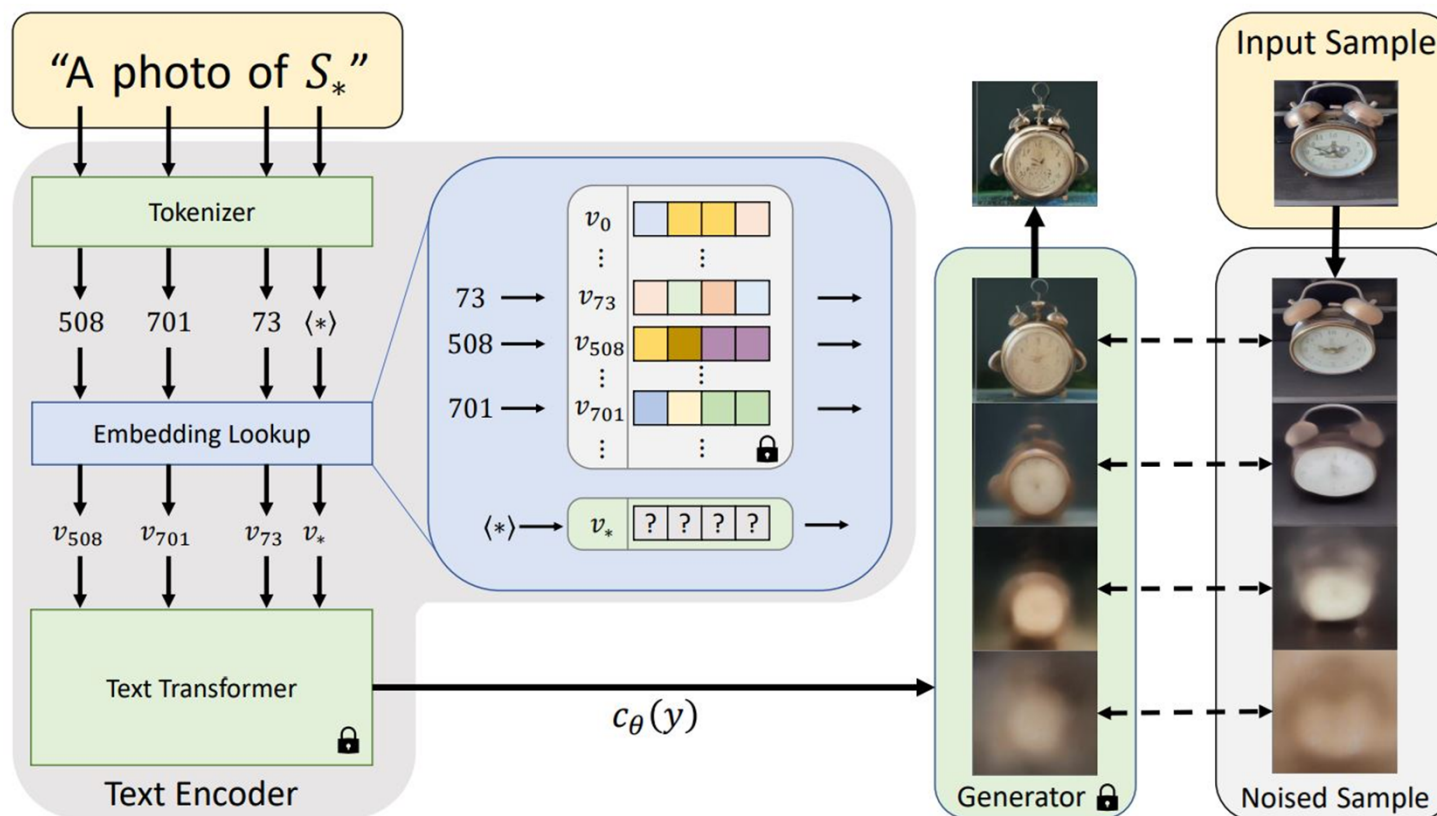
Diffusion Model for Personalization (1): Textual Inversion

- Proposed by *NV Research*, ICLR 2023
- Goal: Learn a special token (e.g., S^*) to represent the concept of interest



Diffusion Model for Personalization (1): Textual Inversion (cont'd)

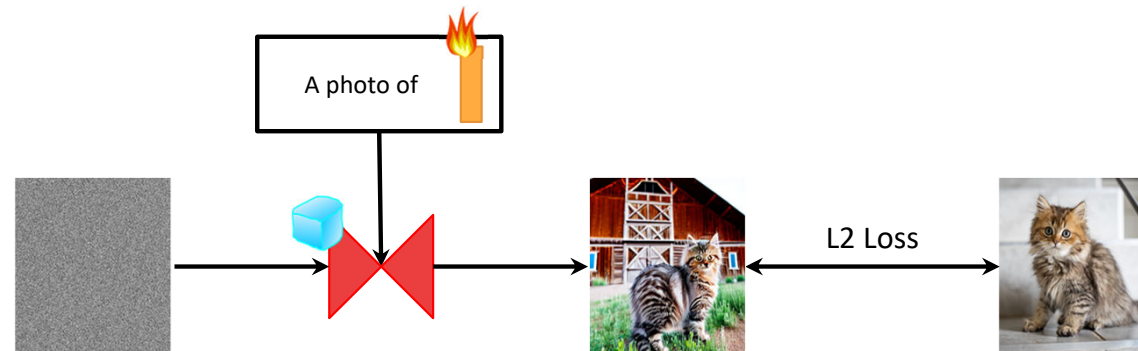
- Learning of special token S^*
 - Pre-train and fix text encoder & diffusion model (i.e., generator)
 - Randomly initialize a token as the text encoder input
 - Optimize this token via image reconstruction objectives



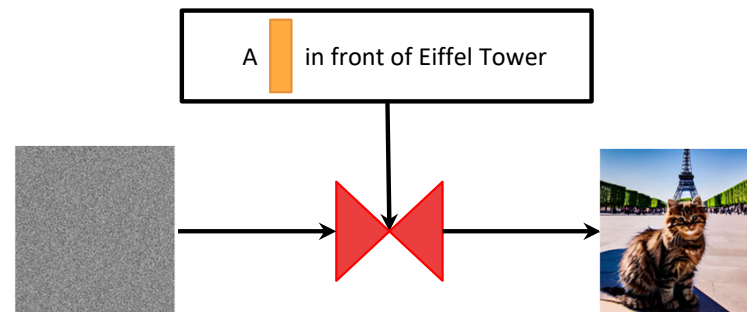
Diffusion Model for Personalization (1): Textual Inversion (cont'd)

- Learning of special token S^*
 - Pre-train and fix text encoder & diffusion model (i.e., generator)
 - Randomly initialize a token as the text encoder input
 - Optimize this token via image reconstruction objectives

- Training:



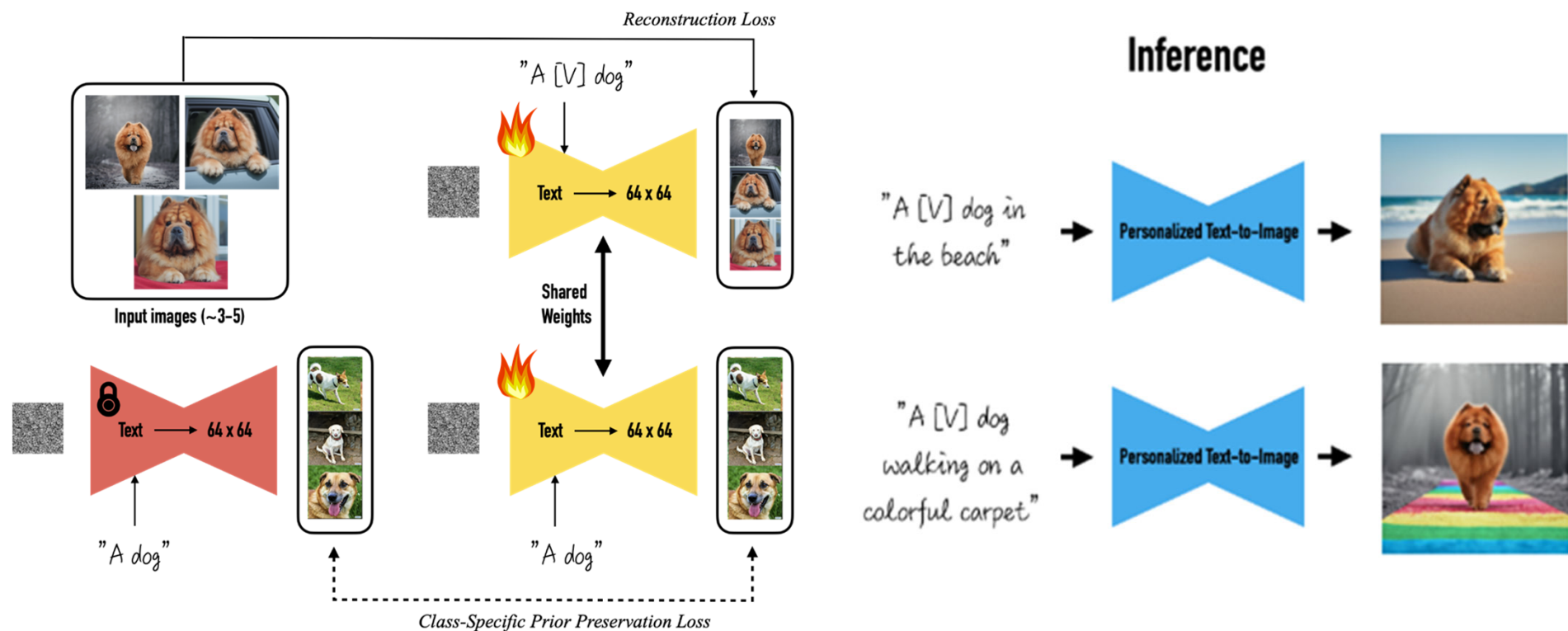
- Inference:



- Any potential concern?

Diffusion Model for Personalization (2): DreamBooth

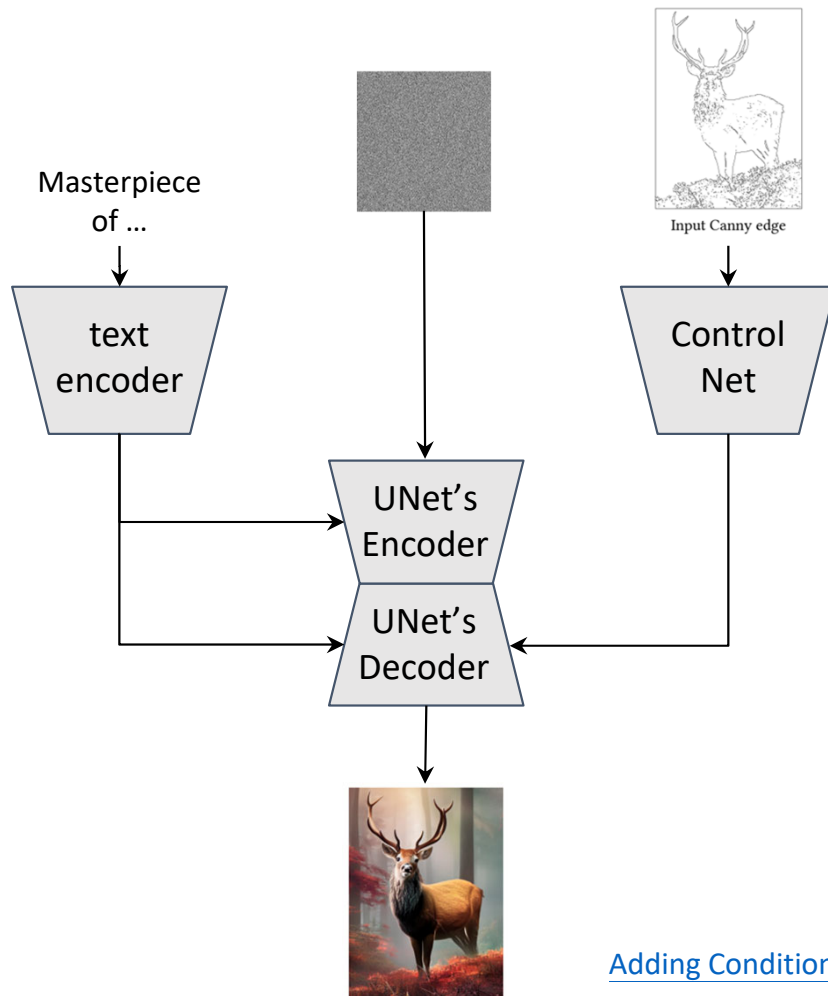
- Proposed by Google Research, CVPR 2023
- Finetune the diffusion model w/ a fixed token to represent the image concept
 - Determine and fix a rare token (e.g., [V])
 - **Finetune the diffusion model** for image restoration objectives
 - Enforce a class-specific prior (**why?**)



- **Any concern?**

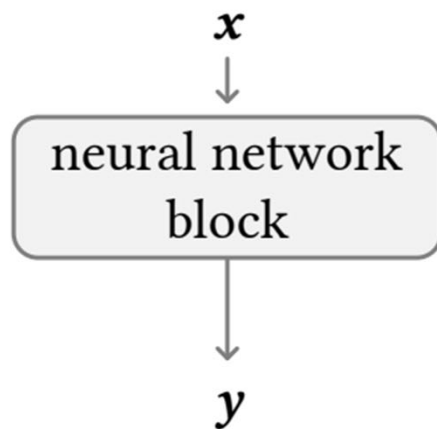
Diffusion Model for Personalization (3): ControlNet

- Proposed by Stanford, ICCV 2023
- Goal: personalization via user-determined condition

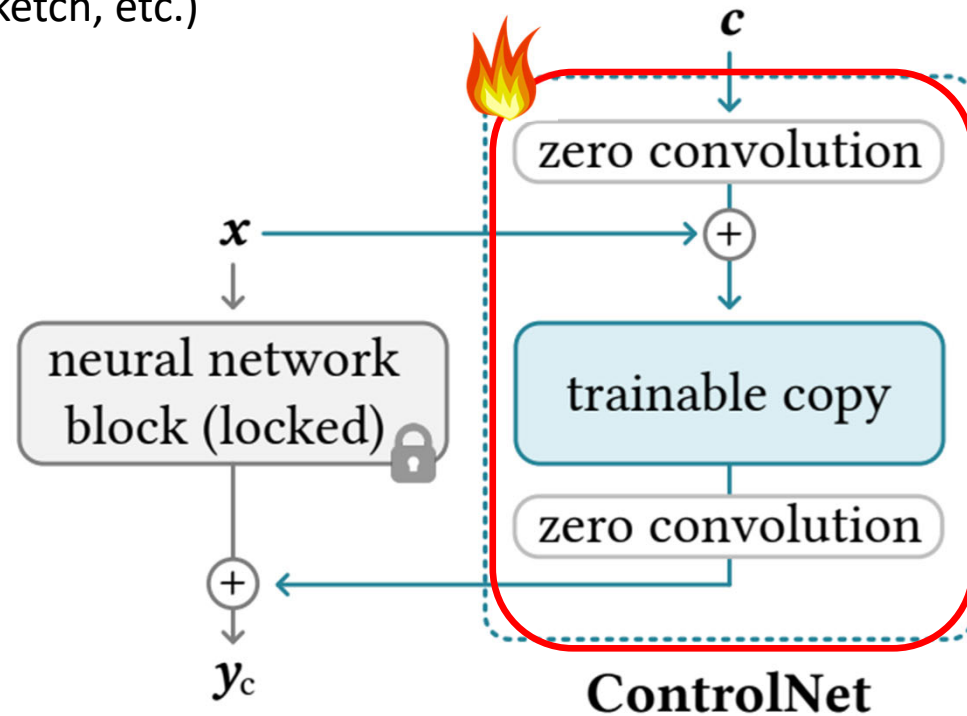


Diffusion Model for Personalization (3): ControlNet

- Initialized from UNet's encoder
- Notations:
 - x : input noise of each layer
 - y : output noise of each layer
 - c : conditions (e.g., edge, pose, sketch, etc.)



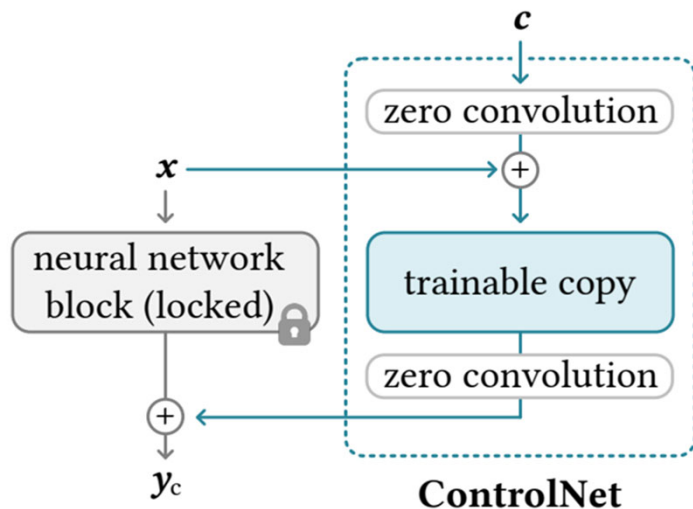
(a) Before



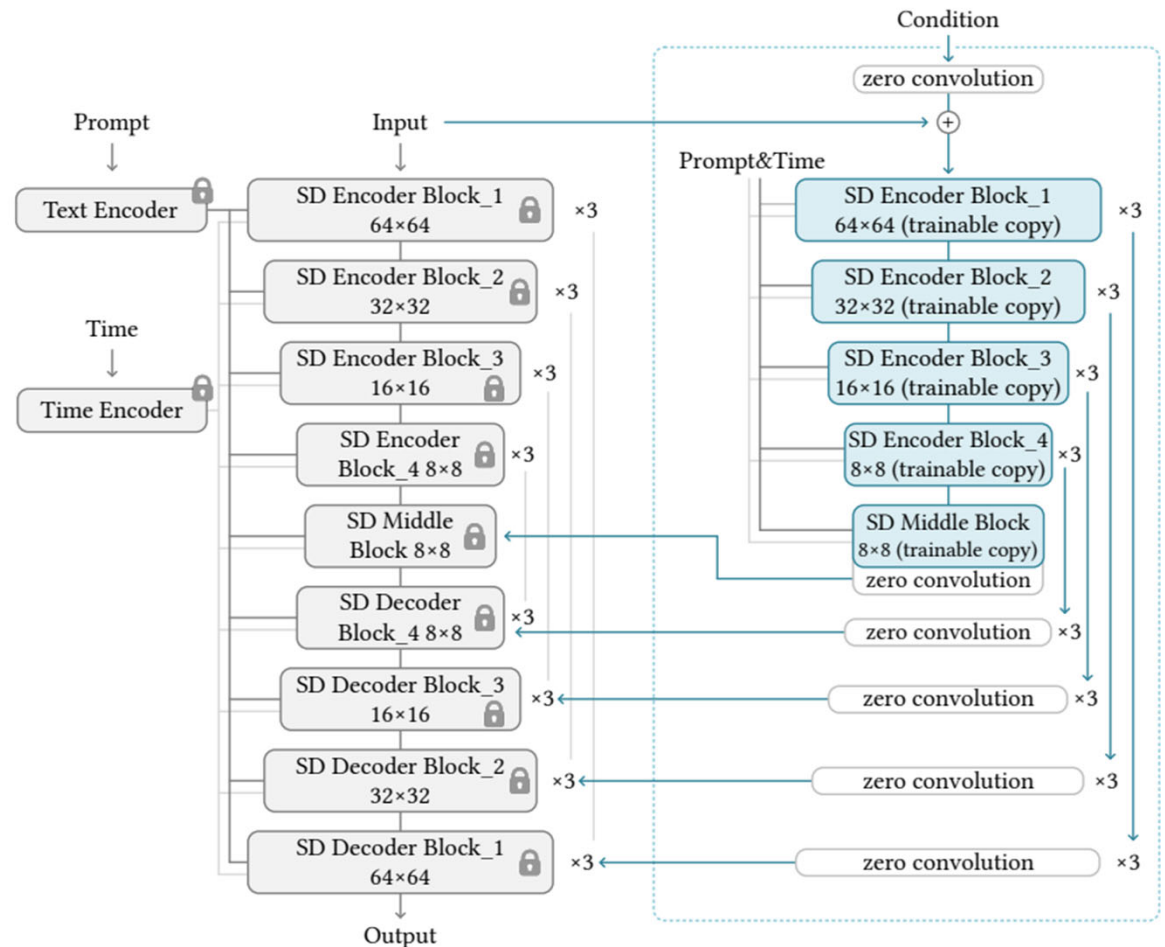
(b) After

Diffusion Model for Personalization (3): ControlNet

- Initialized from UNet's encoder
- Notations:
 - x : input noise of each layer
 - y : output noise of each layer
 - c : conditions (e.g., edge,, etc.)



(b) After

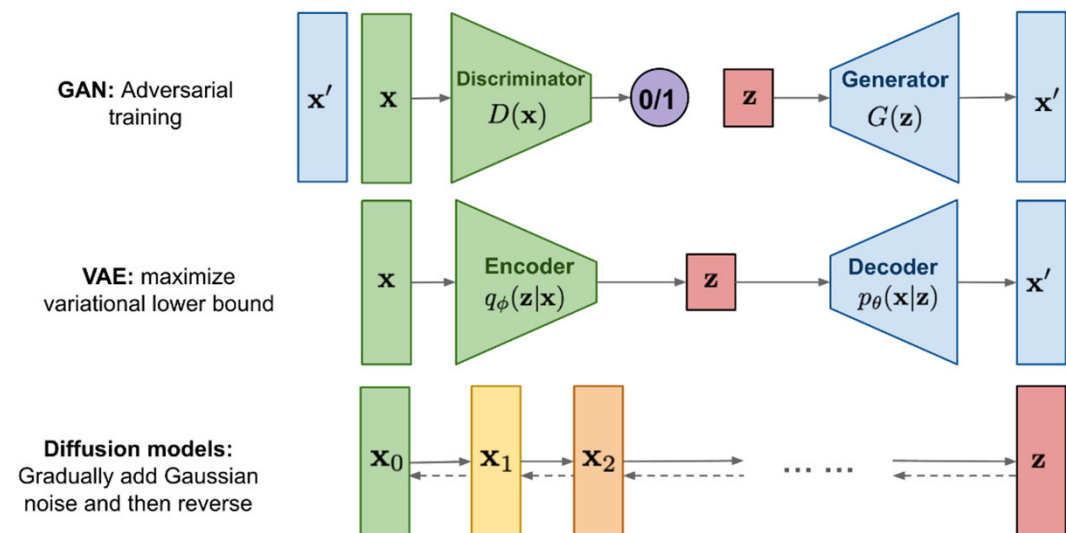
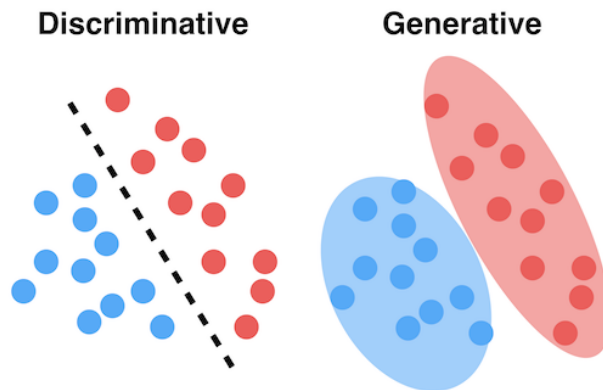
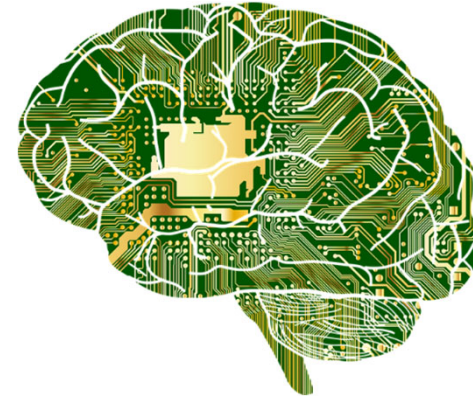


(a) Stable Diffusion

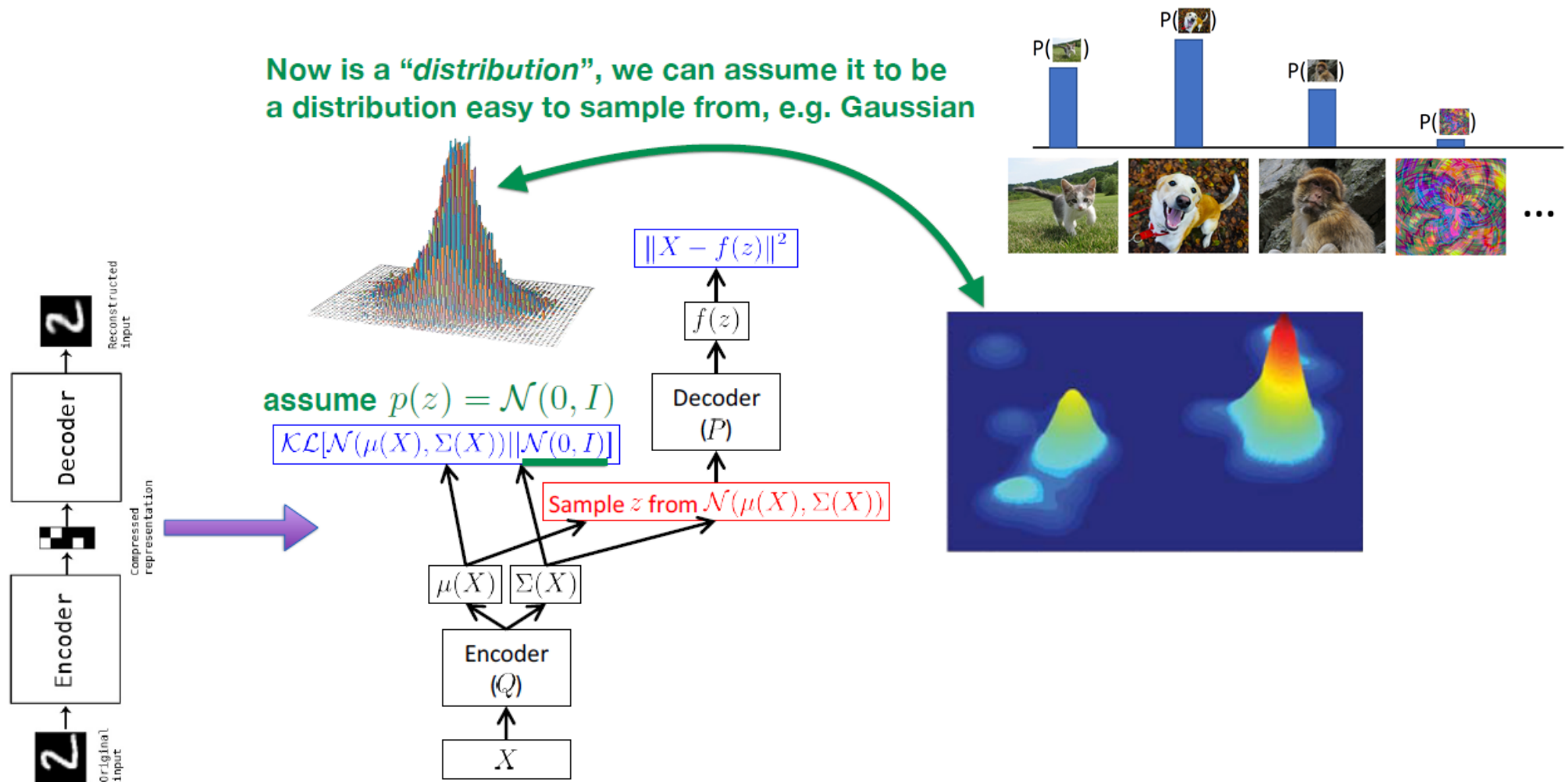
(b) ControlNet

What's to Be Covered Today...

- Generative Models
 - Diffusion Model
 - Conditional Diffusion Model
 - Classifier Guidance
 - Classifier-Free Guidance
 - Text/Image Guidance
 - Personalization via Diffusion Model
 - Generative Adversarial Network

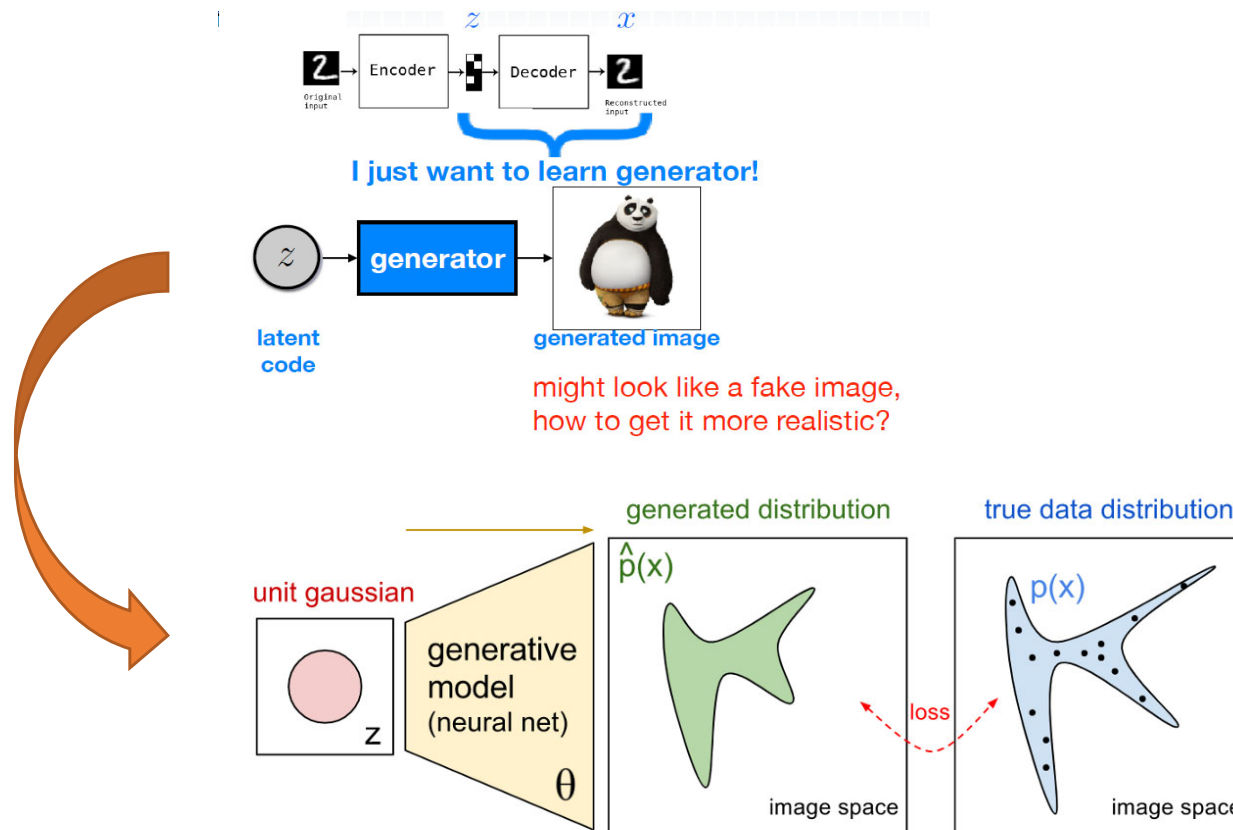
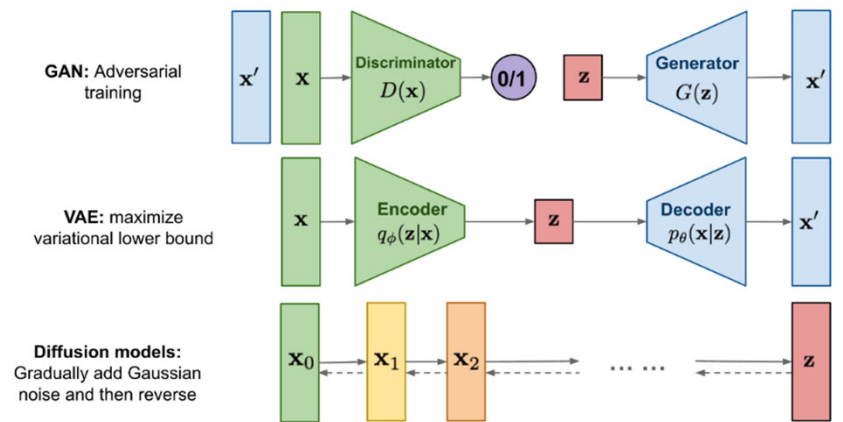


From VAE to *Generative Adversarial Networks (GAN)*



From VAE to GAN

- Remarks
 - We only need the decoder/generator in practice.
 - We prefer fast generation.
 - How do we know if the output images are sufficiently good?



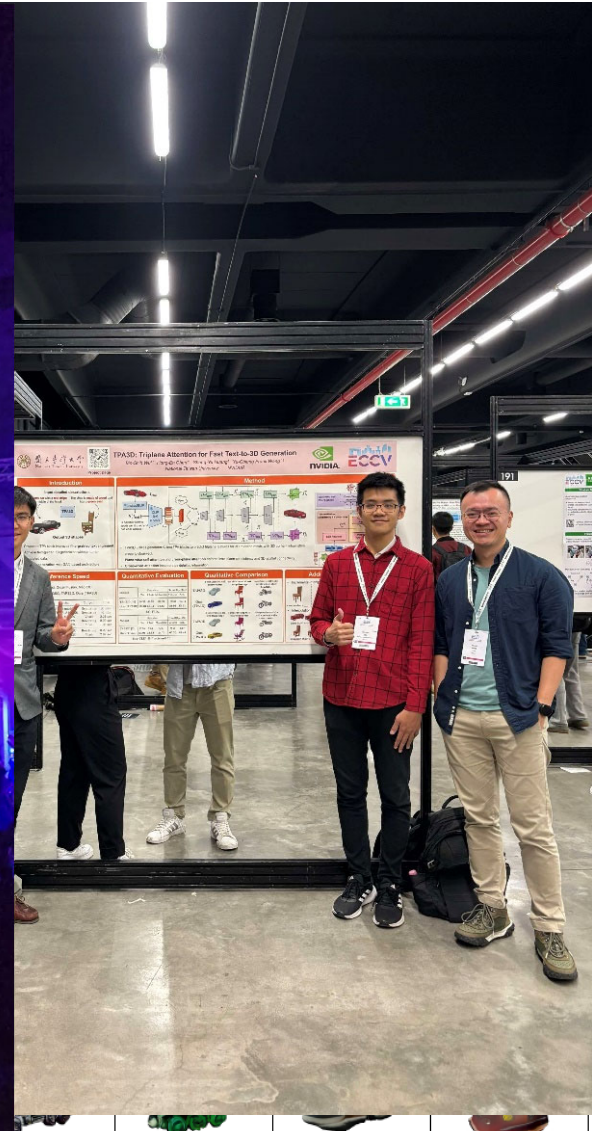
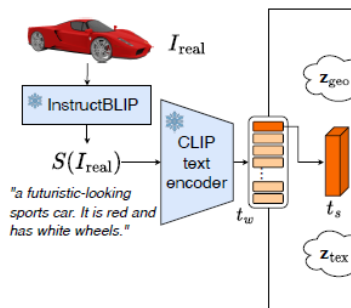
GAN is NOT an outdated DL technology

• Remark

- W
- W
- H

• Example

- T
- B



antique-style motorbike
with a red and white
color scheme.



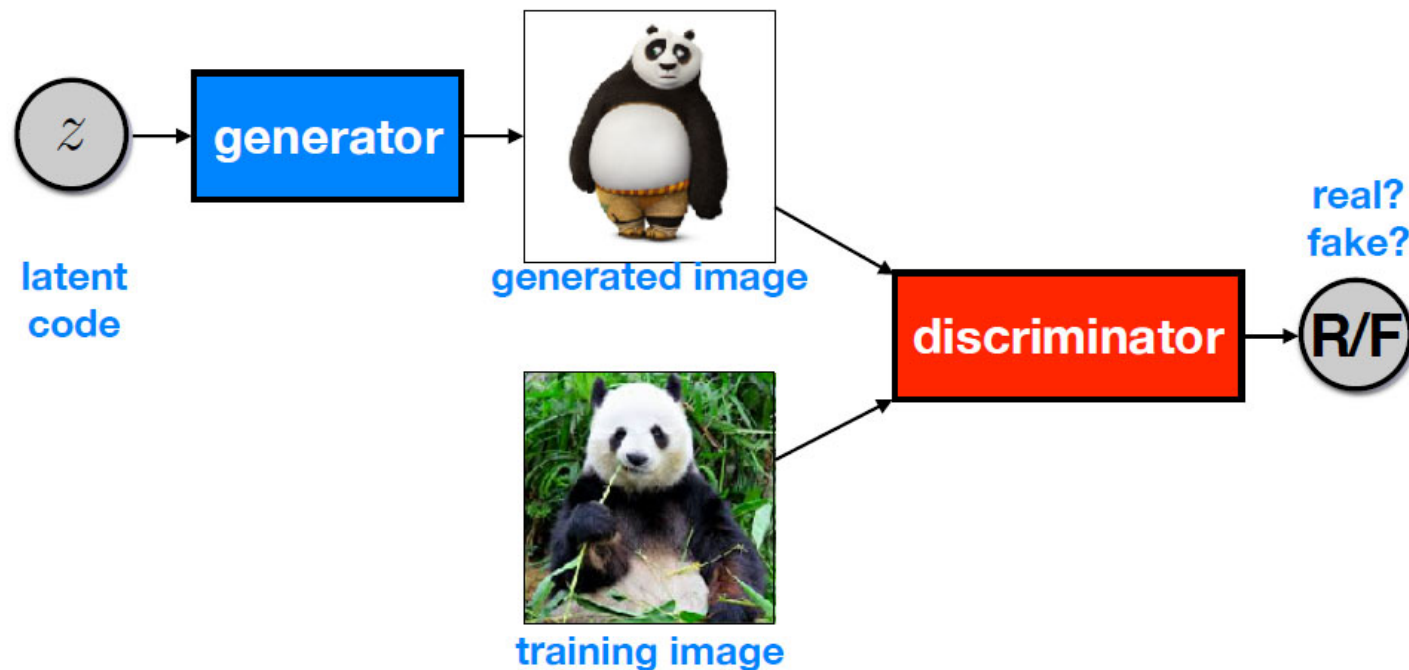
the gloves are made of
leather and have a
brown color.



Generative Adversarial Network

- Idea

- Generator** to convert a vector z (sampled from P_z) into fake data x (from P_G), while we need $P_G = P_{\text{data}}$
- Discriminator** classifies data as real or fake (1/0)
- How? Impose an **adversarial loss** on the observed data distribution!

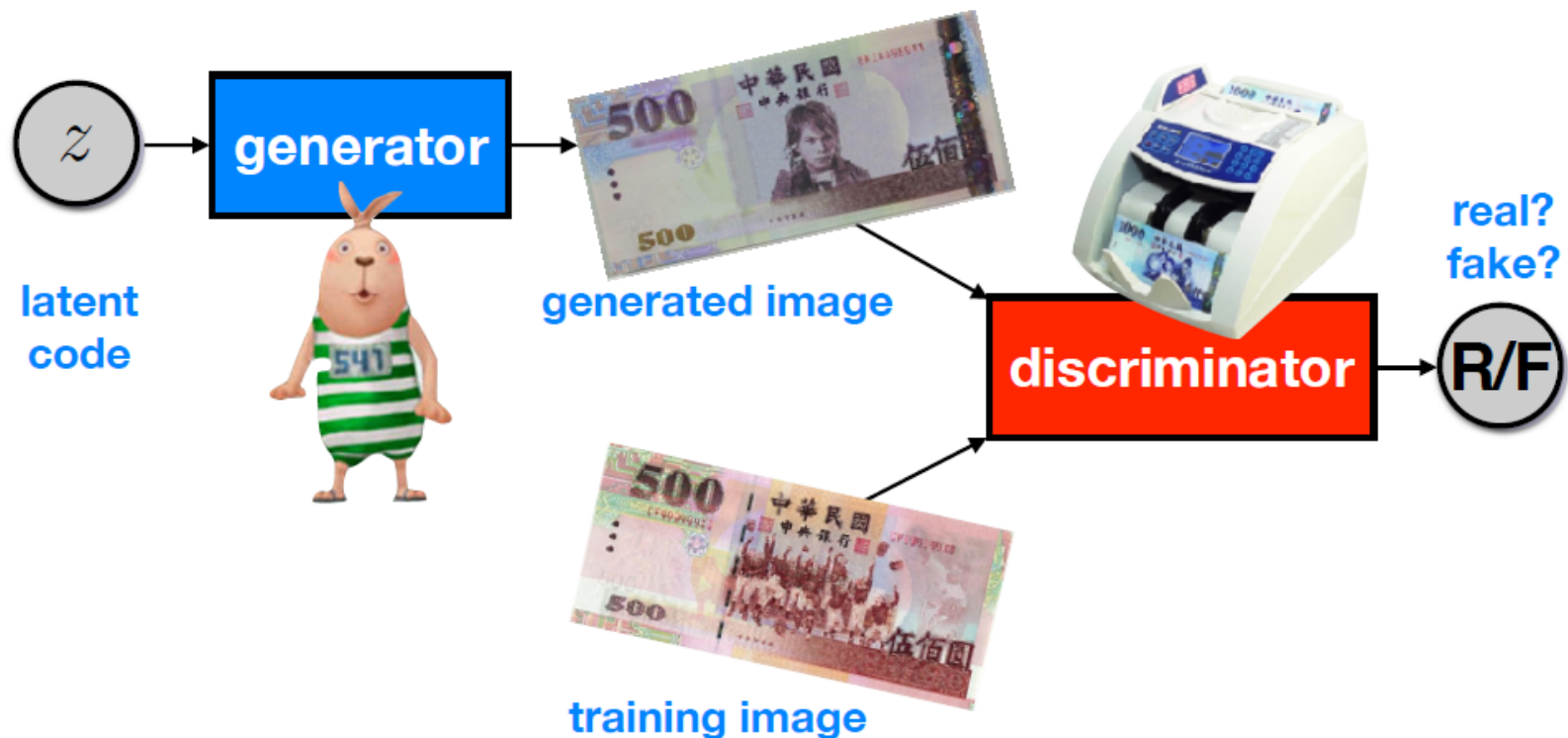


Generative Adversarial Network (cont'd)

- Key idea:
 - Impose *adversarial loss* on data distribution
 - Let's see a practical example...

generator: try to generate more realistic images to cheat discriminator

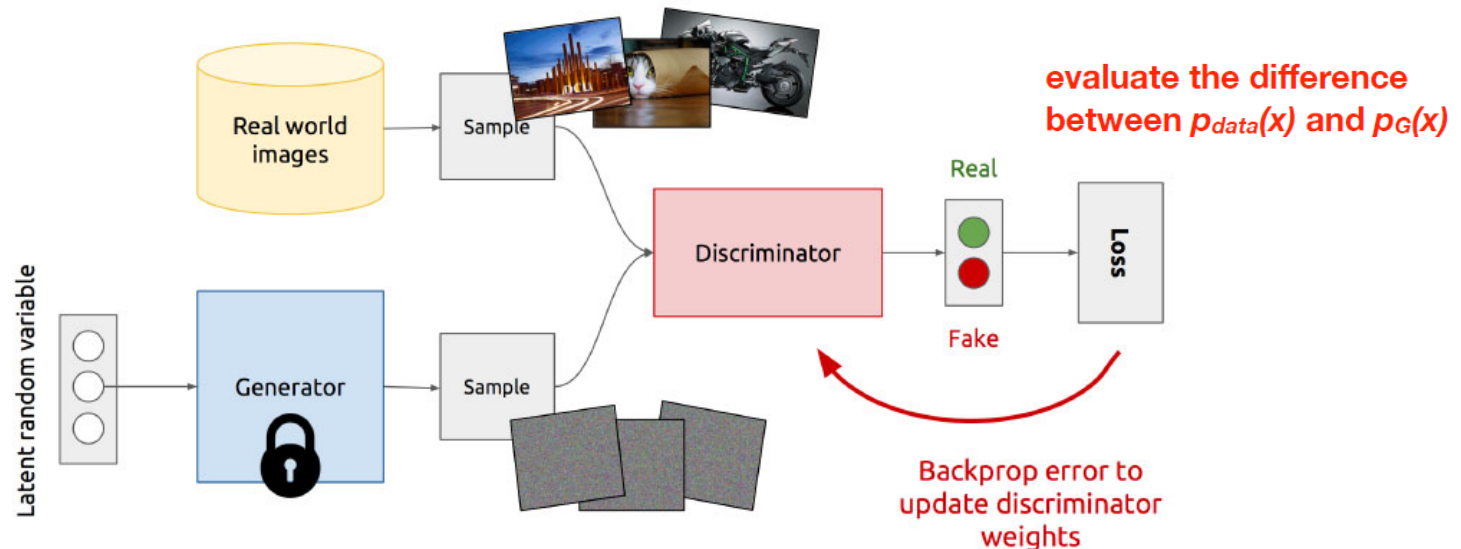
discriminator: try to distinguish whether the image is generated or real



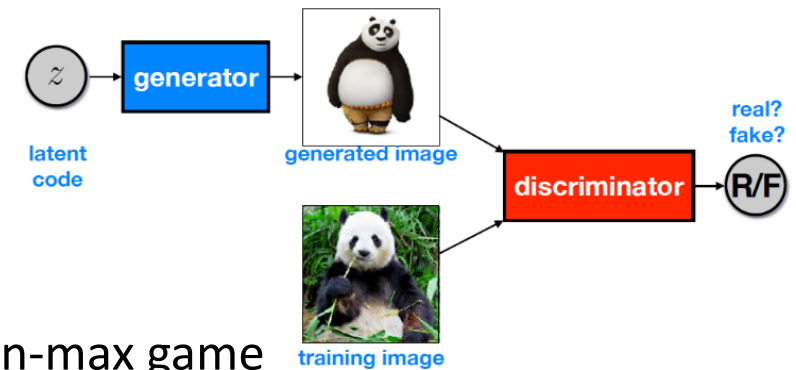
GAN (cont'd)

- Remarks
 - A function maps **normal distribution** $N(\mathbf{0}, I)$ to P_{data}
 - How good we are in mapping P_g to P_{data} ?
 - Train & ask the discriminator!
 - Conduct a two-player min-max game (see next slide for more details)

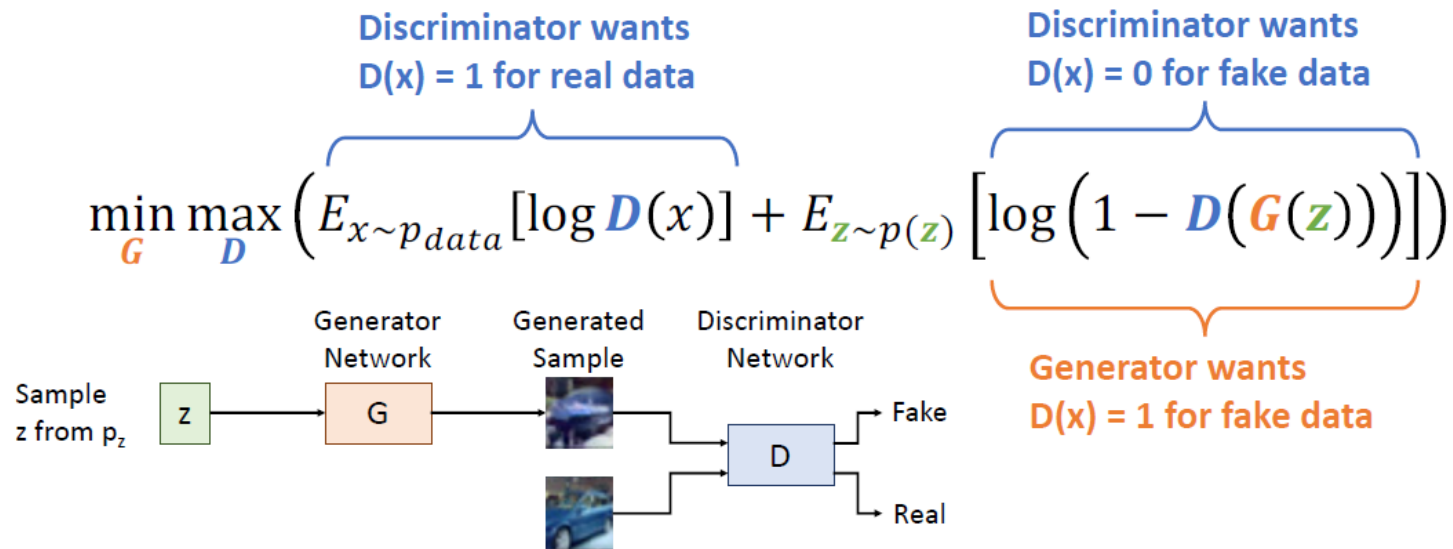
$$\min_G \max_D V(D, G) = \mathbb{E}_{\mathbf{x} \sim p_{data}(\mathbf{x})} [\log D(\mathbf{x})] + \mathbb{E}_{\mathbf{z} \sim p_z(\mathbf{z})} [\log(1 - D(G(\mathbf{z})))]$$



Training Objective of GAN



- Jointly train generator G and discriminator D with a min-max game



- Train G & D with alternating gradient updates

$$\min_G \max_D V(G, D)$$

For t in $1, \dots, T$:

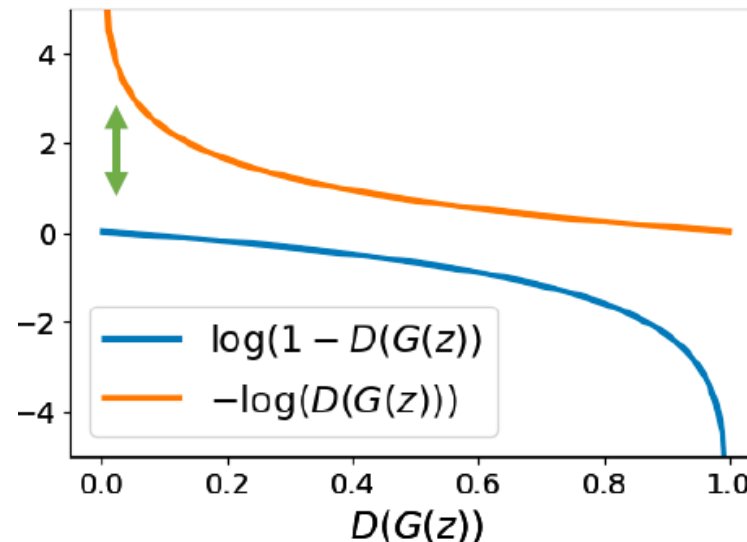
- (Update D) $D = D + \alpha_D \frac{\partial V}{\partial D}$
- (Update G) $G = G - \alpha_G \frac{\partial V}{\partial G}$

Training Objective of GAN (*optional trick*)

- Potential Problem

$$\min_G \max_D \left(E_{x \sim p_{data}} [\log D(x)] + E_{z \sim p(z)} [\log (1 - D(G(z)))] \right)$$

- At start of training, G is not OK yet (obviously);
D easily tells apart real/fake data (i.e., $D(G(z))$ close to 0).
- Possible Solution:
 - Instead of training G to minimize $\log(1 - D(G(z)))$ in the beginning, we train G to minimize $-\log(D(G(z)))$.
 - With strong gradients from G, we start the training of the above min-max game.



Optimality of GAN

- Why the min-max game as objective a good idea?

$$\begin{aligned}
 & \min_G \max_D \left(E_{x \sim p_{data}} [\log D(x)] + E_{z \sim p(z)} [\log (1 - D(G(z)))] \right) \\
 &= \min_G \max_D \left(E_{x \sim p_{data}} [\log D(x)] + E_{x \sim p_G} [\log (1 - D(x))] \right) \\
 &= \min_G \int_X \max_D (p_{data}(x) \log D(x) + p_G(x) \log (1 - D(x))) dx \\
 & \left. \begin{aligned} f(y) &= a \log y + b \log(1 - y) \\ f'(y) &= \frac{a}{y} - \frac{b}{1 - y} \end{aligned} \right\} f'(y) = 0 \Leftrightarrow y = \frac{a}{a+b} \text{ (local max)} \\
 & \text{Optimal Discriminator: } D_G^*(x) = \frac{p_{data}(x)}{p_{data}(x) + p_G(x)}
 \end{aligned}$$

Optimality of GAN

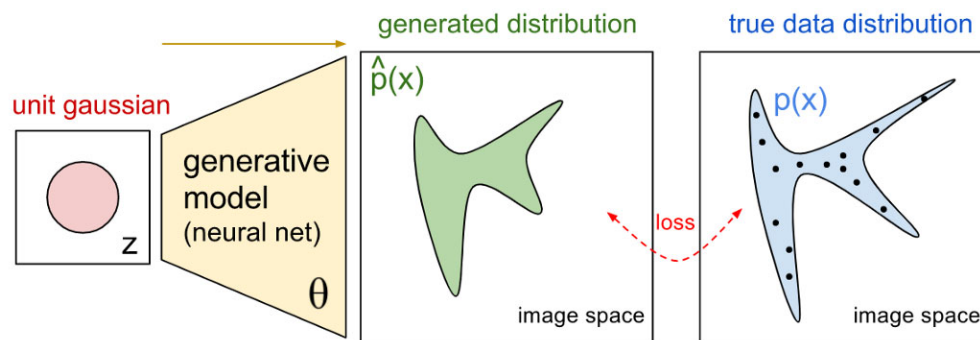
- Why the min-max game as objective a good idea? (cont'd)

$$\begin{aligned} & \min_G \max_D \left(E_{x \sim p_{data}} [\log D(x)] + E_{z \sim p(z)} \left[\log \left(1 - D(G(z)) \right) \right] \right) \\ \Rightarrow & \min_G \int_X \left(p_{data}(x) \log \frac{p_{data}(x)}{p_{data}(x) + p_G(x)} + p_G(x) \log \frac{p_G(x)}{p_{data}(x) + p_G(x)} \right) dx \\ & = \min_G \left(E_{x \sim p_{data}} \left[\log \frac{2}{2} \frac{p_{data}(x)}{p_{data}(x) + p_G(x)} \right] + E_{x \sim p_G} \left[\log \frac{2}{2} \frac{p_G(x)}{p_{data}(x) + p_G(x)} \right] \right) \\ & = \min_G \left(E_{x \sim p_{data}} \left[\log \frac{2 * p_{data}(x)}{p_{data}(x) + p_G(x)} \right] + E_{x \sim p_G} \left[\log \frac{2 * p_G(x)}{p_{data}(x) + p_G(x)} \right] - \log 4 \right) \end{aligned}$$

Optimality of GAN

- Why the min-max game as objective a good idea? (cont'd)

$$\begin{aligned}
 & \min_G \max_D \left(E_{x \sim p_{data}} [\log D(x)] + E_{z \sim p(z)} [\log (1 - D(G(z)))] \right) \\
 &= \min_G \left(E_{x \sim p_{data}} \left[\log \frac{2 * p_{data}(x)}{p_{data}(x) + p_G(x)} \right] + E_{x \sim p_G} \left[\log \frac{2 * p_G(x)}{p_{data}(x) + p_G(x)} \right] - \log 4 \right) \\
 &= \min_G \left(KL \left(p_{data}, \frac{p_{data} + p_G}{2} \right) + KL \left(p_G, \frac{p_{data} + p_G}{2} \right) - \log 4 \right)
 \end{aligned}$$



Kullback-Leibler Divergence:

$$KL(p, q) = E_{x \sim p} \left[\log \frac{p(x)}{q(x)} \right]$$

Optimality of GAN

- Why the min-max game as objective a good idea? (cont'd)

$$\begin{aligned}
 & \min_G \max_D \left(E_{x \sim p_{data}} [\log D(x)] + E_{z \sim p(z)} \left[\log \left(1 - D(G(z)) \right) \right] \right) \\
 &= \min_G \left(E_{x \sim p_{data}} \left[\log \frac{2 * p_{data}(x)}{p_{data}(x) + p_G(x)} \right] + E_{x \sim p_G} \left[\log \frac{2 * p_G(x)}{p_{data}(x) + p_G(x)} \right] - \log 4 \right) \\
 &= \min_G \left(KL \left(p_{data}, \frac{p_{data} + p_G}{2} \right) + KL \left(p_G, \frac{p_{data} + p_G}{2} \right) - \log 4 \right) \\
 &= \min_G (2 * JSD(p_{data}, p_G) - \log 4)
 \end{aligned}$$

JSD is always nonnegative, and zero only when the two distributions are equal!
 Thus $p_{data} = p_G$ is the global min, QED

Jensen-Shannon Divergence:

$$JSD(p, q) = \frac{1}{2} KL \left(p, \frac{p + q}{2} \right) + \frac{1}{2} KL \left(q, \frac{p + q}{2} \right)$$

Remarks on Optimality of GAN

$$\min_G \max_D \left(E_{x \sim p_{data}} [\log D(x)] + E_{z \sim p(z)} \left[\log \left(1 - D(G(z)) \right) \right] \right) \\ = \min_G (2 * JSD(p_{data}, p_G) - \log 4)$$

- Summary

- The global min of the minmax game happens when

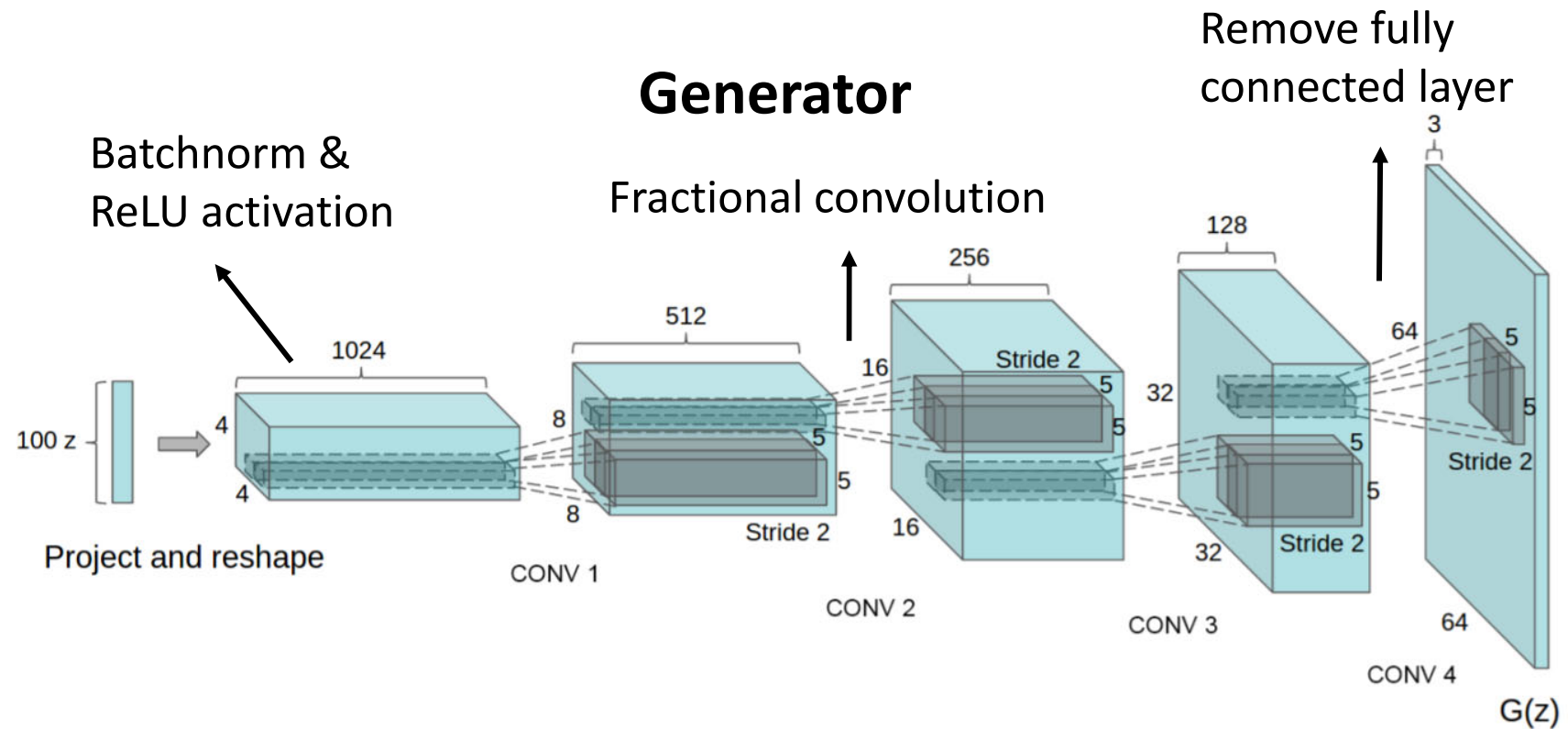
1. $D_G^*(x) = \frac{p_{data}(x)}{p_{data}(x) + p_G(x)}$ (Optimal discriminator for any G) ➡
2. $p_G(x) = p_{data}(x)$ (Optimal generator for optimal D) ➡

- **Caution!**

- G and D are learned models (i.e., DNNs) with fixed architectures.
We don't know whether we can actually represent the **optimal D & G**.
- Optimality of GAN does not tell anything about **convergence** to the optimal D/G.

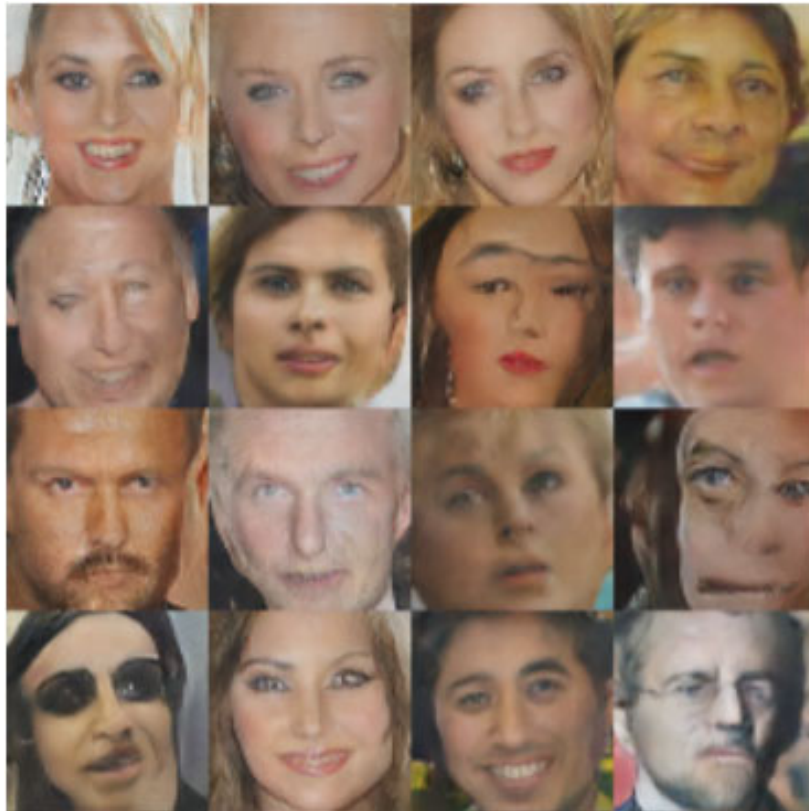
Deep Convolutional GAN (DC-GAN)

- Remarks
 - ICLR 2016
 - A CNN+GAN architecture
 - Empirically make training of GAN more stable

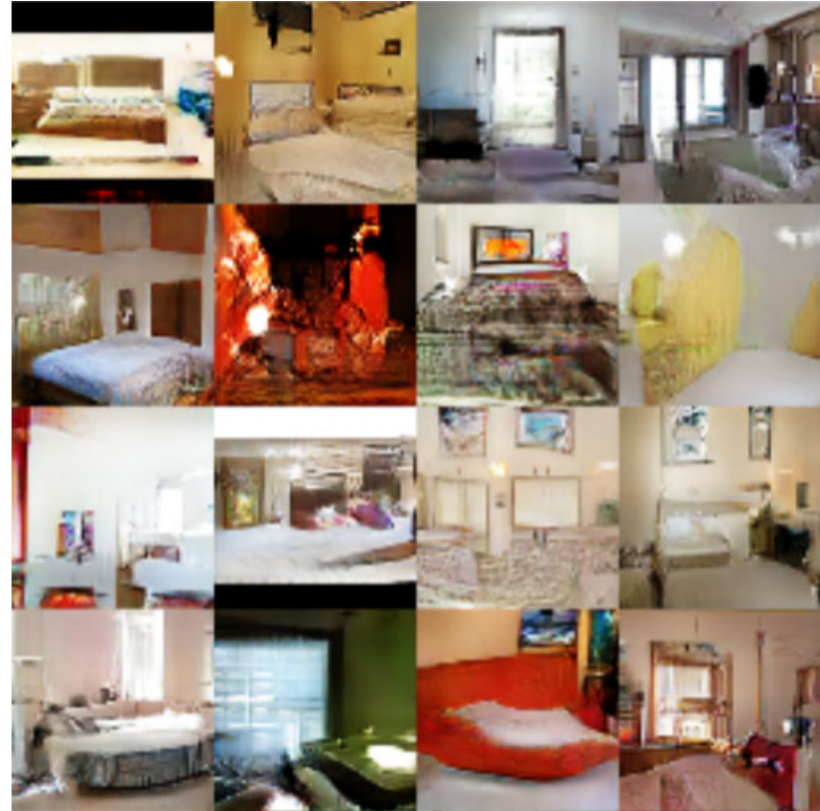


Deep Convolutional GAN (DC-GAN)

- Example Results



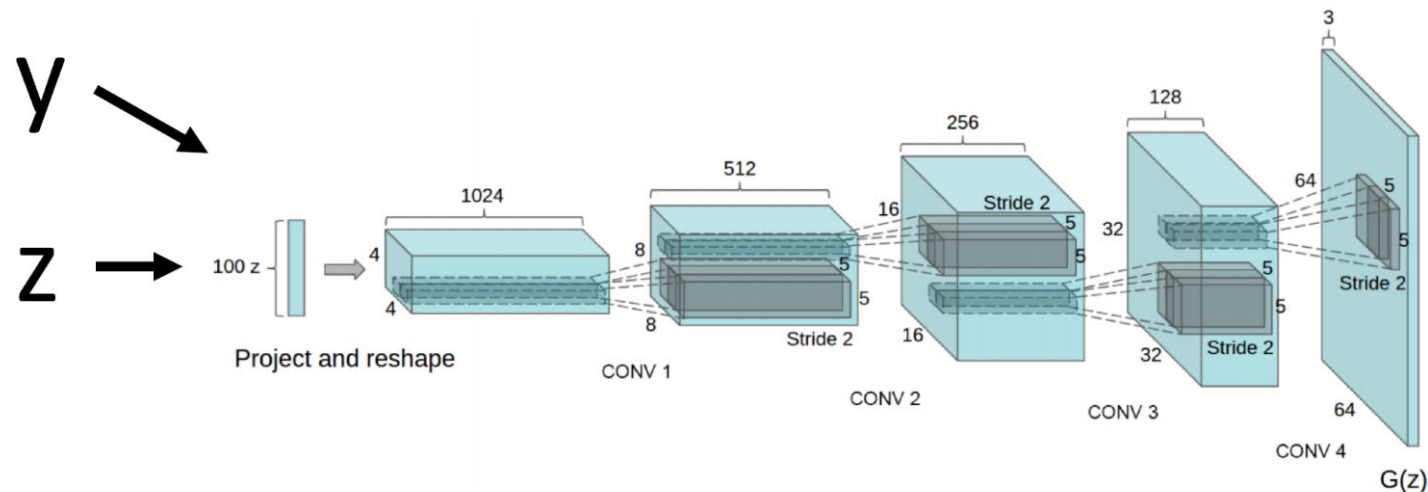
Collected face dataset



LSUN dataset

Conditional GANs

- Remarks
 - ICLR 2016
 - Conditional generative model $p(x|y)$ instead of $p(x)$
 - Both G and D take the label y as an additional input... i.e., a **conditional discriminator** is deployed....Why?
 - **Why not just use D as designed in the standard GAN?**



Conditional GANs

- Example Results

Welsh springer spaniel



Fire truck

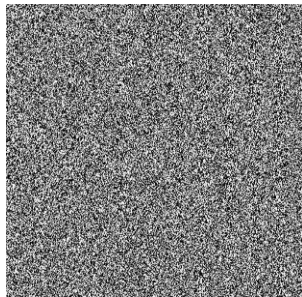


Daisy

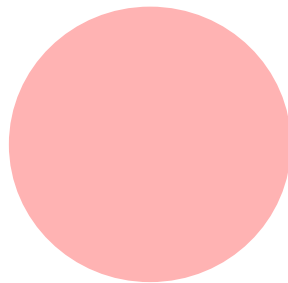


Problems in Training GANs: Vanishing Gradients

- What Might Go Wrong?
 - GAN training is often unstable.
 - In other words, training might not converge properly.
 - The discriminator which we prefer is...



0



0.2



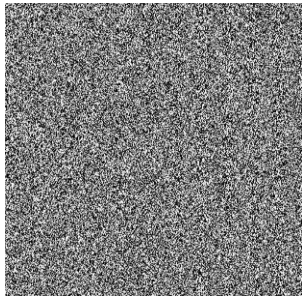
0.6



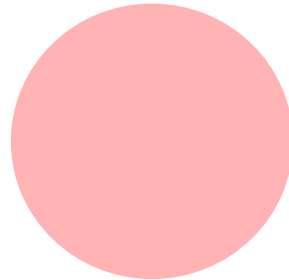
1

Problems in Training GANs: Vanishing Gradients (cont'd)

- What Might Go Wrong?
 - GAN training is often unstable.
 - In other words, training might not converge properly.
 - The discriminator we trained might be as follows.
In other words, no gradient to guide the generator to output proper images.



0



0



0



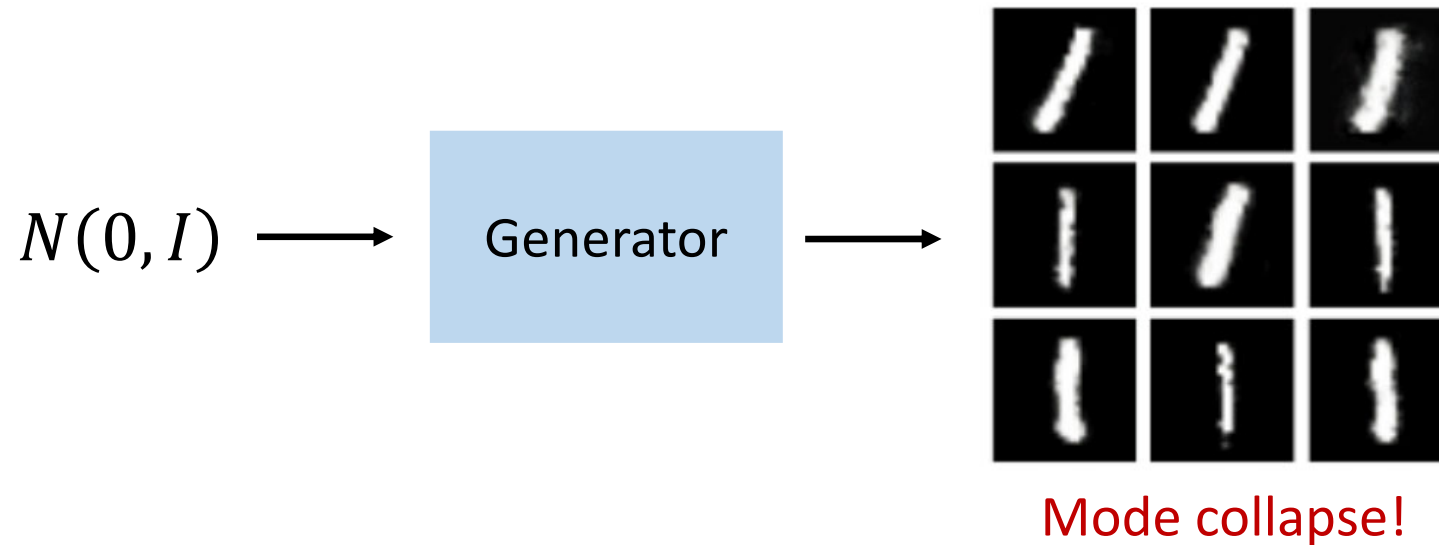
1

- This is known as the problem of *vanishing gradients*.

Problems in Training GANs:

Mode Collapse

- Remarks
 - The generator only outputs a limited number of image variants regardless of the inputs.



Problems in Training GANs: Mode Collapse (cont'd)

- Remarks
 - The generator only outputs a limited number of image variants regardless of the inputs.

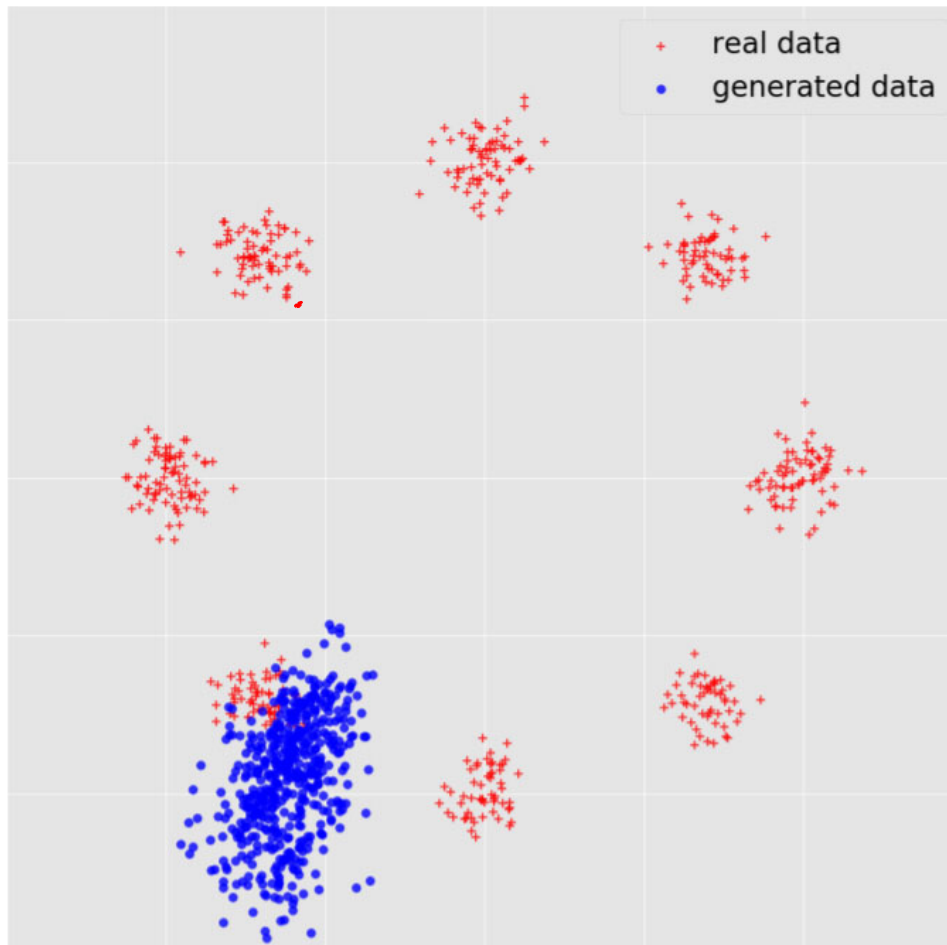
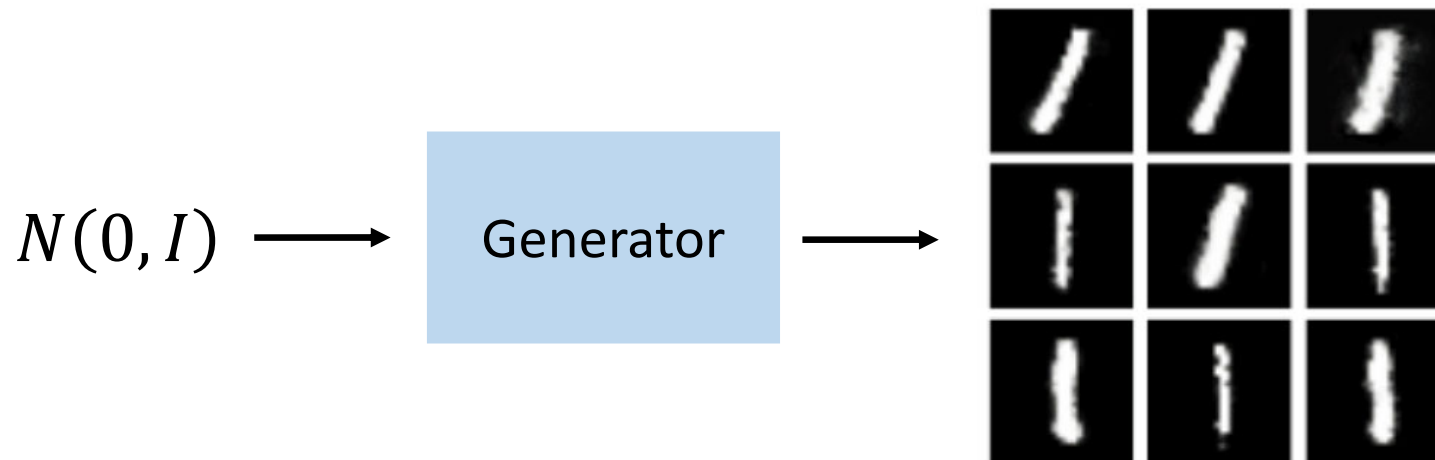


Photo credit:
<https://openreview.net/pdf?id=rkmu5b0a->

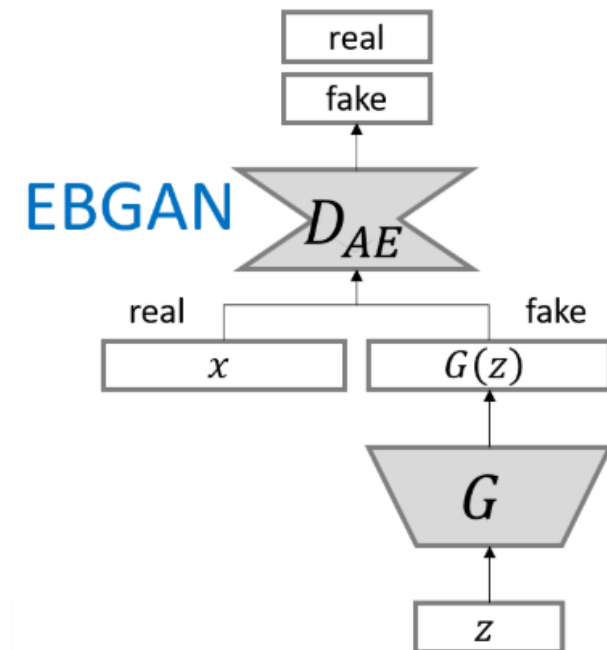
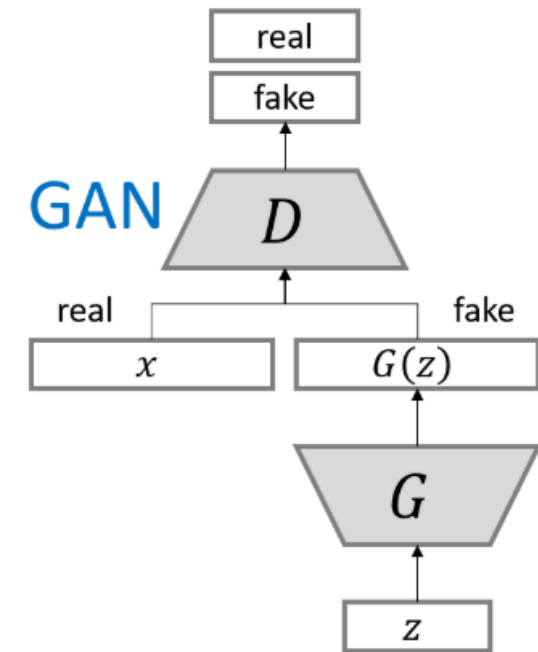
Problems in Training GANs: Mode Collapse (cont'd)

- Why Mode Collapse Happens?
 - The objective of GANs assesses the **image authenticity**, not **diversity**.
 - Imbalance training between generator/discriminator (exploding/vanishing gradients)



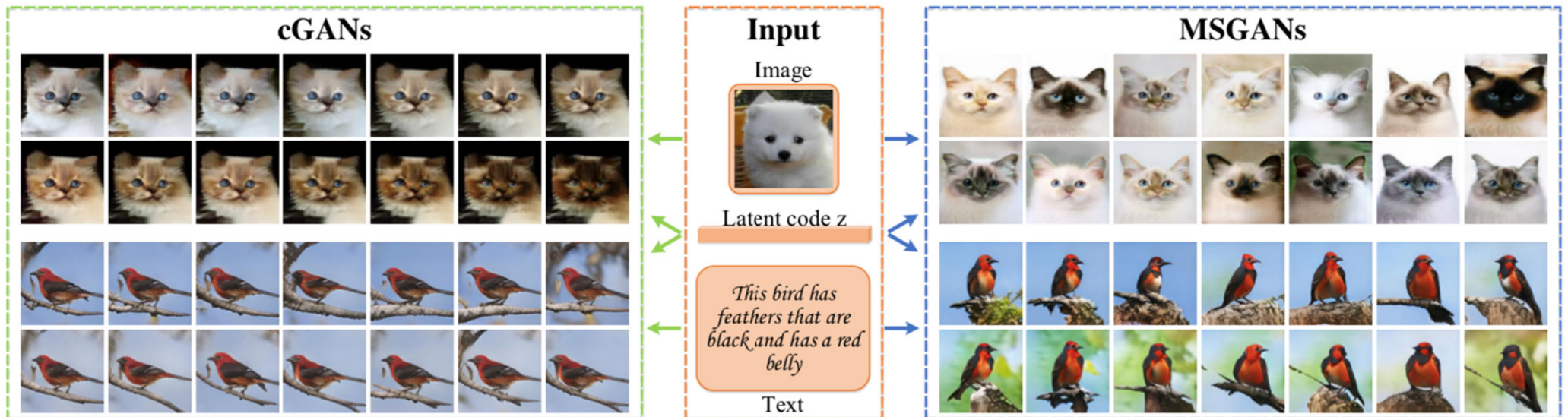
Energy-Based GAN

- Energy Function
 - Converting input data into scalar outputs, viewed as energy values
 - Desired configuration is expected to output low energy values & vice versa.
- Energy Function as Discriminator
 - Use of autoencoder; can be pre-trained!
 - Reconstruction loss outputs a range of values instead of binary logistic loss.
 - Empirically better convergence



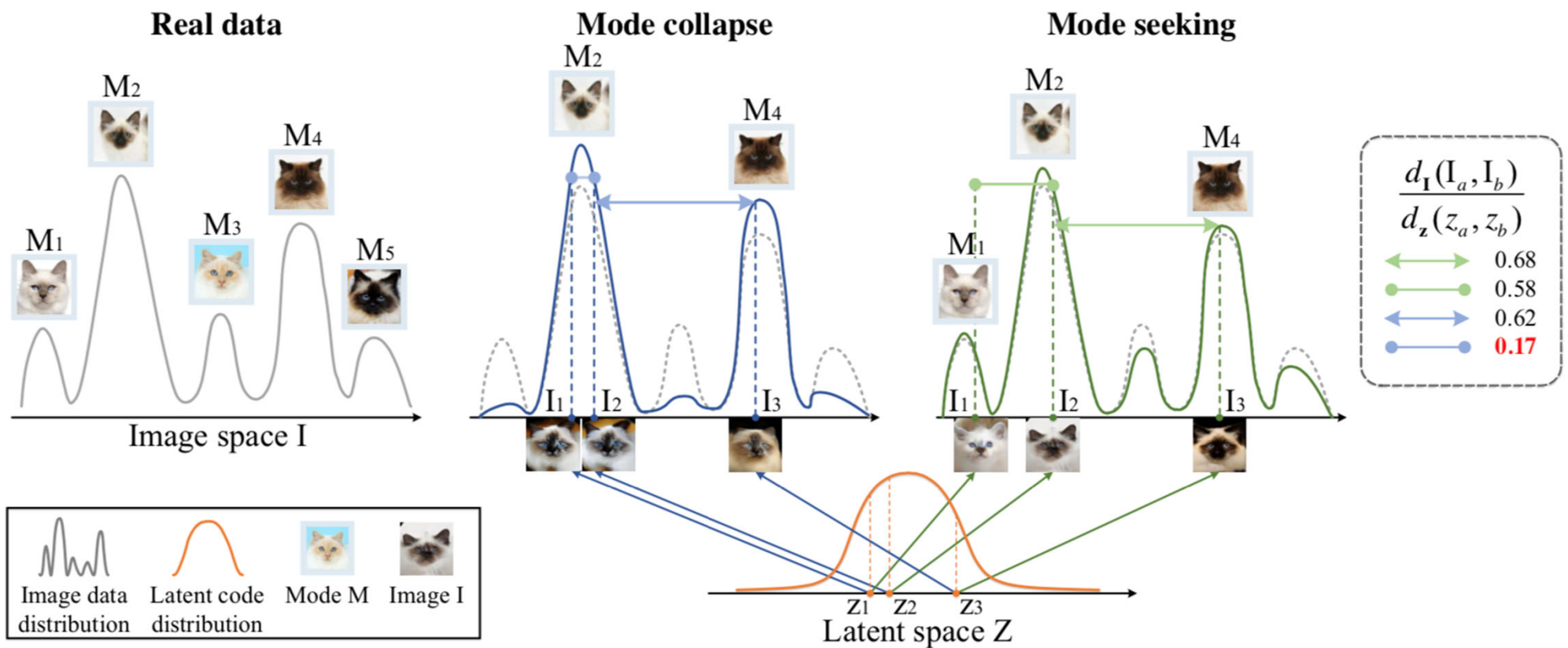
MSGAN

- To address the **mode collapse** issue by **conditional GANs**
- Mode Seeking Generative Adversarial Networks for Diverse Image Synthesis
- With the goal of producing **diverse** image outputs.



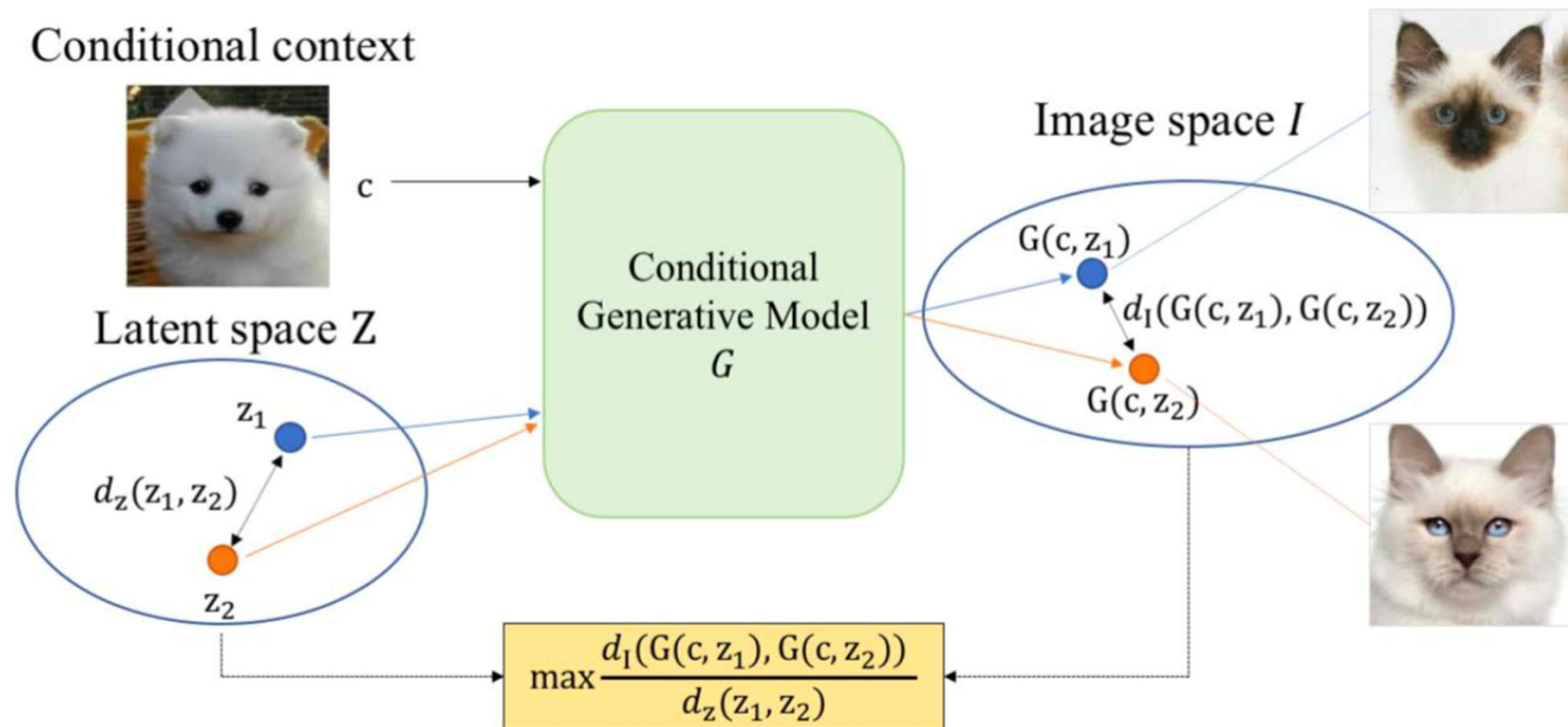
MSGAN (cont'd)

- Motivation (for unconditional GAN)



MSGAN (cont'd)

- Proposed Regularization (for conditional GAN)



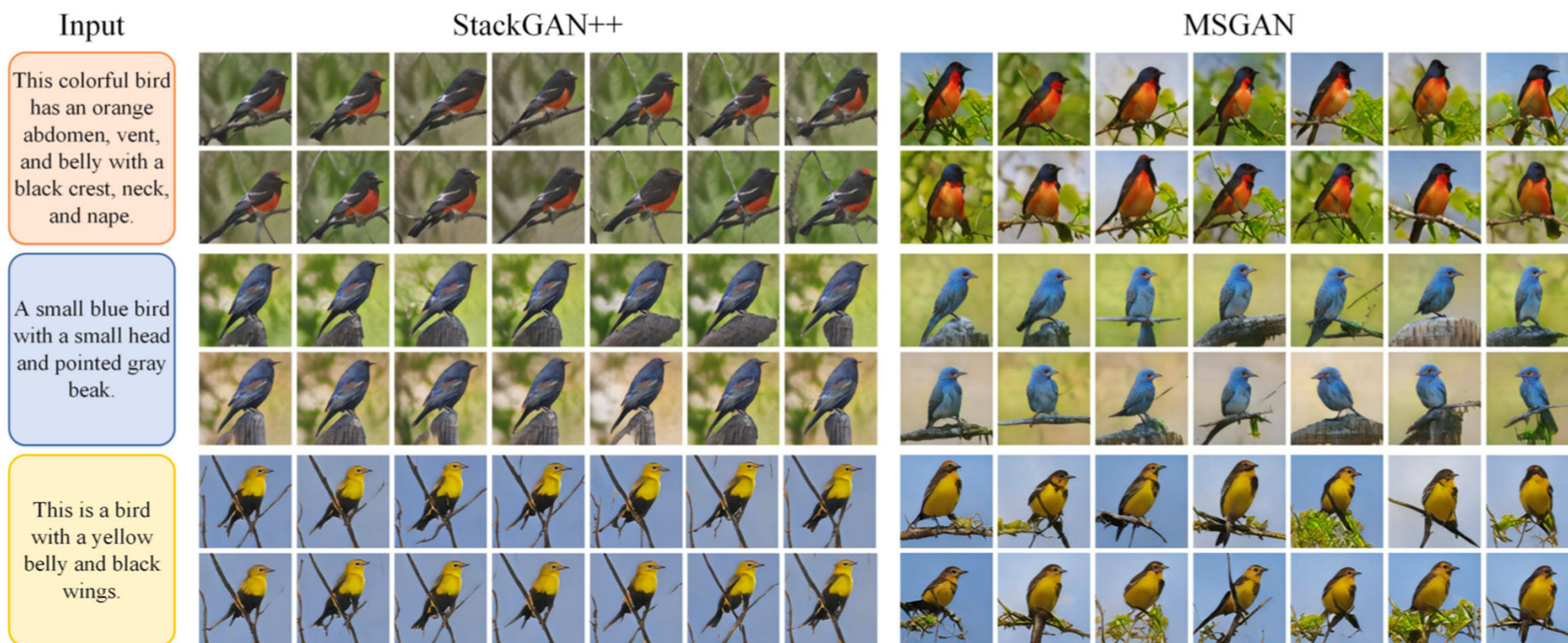
MSGAN (cont'd)

- Qualitative results
 - Conditioned on [paired images](#)



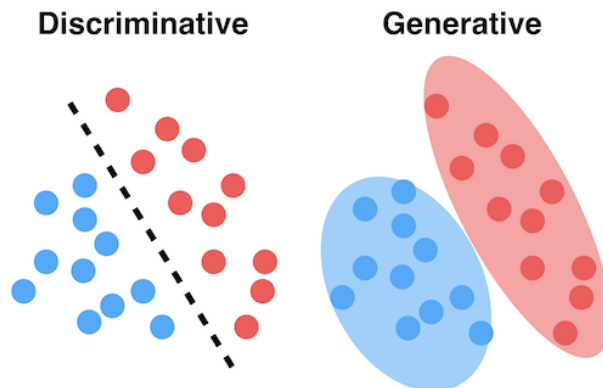
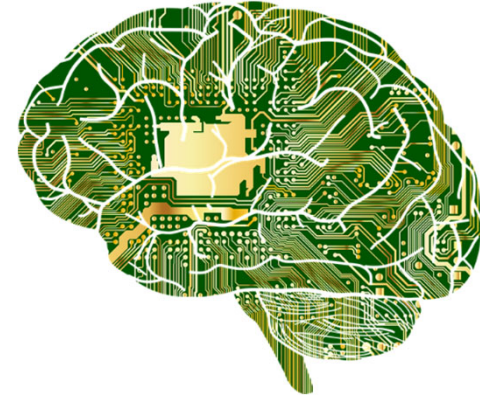
MSGAN (cont'd)

- Qualitative results
 - Conditioned on **text** (will talk about Vision & Language later this semester)

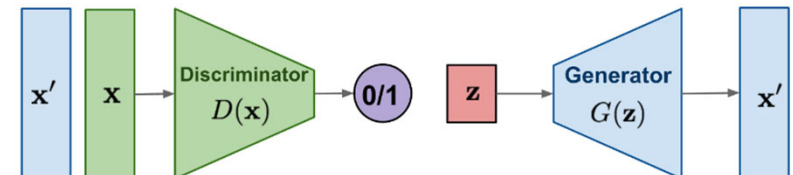


What We've Covered Today...

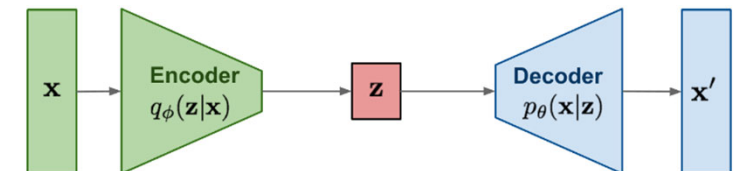
- Generative Models
 - Diffusion Model
 - Conditional Diffusion Model
 - Classifier Guidance
 - Classifier-Free Guidance
 - Text/Image Guidance
 - Personalization via Diffusion Model
 - Generative Adversarial Network
 - HW #2 is out & due in 3 weeks (Oct. 22nd)



GAN: Adversarial training



VAE: maximize variational lower bound



Diffusion models:
Gradually add Gaussian noise and then reverse

